

Le query

GSTRUMSVIL - Guida allo sviluppo in BC - © SISTEMI S.p.A.

- [Le query in SQL](#)
- [Le query in BC](#)
- [Le subquery in BC](#)
- [Le common table expressions \(CTEs\) in BC](#)
- [Analisi e ottimizzazione delle query](#)
- [Esercitazioni di approfondimento \(4\)](#)
- [Operazioni su DataBase](#)
- [Legacy](#)

Le query in SQL

Il linguaggio BC e l'ambiente di sviluppo BUILDER/Sistemi "inglobano" il linguaggio di interrogazione SQL.

L'ambiente di esecuzione BC può operare su archivi BTRIEVE e su tabelle di database SQL. L'utilizzo dei due gestori dati è del tutto trasparente. Ad esempio, l'esecuzione dell'istruzione '@FINDDBDATA' per leggere un record è controllata dalle librerie d'ambiente, le quali eseguono l'operazione corretta in funzione del gestore della base dati in uso in quel momento.

Le funzioni per la gestione delle query consentono di operare sui dati in maniera più efficiente, ovviamente solo per SQL-Server perché non disponibili per Pervasive.

Come di seguito viene descritto in maggior dettaglio, le query permettono di estrarre i dati dal database in maniera molto efficiente: con un'unica richiesta recuperano dati aggregandoli da tabelle diverse e con condizioni di selezione complesse.

Pertanto quello che prima richiedeva la stesura di un programma BC di interrogazione delle tabelle btrieve può essere ottenuto in maniera più semplice ed efficiente mediante una query.

Per sfruttare appieno tali potenzialità è necessario adeguarsi ad alcuni concetti impiegati nella definizione delle tabelle come anche nella stesura dei programmi.

In particolare è necessaria una maggiore attenzione alle prestazioni perché fino ad ora i programmi BC e le specifiche di accesso ai dati hanno preso come riferimento il modello di gestione dei dati definito da pervasive che è profondamente diverso dal modello di accesso SQL.

⚠ Introdurre le query su programmi che possono essere eseguiti sia con un DBMS Pervasive che SQL-Server, richiede la definizione di due programmi diversi; perlomeno devono essere definiti specifici processi di lettura dei dati che devono essere eseguiti per lo specifico DBMS in uso.

Alcune differenze tra le due gestioni degli dati:

Btrieve	SQL
Le tabelle in Btrieve sono rappresentate mediante file fisici, di cui il programmatore deve conoscere la collocazione.	Le tabelle in un database relazionale sono entità logiche, la posizione "fisica" delle tabelle è demandata al motore relazionale e al suo amministratore.
La definizione di un file BTRIEVE è statica, in particolare gli indici di ordinamento vanno definiti a priori.	La definizione di una tabella è modificabile nel tempo, gli indici di ordinamento sono definibili al momento dell'interrogazione della base dati.
L'accesso al dato avviene record per record, una scansione di record comporta una lettura sequenziale.	L'accesso al dato avviene in base al recordset, ovvero ad un insieme di record.
È possibile trattare una sola tabella alla volta.	Mediante le query posso accedere ai dati mettendo insieme i dati di più tabelle e produrre una "tabella" virtuale.

Una query è una richiesta di dati memorizzati in un database. La richiesta definisce le regole per recuperare i dati dal server SQL. Il server analizza la richiesta e restituisce i dati. I dati di risposta sono organizzati in forma di tabella, equivalente ad una tabella definita sul database. Tale tabella di result è indicata con il nome di "result set" oppure "record set".

La sintassi di una query può essere molto complessa, ma la maggior parte di esse è costituita dalle seguenti parti:

- il numero e le colonne che costituiscono il result set. L'indicazione dei campi fornisce inoltre informazioni supplementari e fondamentali, quali il tipo della colonna, la dimensione della colonna (e per le colonne numeriche anche il numero di decimali, ...);
- le tabelle da cui recuperare i dati e le relazioni logiche tra di esse;
- le condizioni che le righe devono soddisfare per essere comprese nel result set;
- l'ordine delle righe nel result set.

Elementi di progettazione delle query

Query semplici

Una volta che abbiamo memorizzato i dati nel database, possiamo impiegare il linguaggio SQL per ricavare altri fatti non direttamente memorizzati nelle tabelle, ma che sono ottenibili logicamente da essi. Prendiamo in esame le seguenti tabelle:

Autori	Libri
Codice Fiscale (PK)	ISBN (PK)
Cognome	Titolo
Nome	Genere
	RagioneSocialeEditore
Editore	Libri Autori
RagioneSocialeEditore (PK)	ISBN (PFK)
Telefono Editore	Codice Fiscale (PFK)
SitoWebEditore	
Indirizzo Editore	
CapEditore	
Città Editore	
Libri Libreria	Libreria
RagioneSocialeLibreria (PFK)	RagioneSocialeLibreria (PK)
ISBN (PFK)	Telefono Libreria
NumeroCopie	Indirizzo Libreria
	CapLibreria
	Città Libreria

Le informazioni che vogliamo ottenere sono:

"Elenco in ordine di numero di copie dei numeri telefonici delle librerie in cui ci sono almeno 5 copie di manuali di informatica pubblicati dall'editore "MacGraw Hill"

Proviamo a rappresentare questa richiesta in forma tabellare, individuando i campi che occorrono nella risposta ed i campi su cui devono essere effettuati i filtri:

Tabella	Campo	In uscita	Filtro
Libreria	RagioneSocialeLibreria	Si	
Libreria	Telefono Libreria	Si	
Libri	Genere		=Manuali di informatica
Libri Libreria	Numero Copie	Si	>=5
Editore	RagioneSocialeEditore		="MacGraw Hill"

In questo primo passo abbiamo determinato i campi e le tabelle coinvolte e, implicitamente, dallo schema si ricavano le relazioni che esistono tra le tabelle (tutte le tabelle coinvolte devono essere in relazione tra loro). In caso contrario si aggiungono le tabelle mancanti per permettere di raggiungere le tabelle non in relazione tra di loro.

Nella progettazione della query è quindi di fondamentale importanza conoscere le relazioni che esistono tra le tabelle.

Una volta che si ha a disposizione tutti gli elementi per costruire la query:

- Campi (di uscita);
- Tabelle e le relazioni tra tabelle (che ricavo dalla definizione della base dati);
- Condizioni (i campi di filtro).

Queste informazioni vanno poi collocate nel posto giusto previsto dalla sintassi SQL per l'interrogazione, la forma tipica dell'istruzione di interrogazione è:

```

SELECT <campi di uscita>
FROM <tabella> JOIN <tabelle in relazione>
WHERE <condizioni>

--Traducendo in SQL la tabella precedente ottengo la seguente istruzione:
SELECT Libreria.RagioneSocialeLibreria, Libreria.TelefonoLibreria
FROM Libreria
JOIN LibriLibreria ON
      ( LibriLibreria.RagioneSocialeLibreria = Libreria.RagioneSocialeLibreria)
JOIN Libri ON ( Libri.ISBN = Libri Libreria.ISBN)
JOIN Editore ON Editore.RagioneSocialeEditore = Libri.RagioneSocialeEditore

WHERE Libri.Genere='Manuali di informatica'
AND Libri Libreria.NumeroCopie >= 5

```

AND Editore.RagioneSocialeEditore = "MacGraw Hill"

ORDER BY Libreria.NumeroCopie

Tipologie di relazione tra tabelle

Le relazioni tra tabelle sono definite nella fase di disegno del database, la relazione esprime il legame tra due tabelle e possiamo identificare le seguenti proprietà:

- due nomi che descrivono, partendo dalle rispettive tabelle, la relazione dal punto di vista semantico;
- la condizione logica che mette a confronto un campo della prima tabella con un campo della seconda tabella e così per tutti i campi della relazione;
- un elenco di campi aggiuntivi che completano le informazioni di pertinenza della relazione.

Facciamo un esempio con due tabelle:

Libri	Editore
ISBN (PK)	RagioneSocialeEditore (PK)
Titolo	Telefono Editore
Genere	SitoWebEditore
RagioneSocialeEditore (FK)	Indirizzo Editore
	CapEditore
	Città Editore

Tra la tabella Libri e la tabella Editore esiste una relazione sul campo RagioneSocialeEditore e tale relazione è utilizzata all'interno della clausola JOIN della query:

```
SELECT <campi da estrarre>
FROM Libri
JOIN Editore ON Libri.RagioneSocialeEditore = Editore.RagioneSocialeEditore
WHERE <condizione>
```

In questo modo il record-set è limitato alle sole righe che soddisfano la condizione della relazione.

Ad esempio, immaginando di avere i seguenti dati nelle tabelle:

Libri:

ISBN	Titolo	Genere	RagioneSocialeEditore
112-211212-12121	Il signore degli anelli	Fantasy	Mondadori
11221122445522-3322	Il portiere di calcio a 5	Manuale calcio	Bompiani
999801212-12-12121	C#	Manuale informatica	MacGraw Hill
12987232-7842729-33	Barnum	Romanzo	Mondadori
18878234-2344234-22	Donne informate sui fatti	Romanzo	Mondadori
92392348923-1234231	I pilastri della terra	Romanzo	Rizzoli

Editore:

RagioneSocialeEditore	Telefono Editore	SitoWebEditore	Indirizzo Editore	CapEditore	CittàEditore
Mondadori	02-221122	www.mondadori.it	Via Montenapoleone 3	20100	Milano
Bompiani	02-3232	www.bompiani.it	C.so Trieste 2	20100	Milano
Einaudi	011-20021	www.einaudi.it	C.so Vittorio 43	10100	Torino

Se si esegue la seguente query:

```
SELECT Libri.Titolo , Editore.SitoWebEditore
FROM Libri
JOIN Editore ON Libri.RagioneSocialeEditore = Editore.RagioneSocialeEditore
```

si ottiene il seguente record-set:

Titolo	SitoWebEditore
Il signore degli anelli	www.mondadori.it
Il portiere di calcio a 5	www.bompiani.it
Barnum	www.mondadori.it
Donne informate sui fatti	www.mondadori.it

Nella tabella non si trova né il libro "C#", in quanto l'editore "MacGraw Hill" non è presente nella tabella EDITORE, né il sito dell'editore "Einaudi" in quanto nella tabella Libri non ci sono sue pubblicazioni.

È possibile modificare il comportamento della relazione, ad esempio se occorre l'elenco completo dei libri, a prescindere dalla presenza dell'editore, si può scrivere la seguente istruzione SQL:

```
SELECT Libri.Titolo, Editore.SitoWebEditore
FROM Libri
LEFT OUTER JOIN Editore ON Libri.RagioneSociale = Editore.RagioneSocialeEditore
```

La particolarità dell'istruzione SQL è l'utilizzo di LEFT OUTER JOIN al posto di JOIN. Il termine LEFT indica che la tabella di cui si vogliono tutte le righe è la tabella di sinistra della relazione, ovvero la tabella Libri.

L'esecuzione dell'istruzione precedente produce il seguente result-set:

Titolo	SitoWebEditore
Il signore degli anelli	www.mondadori.it
Il portiere di calcio a 5	www.bompiani.it
C#	NULL
Barnum	www.mondadori.it
Donne informate sui fatti	www.mondadori.it
I pilastri della terra	NULL

Come si vede nell'esempio, l'istruzione produce una tabella dove per la colonna SitoWebEditore, compare il valore "NULL" in corrispondenza delle righe che hanno un libro di un editore non presente nella tabella Editore. Questo valore esprime in questo caso il fatto che il dato non è noto.

La sua presenza può complicare la definizione di eventuali filtri.

Se viceversa interessano i siti web a prescindere dai libri pubblicati, posso impiegare l'istruzione SQL:

```
SELECT Libri.Titolo, Editore.SitoWebEditore
FROM Libri
RIGHT OUTER JOIN Editore ON Libri.RagioneSociale = Editore.RagioneSocialeEditore
```

L'esecuzione dell'istruzione precedente produce il seguente result-set:

Titolo	SitoWebEditore
Il signore degli anelli	www.mondadori.it
Il portiere di calcio a 5	www.bompiani.it
Barnum	www.mondadori.it
Donne informate sui fatti	www.mondadori.it
NULL	www.einaudi.it

È possibile anche ottenere sia i libri sia i siti web a prescindere dalla relazione, usando l'istruzione SQL:

```
SELECT Libri.Titolo , Editore.SitoWebEditore
FROM Libri
FULL OUTER JOIN Editore ON Libri.RagioneSociale = Editore.RagioneSocialeEditore
```

In questo caso l'esecuzione dell'istruzione precedente il seguente risultato:

Titolo	SitoWebEditore
Il signore degli anelli	www.mondadori.it
Il portiere di calcio a 5	www.bompiani.it
C#	NULL
Barnum	www.mondadori.it
Donne informate sui fatti	www.mondadori.it
I pilastri della terra	NULL
NULL	www.einaudi.it

Da notare che l'ordine in cui sono state scritte le righe del result-set delle diverse interrogazioni è casuale in quanto la relazione non impone alcun ordine particolare.

Esiste un'ultima possibilità di incrociare i dati di due tabelle e prende il nome in SQL di "cross join". È raro l'impiego di tale relazione ed inoltre andrebbe impiegato su tabelle con poche righe in quanto la tabella prodotta dall'applicazione di tale operazione è il prodotto del numero di righe delle due tabelle. Con due tabelle di 500 e 1500 righe rispettivamente, applicando l'operazione di cross join si ottiene una tabella di 750.000 righe!

L'operazione di CROSS JOIN rappresenta l'equivalente SQL dell'operazione su insiemi detta prodotto cartesiano. In poche parole, ogni riga della prima tabella è messo in relazione con tutte le righe della seconda tabella:

a1	CROSS JOIN	b1	=	a1	b1
a2		b2		a1	b2
				a2	b1
				a2	b2

L'istruzione SQL per effettuare l'operazione SQL è la seguente:

```
SELECT Tabella1.*, Tabella2.* FROM Tabella1 CROSS JOIN Tabella2
```

Un possibile impiego pratico della operazione di CROSS JOIN si ha quando le due tabelle rappresentano i possibili valori ammessi per una terza tabella. Supponiamo ad esempio di avere la tabella Ruolo (Nome Ruolo) e Permessi (Nome Permesso).

Nel caso si voglia popolare la tabella PermessiAssegnati (Nome Ruolo, Nome Permesso, Assegnato) con i possibili ruoli e permessi, posso utilizzare la seguente istruzione:

```
SELECT Ruolo. nome Ruolo, Permessi.NomePermesso FROM Ruolo CROSS JOIN Permessi
```

Il CROSS JOIN è la base per costruire tabelle PIVOT dove queste non fossero disponibili.

Esempio - date la seguenti tabelle:

Mesi:

NomeMese
Gennaio
Febbraio
Marzo
...

Prodotti:

NomeProdotto
Pasta
Caffè
Latte
Frutta
...

Movimenti Vendita:

IdMovimento	Nome Mese	Nome Prodotto	ImportoVenduto
1	Gennaio	Pasta	10 €
2	Gennaio	Pasta	20 €
3	Febbraio	Latte	15 €
4	Marzo	Frutta	40 €
...	...	...	...

Posso costruire la base per popolare una tabella pivot usando la seguente istruzione:

```
SELECT Mesi.NomeMese, Prodotti.NomeProdotto, SUM (MovimentiVendita.ImportoVenduto)
FROM Mesi
CROSS JOIN Prodotti
LEFT OUTER JOIN MovimentiVendita ON Mesi.NomeMese = MovimentiVendita.NomeMese AND
Prodotti.NomeProdotto = MovimentiVendita.NomeProdotto.
GROUP BY Mesi.NomeMese, Prodotti.NomeProdotto
```

Il risultato della query:

Nome Mese	Nome Prodotto	Importo Venduto
Gennaio	Pasta	30 €
Gennaio	Caffè	NULL
Gennaio	Latte	NULL
Gennaio	Frutta	NULL
Febbraio	Pasta	NULL
Febbraio	Caffè	NULL
Febbraio	Latte	15 €
Febbraio	Frutta	NULL
Marzo	Pasta	NULL
Marzo	Caffè	NULL
Marzo	Latte	NULL
Marzo	Frutta	40 €
...	...	...

Nella tabella, le prime due colonne rappresentano le due coordinate (righe/colonne) della tabella pivot e la terza colonna rappresenta il valore della cella. In BC è possibile costruire una griglia in maniera dinamica.

Riepilogo delle diverse tipologie di relazione:

Tipo di relazione (istruzione SQL)	Significato
(INNER) JOIN	Solo le righe che soddisfano la relazione
LEFT OUTER JOIN	Le righe che soddisfano la relazione + le righe della tabella di sinistra che non soddisfano la relazione. I campi della tabella risultato relativi a campi della tabella di destra sono valorizzati a NULL per le righe della tabella di sinistra che non soddisfano la relazione.
RIGHT OUTER JOIN	Le righe che soddisfano la relazione + le righe della tabella di destra che non soddisfano la relazione. I campi della tabella risultato relativi a campi della tabella di sinistra sono valorizzati a NULL per le righe della tabella di destra che non soddisfano la relazione.
FULL OUTER JOIN	Le righe che soddisfano la relazione + le righe della tabella di sinistra che non soddisfano la relazione + le righe della tabella di destra che non soddisfano la relazione. I campi della tabella risultato relativi a campi della tabella di destra sono valorizzati a NULL per le righe della tabella di sinistra che non soddisfano la relazione I campi della tabella risultato relativi a campi della tabella di sinistra sono valorizzati a NULL per le righe della tabella di destra che non soddisfano la relazione.
CROSS JOIN	Prodotto cartesiano

Condizioni di ricerca

Le condizioni di ricerca esprimono le condizioni logiche che le righe estratte dalla tabella devono soddisfare. In SQL la condizione è posta dopo la clausola WHERE e la sintassi completa è molto articolata.
E' possibile impiegare espressioni come operandi nelle operazioni di confronto (es.: Conto.Dare-Conto.Avere > 10), l'utilizzo di operatori logici per unire più condizioni (AND, OR, NOT) oppure di operatori di confronto, ecc.
Alcune delle funzioni particolari che possono essere impiegate per definire una condizione sono:

Funzione	Descrizione
campo [NOT] LIKE '%valore%'	Ritorna TRUE se il valore del campo contiene 'valore'
campo [NOT] LIKE 'valore%'	Ritorna TRUE se il valore del campo inizia per 'valore'
campo [NOT] LIKE '%valore'	Ritorna TRUE se il valore del campo termina per 'valore'
campo BETWEEN valore1 AND valore2	Ritorna TRUE se il valore del campo è compreso nell'intervallo valore1...valore2
campo IN (elenco valori)	Ritorna TRUE se il valore del campo è compreso nell'elenco di valori

È importante stabilire se una determinata richiesta può essere soddisfatta mediante la definizione di una query oppure se è necessario scrivere del codice BC in alternativa; in molti casi la risposta è affermativa.

Le query in BC

Le specifiche di lettura sono interpretate dall'ambiente di esecuzione per accedere al DBMS SQL con le modalità adeguate, su SQL una lettura è sempre effettuata tramite una query e viene gestita in modo compatibile dal programma e dall'ambiente.
Per SQL è possibile costruire direttamente una query e il BC mette a disposizione diverse specifiche a tale scopo.

Definire una query in BC

Di seguito riportiamo l'esempio di una query molto semplice e vediamo la sua definizione in BC e ne confrontiamo subito la corrispondenza con il linguaggio SQL.

BC - BLD OBJQRY_QUERYBASE_DEFINIZIONE

```
'@PGQUERY BLD OBJQRY_QUERYBASE_DEFINIZIONE STRICT[4] ' Esercitazione query base - Definizione query

'@SBP QueryObject[BCQUERY] [INOUT]

'@DEFQUERY QUERY[QueryObject] NOME["BLD OBJQRY_QUERYBASE_DEFINIZIONE"]

' Definizione della tabella di riferimento della query
' Tabella ARTICOLI
'@FROM TABELLA[BLDART]

' Definizione dei campi della query
' Una sola COLUMN con tanti ESPR quanti sono i campi
'@COLUMN ESPR[BLDART.COD      AS [CodiceArticolo]]      _ ' Codice articolo
            ESPR[BLDART.DES    AS [Descrizione]]          _ ' Descrizione
            ESPR[BLDART.CODFAM AS [Famiglia]]              _ ' Codice famiglia
            ESPR[BLDART.PREZZO AS [PrezzoUnitario]]        _ ' Prezzo unitario di listino
            ESPR[BLDART.SCONTO AS [ScontoArticolo]]        _ ' Sconto articolo
            ESPR[BLDART.QUA    AS [QuantitaMagazzino]]     _ ' Quantità in magazzino

' Filtri (Where)
' Applicazione del filtro: Selezionare tutti i dati DOVE il prezzo è superiore a 30
'@WHERE ESPR[BLDART.PREZZO>30]

' Ordinamento
' Definizione ordinamento ASCENDENTE per DESCRIZIONE
'@ORDERBY IND[1] ESPR[BLDART.DES/ASC]

'@ENDQUERY

End
```

La sub SUB80_QUERYBASE.BC costruisce, attraverso il metodo DEFINIZIONE, una query sulla tabella BLDART per cercare gli articoli con il prezzo maggiore di 30, la query è poi visualizzata in griglia:

Cod.	Descrizione	Famiglia	Prezzo	Sconto	Quantita'
002	Articolo 002	02	134,00	26	87
003	Articolo 003	03	125,00	25	81
004	Articolo 004	04	123,00	24	79
011	Articolo 011	05	33,00	6	21
012	Articolo 012	03	32,00	6	20
013	Articolo 013	01	45,00	9	29
014	Articolo 014	02	35,00	7	22
016	Articolo 016	06	129,00	25	83
017	Articolo 017	05	120,00	24	78
018	Articolo 018	05	134,00	26	87
019	Articolo 019	01	129,00	25	83
025	Articolo 025	01	34,00	6	22

La definizione della query ('@DEFQUERY) richiede la tabella su cui deve essere effettuata ('@FROM e/o '@JOIN) e i campi che devono essere letti, che sono le colonne del record-set restituito ('@COLUMN):

```
SELECT CodArt AS [CodiceArticolo] ,
      DesArt AS [Descrizione] ,
      CodFamiglia AS [Famiglia] ,
      Prezzo AS [PrezzoUnitario] ,
      Sconto AS [ScontoArticolo] ,
      MagQuantita AS [QuantitaMagazzino]
FROM   BLDArticoli AS [BLDArticoli]
```

```
WHERE (PrezzoUnitario > 30 )
ORDER BY [Descrizione] ASC
```

dove sulla specifica ' @FROM il parametro TABELLA[] (da solo) definisce la clausola FROM.

è prevista la condizione in lettura (' @WHERE):

```
SELECT  CodArt AS [CodiceArticolo] ,
        DesArt AS [Descrizione] ,
        CodFamiglia AS [Famiglia] ,
        Prezzo AS [PrezzoUnitario] ,
        Sconto AS [ScontoArticolo] ,
        MagQuantita AS [QuantitaMagazzino]
FROM    BLDArticoli AS [BLDArticoli]
WHERE   (PrezzoUnitario > 30 )
ORDER BY [Descrizione] ASC
```

ed è stato definito l'ordinamento (' @ORDERBY):

```
SELECT  CodArt AS [CodiceArticolo] ,
        DesArt AS [Descrizione] ,
        CodFamiglia AS [Famiglia] ,
        Prezzo AS [PrezzoUnitario] ,
        Sconto AS [ScontoArticolo] ,
        MagQuantita AS [QuantitaMagazzino]
FROM    BLDArticoli AS [BLDArticoli]
WHERE   (PrezzoUnitario > 30 )
ORDER BY [Descrizione] ASC
```

eseguendo la query di cui sopra otteniamo il seguente record-set:

	CodiceArticolo	Descrizione	Famiglia	PrezzoUnitario	ScontoArticolo	QuantitaMagazzino
1	16	GOMMA COLORE	02	500.00	12.00	150
2	07	GOMMA STANDLER	04	500.00	25.00	150
3	06	PENNA BIC	01	100.00	15.00	200
4	15	PENNA BIC ROSSA	02	100.00	5.00	200

Il wizard richiamabile dall'editor (combinazione tasti <Ctrl+W>) è un ottimo supporto alla costruzione di una query e semplifica di molto la scrittura del sorgente BC.

Corrispondenza tra BC e SQL

La seguente tabella mostra l'associazione tra le singole istruzioni BC e il comando SQL generato. Per maggior chiarezza, le specifiche sono state spezzate in più righe nella tabella ed il loro ordine potrebbe risultare modificato rispetto al vero sorgente BC.

BC	SQL
'@DEFQUERY QUERY[QueryObject]	SELECT
'@COLUMN ESPR[BLDART.COD AS [CodiceArticolo] VAR[ARCODART\$]] ESPR[BLDART.DES AS [Descrizione] VAR[ARDESC\$]] ESPR[BLDART.CODFAM AS [Famiglia] VAR[ARCODFAM\$]] ESPR[BLDART.PREZZO AS [PrezzoUnitario] VAR[ARPRE#]] ESPR[BLDART.SCONTO AS [ScontoArticolo] VAR[ARSCO#]] ESPR[BLDART.QUA AS [QuantitaMagazzino] VAR[ARQUA]]	CodArt AS [CodiceArticolo] DesArt AS [Descrizione] CodFamiglia AS [Famiglia] Prezzo AS [PrezzoUnitario] Sconto AS [ScontoArticolo] MagQuantita AS [QuantitaMagazzino]
'@FROM TABELLA[BLDART]	FROM BLDArticoli AS [BLDArticoli]
'@WHERE ESPR[BLDART.PREZZO>30]	WHERE (PrezzoUnitario > 30)
'@ORDERBY IND[1] ESPR[BLDART.DES/ASC]	ORDER BY [Descrizione] ASC

La clausola JOIN

È possibile indicare le tabelle in JOIN ('@JOIN):

```
BC - BLDOBJQRY_JOIN_DEFINIZIONE

'@PGQUERY BLDOBJQRY_JOIN_DEFINIZIONE STRICT[4] ' Esercitazione query con Join - Definizione query

'@SBF QueryObject[BCQUERY] [INOUT]

'@DEFQUERY QUERY[QueryObject] NOME["BLDOBJQRY_JOIN_DEFINIZIONE"]

' Definizione della tabella di riferimento della query
' Tabella BLDART
'@FROM TABELLA[BLDART]

' Definizione della seconda tabella all'interno della query.
' Ogni tabella inserita deve essere in relazione con una tabella già presente

' Relazione tra BLDFAMIGLIA (FamiglieArticoli) - Famiglie
' e BLDART (Articoli) - Articoli
'@JOIN TABELLA[BLDFAMIGLIA] TABELLAPADRE[BLDART] _
REL[BLDFAMIGLIA.COD=BLDART.CODFAM] OPERATOREJOIN["LEFT"]

' Definizione dei campi della query
' Una sola DEFQUERYFIELD con tanti ESPR quanti sono i campi (anche se puntano a tabelle differenti)
'@COLUMN ESPR[BLDART.COD AS [CodiceArticolo]] _ ' Codice articolo _
          ESPR[BLDART.DES AS [Descrizione]] _ ' Descrizione _
          ESPR[BLDART.PREZZO AS [PrezzoUnitario]] _ ' Prezzo unitario di listino _
          ESPR[BLDART.SCONTO AS [ScontoArticolo]] _ ' Sconto articolo _
          ESPR[BLDART.QUA AS [QuantitaMagazzino]] _ ' Quantità in magazzino _
          ESPR[BLDFAMIGLIA.DES AS [DesFamiglia]] _ ' Descrizione

' Ordinamento
' Definizione ordinamento ASCENDENTE per DESCRIZIONE
'@ORDERBY IND[1] ESPR[BLDART.DES/ASC]

'@ENDQUERY

End
```

dove sulla specifica '@JOIN il parametro TABELLA[] definisce la tabella da mettere in join con la tabella principale indicata nel parametro TABELLAPADRE[].

e i campi di tutte le tabelle per il record-set ('@COLUMN):

```
SELECT BLDArticoli.CodArt AS [CodiceArticolo]
      BLDArticoli.DesArt AS [Descrizione]
      BLDArticoli.CodFamiglia AS [Famiglia]
      BLDArticoli.Prezzo AS [PrezzoUnitario]
      BLDArticoli.Sconto AS [ScontoArticolo]
      BLDArticoli.MagQuantita AS [QuantitaMagazzino]

      BLDFamiglie.CodFamiglia AS [CodiceFamiglia]
      BLDFamiglie.DesFamiglia AS [DesFamiglia]

FROM BLDArticoli AS [BLDArticoli]

LEFT OUTER JOIN BLDFamiglie AS [BLDFamiglie] ON (
  BLDFamiglie.CodFamiglia = BLDArticoli.CodFamiglia

  AND (BLDFamiglie.DBGruppo = BLDArticoli.DBGruppo)

)

ORDER BY [Descrizione] ASC
```

Corrispondenza tra BC e SQL

La seguente tabella mostra l'associazione tra le singole istruzioni BC e il comando SQL generato. Per maggior chiarezza, le specifiche sono state spezzate in più righe nella tabella ed il loro ordine potrebbe risultare modificato rispetto al vero sorgente BC.

BC	SQL
...	...
'@JOIN TABELLA[BLDFAMIGLIA] TABELLAPADRE[BLDART] REL[BLDFAMIGLIA.COD=BLDART.CODFAM] OPERATOREJOIN["LEFT"]	LEFT OUTER JOIN BLDfamiglie AS [BLDFamiglie] ON (BLDfamiglie.CodFamiglia = BLDArticoli.CodFam)
...	...

Tabelle di sessione e di servizio

Esistono due tipologie di tabelle che possono essere utilizzate in una JOIN di una query:

- Le tabelle di sessione
- Le tabelle di servizio

Le tabelle di sessione sono delle tabelle vere e proprie create a runtime ed eliminate subito dopo il loro utilizzo dall'ambiente di esecuzione. Queste tabelle vengono popolate tramite diverse fonti dati (variabili di tipo blob, collezioni, ecc...) presenti in memoria in quel momento.

 Per maggiori dettagli sul funzionamento e utilizzo consultare la scheda ["Funzionamento delle tabelle di sessione in query"](#).

Le tabelle di servizio sono delle tabelle che vengono definite in prodotti differenti per cui non si hanno a disposizione i DFX per effettuare la definizione della query nei propri sorgenti.

 Per maggiori dettagli consultare la scheda del *Manuale di riferimento del BC* ["@DEFQUERYTABLESERVICE"](#).

Campi calcolati in query e clausola GROUPBY

È possibile inserire nel record-set un campo calcolato ('@COMPUTEDCOLUMN').

```
BC - SUB82_QUERY_CCALC

'@PGQUERY BLDobjQRY_QUERY_CCALC_DEFINIZIONE STRICT[4]

'@DEFQUERY QUERY[QueryObject] NOME["BLDOBJQRY_QUERY_CCALC_DEFINIZIONE"]

' Definizione della tabella di riferimento della query
' Tabella BLDART
'@FROM TABELLA[BLDART]

' Definizione della seconda tabella all'interno della query.
' Ogni tabella inserita deve essere in relazione con una tabella già presente

' Relazione tra BLDfamiglia (FamiglieArticoli) - Famiglie
' e BLDART (Articoli) - Articoli
'@JOIN TABELLA[BLDFAMIGLIA] TABELLAPADRE[BLDART] _
REL[BLDFAMIGLIA.COD = BLDART.CODFAM] OPERATOREJOIN["LEFT"]

' Definizione dei campi della query
' Una sola DEFQUERYFIELD con tanti ESPR quanti sono i campi (anche se puntano a tabelle differenti)
'@COLUMN ESPR[GROUPBY(BLDART.SCONTO) AS [ScontoArticolo]] _ ' Sconto articolo _
ESPR[GROUPBY(BLDART.PREZZO) AS [PrezzoUnitario]] _ ' Prezzo unitario di listino _
ESPR[GROUPBY(BLDART.COD) AS [CodiceArticolo]] _ ' Codice articolo _
ESPR[GROUPBY(BLDART.DES) AS [Descrizione]] _ ' Descrizione _
ESPR[GROUPBY(BLDFAMIGLIA.DES) AS [DesFamiglia]] _ ' Descrizione

' Definizione di un campo calcolato che deve corrispondere a:
' SE lo sconto = 0 allora il campo calcolato è uguale al campo prezzo
' altrimenti il campo calcolato è uguale al campo prezzo - la percentuale di sconto
'@COMPUTEDCOLUMN ESPR[IF BLDART.SCONTO=0 THEN BLDART.PREZZO _
ELSE BLDART.PREZZO-((BLDART.PREZZO*BLDART.SCONTO)/100) [PrezzoDefinitivo] FMT[NUMERICO[8,2]]]

' Filtri (Where)
' Applicazione del filtro: Selezionare tutti i dati DOVE il prezzo è maggiore o uguale a 50
'@WHERE ESPRHAVING[SUM([PrezzoDefinitivo])>=50]

'@ENDQUERY
End
```

o introdurre la clausola GROUPBY ('@COLUMN)

BC - SUB83_QUERY_RAG

```
'@PGQUERY BLD OBJQRY_QUERY_RAG_DEFINIZIONE STRICT[4]

'@DEFQUERY QUERY[QueryObject] NOME["BLD OBJQRY_QUERY_RAG_DEFINIZIONE"]

' Definizione della tabella di riferimento della query
' Tabella BLD FAMIGLIA
'@FROM TABELLA[BLD FAMIGLIA]

' Definizione della seconda tabella all'interno della query.
' Ogni tabella inserita deve essere in relazione con una tabella già presente

' Relazione tra BLDART (Articoli) - Articoli
' e BLD FAMIGLIA (FamiglieArticoli) - Famiglie
'@JOIN TABELLA[BLDART] TABELLAPADRE[BLD FAMIGLIA] _
REL[BLDART.CODFAM = BLD FAMIGLIA.COD] OPERATOREJOIN["LEFT"]

' Definizione dei campi della query
' Una sola DEFQUERYFIELD con tanti ESPR quanti sono i campi (anche se puntano a tabelle differenti)
' All'interno del parametro ESPR viene definita una funzione di RAGGRUPPAMENTO (GROUPBY/SUM).
' Se definite TUTTI i campi devono avere una funzione associata.
'@COLUMN ESPR[GROUPBY(BLD FAMIGLIA.COD) AS [CodiceFamiglia]] _ ' Codice famiglia _
ESPR[GROUPBY(BLD FAMIGLIA.DES) AS [Descrizione]] _ ' Descrizione _
ESPR[SUM(BLDART.PREZZO) AS [PrezzoUnitario]] ' Prezzo unitario di listino

'@ENDQUERY

End
```

sintassi SQL:

```
SELECT BLD Famiglie.CodFamiglia AS [CodiceFamiglia],
BLD Famiglie.DesFamiglia AS [Descrizione],
SUM(BLDArticoli.Prezzo) AS [PrezzoUnitario]

FROM BLD Famiglie AS [BLD Famiglie]
LEFT OUTER JOIN BLDArticoli AS [BLDArticoli] ON (
BLDArticoli.CodFamiglia = BLD Famiglie.CodFamiglia
AND (BLDArticoli.DBGruppo = BLD Famiglie.DBGruppo)
)

GROUP BY BLD Famiglie.CodFamiglia
,BLD Famiglie.DesFamiglia
```

Definizione delle tabelle e delle relazioni

La definizione delle relazioni tra tabelle è necessaria quando la query ha necessità di operare su più tabelle e non solo su una.

La definizione di una relazione avviene tramite l'utilizzo della specifica '@JOIN su cui va indicato:

- le tabelle in relazione: il parametro TABELLA serve per indicare la tabella da relazionare e TABELLAPADRE la tabella su cui avviene la relazione;
- la relazione tramite i campi chiave tramite il parametro REL;
- la tipologia della relazione tramite il campo OPERATOREJOIN.

 Per l'elenco delle tipologie di join supportate consultare la scheda [OperatoreJoin](#) del *Manuale di riferimento del BC*.

Il codice BC

Nell'esempio della query che vogliamo costruire, si vuole aggiungere il codice e la descrizione della famiglia articoli contenuti nella tabella BLDFAMIGLIA. Inseriamo quindi la tabella BLDFAMIGLIA in JOIN con la tabella BLDART.

BC

```
...
'@JOIN TABELLA[BLDFAMIGLIA] TABELLAPADRE[BLDART] _
      REL[BLDFAMIGLIA.COD = BLDART.CODFAM] OPERATOREJOIN["LEFT"]
...

```

Esempio di lettura del recordset

Per leggere più record da una tabella è necessario utilizzare il datareader, già incontrato nelle lezioni precedenti.

L'ambiente di esecuzione esegue un preciso flusso operativo per la lettura dei record, ovvero:

- effettua una chiamata al DBMS che restituisce il risultato sotto forma di record-set;
- preleva il record-set e lo memorizza in una struttura tabellare in memoria;
- procede con una lettura sequenziale della struttura.

BC

```
...
While QueryObject.Next()

    QueryObject.Campo1

    ...

EndWhile

```

Definizione dei campi (colonne del record-set)

La definizione di una query deve prevedere la definizione di almeno un campo, non è pertanto possibile, in BC, formalizzare tramite una query una sintassi SQL del tipo: *SELECT * FROM*, nel codice BC devono essere definiti i campi delle tabelle da interrogare con la specifica '@COLUMN.

Si noti che nel parametro ESPR c'è una sintassi del tipo TABELLA.CAMPO [Alias].

Da ora in poi, nelle specifiche di costruzione della query, sarà dunque equivalente indicare TABELLA.CAMPO oppure [Alias] per riferirsi a questo campo.

BC

```
...
'@COLUMN ESPR[BLDART.COD      AS [CodiceArticolo]]      _ ' Codice articolo
          ESPR[BLDART.DES      AS [Descrizione]]          _ ' Descrizione
          ESPR[BLDART.CODFAM    AS [Famiglia]]              _ ' Codice famiglia
          ESPR[BLDART.PREZZO     AS [PrezzoUnitario]]        _ ' Prezzo unitario di listino
          ESPR[BLDART.SCONTO     AS [ScontoArticolo]]        _ ' Sconto articolo
          ESPR[BLDART.QUA        AS [QuantitaMagazzino]]     _ ' Quantità in magazzino

          ESPR[BLDFAMIGLIA.COD   AS [CodiceFamiglia]]        _ ' Codice famiglia
          ESPR[BLDFAMIGLIA.DES   AS [DesFamiglia]]           ' Descrizione
...

```

SQL

...

```

SELECT BLDArticoli.CodArt AS [CodiceArticolo]
      BLDArticoli.DesArt AS [Descrizione]
      BLDArticoli.CodFamiglia AS [Famiglia]
      BLDArticoli.Prezzo AS [PrezzoUnitario]
      BLDArticoli.Sconto AS [ScontoArticolo]
      BLDArticoli.MagQuantita AS [QuantitaMagazzino]

      BLDFamiglie.CodFamiglia AS [CodiceFamiglia]
      BLDFamiglie.DesFamiglia AS [DesFamiglia]

```

...

Definizione dei filtri (condizioni)

La definizione di una query normalmente prevede la definizione di almeno un filtro in lettura. Se da un lato i campi operano un filtro sulle colonne delle tabelle da leggere, le condizioni di lettura operano un filtro sulle righe.

Quanto migliori sono i filtri di lettura, tanto più preciso e limitato in grandezza sarà il risultato della query (record-set) e, di conseguenza, tanto più veloce e performante sarà il programma scritto.

Nel codice BC i filtri in lettura (clausola WHERE) sono definiti tramite la specifica '@WHERE.

BC	SQL
...	...
'@WHERE ESPR[BLDART.QUA>100]	WHERE (BLDArticoli.MagQuantita > 100)
...	...

Tabelle di gruppo

Sulle tabelle di gruppo il gruppo è fisicamente la prima colonna della tabella:

	DBGruppo	BCQVerAmbID	BCQVerAmbDesc	BCQVerAmbPath	BCQVerAmb
1	B1	V_18.10	Versione 18.10	\\NTSIST\F\AMBIENTE\w_1810\	0
2	B1	V_18.10a	Versione 18.10a	\\NTSIST\F\AMBIENTE\w_1810a\	0
3	B1	V_18.11	Versione 18.11	\\NTSIST\F\AMBIENTE\w_1811\	0
4	B1	V_18.11a	Versione 18.11a	\\NTSIST\F\AMBIENTE\w_1811a\	0
5	B1	V_18.12	Versione 18.12	\\NTSIST\F\AMBIENTE\w_1812\	0
6	B1	V_18.12a	Versione 18.12a	\\NTSIST\F\AMBIENTE\w_1812a\	0
7	B1	V_18.12b	Versione 18.12b	\\NTSIST\F\AMBIENTE\w_1812b\	0
8	B1	V_18.13	Versione 18.13	\\NTSIST\F\AMBIENTE\w_1813\	0
9	B1	V_18.14	Versione 18.14	\\NTSIST\F\AMBIENTE\w_1814\	0
10	B1	V_18.14a	Versione 18.14a	\\NTSIST\F\AMBIENTE\w_1814a\	0
11	B1	V_18.15	Versione 18.15	\\NTSIST\F\AMBIENTE\w_1815\	0
12	B1	V_18.15a	Versione 18.15a	\\NTSIST\F\AMBIENTE\w_1815a\	0
13	B1	V_18.16	Versione 18.16	\\NTSIST\F\AMBIENTE\w_1816\	0
14	B1	V_18.16a	Versione 18.16a	\\NTSIST\F\AMBIENTE\w_1816a\	0

è, quindi, una informazione fisicamente in tabella gestita in modo automatico dall'ambiente.

Normalmente, in tutte le operazioni fatte sulla tabella, l'ambiente aggiunge in automatico la clausola WHERE come di seguito riportato:

SQL

...

```
WHERE (<tabella>.DBGruppo='XX')
```

...

Per poter fare in modo che il programma gestisca in modo autonomo l'eventuale filtro in lettura sul gruppo è necessario definire la query MULTIGRUPPO.

Definizione degli ordinamenti

Un aspetto importante nella definizione di una query è l'indicazione dell'ordinamento o degli ordinamenti possibili coi quali esporre o consultare i dati.

Un indice di ordinamento si definisce come elenco di campi, ognuno avente la proprietà di ordinamento ascendente (dal più piccolo al più grande) o discendente (dal più grande al più piccolo). Rispetto alla normale definizione degli indici delle tabelle, gli indici della query hanno il vantaggio di poter definire sia campi con proprietà "ascendente" che campi con proprietà "discendente". Rimane ovviamente fondamentale l'ordine con il quale si indicano i campi.

Nel codice BC l'ordinamento è definito tramite la specifica `'@DEFQUERYORDERBY`.

BC	SQL
...	...
@ORDERBY IND[1] ESPR[BLDART.DES/ASC]	ORDER BY [Descrizione] ASC
...	...

Definizione dei raggruppamenti

Tra le funzionalità più importanti di una query c'è il raggruppamento. Un raggruppamento su un campo corrisponde ad una funzione di aggregazione da applicare al campo stesso al termine della lettura del record-set.

Se si indica un raggruppamento su un campo, è obbligatorio indicare un raggruppamento per ognuno di tutti gli altri campi della query.

Le tipologie di raggruppamento consentite sul singolo campo sono:

- GROUPBY - raggruppa per ...
- COUNT - conta i record estratti dalla query
- COUNT DISTINCT - conteggio solamente dei record non doppi
- MIN - Valore minimo
- MAX - Valore massimo.

Per i campi numerici si aggiungono le funzioni di aggregazione:

- SUM - Sommatoria
 - SUM DISTINCT - Calcola la somma di tutti i valori del raggruppamento
- escludendo le righe con valori del campo specificato ripetuti
- AVG - Valore medio.
 - AVG DISTINCT- Calcola la media di tutti i valori del raggruppamento
- escludendo le righe con valori del campo specificato ripetuti.

La definizione del raggruppamento sul campo è fatta tramite la specifica `'@COLUMN`.

BC
...
'@COLUMN ESPR[BLDFAMIGLIA.COD AS [CodiceFamiglia]] - ' Codice famiglia
ESPR[BLDFAMIGLIA.DES AS [DesFamiglia]] - ' Descrizione
ESPR[BLDART.PREZZO AS [PrezzoUnitario]] ' Prezzo unitario di listino
...

SQL
...
SUM(BLDArticoli.Prezzo) AS [PrezzoUnitario]
GROUP BY BLDFamiglie.CodFamiglia,
BLDFamiglie.DesFamiglia
...

E' stato definito un raggruppamento SUM sul campo 'PrezzoUnitario' pertanto è stato inserito un raggruppamento GROUPBY sugli altri campi.

Definizione campi calcolati e filtri su campi calcolati

Non sempre, all'interno della base dati, si hanno a disposizione tutte le informazioni già formattate secondo le esigenze del programma. Il caso più semplice in questo senso è la Ragione Sociale dell'anagrafica generale: essa è divisa in due campi (RagioneSociale1 e RagioneSociale2) e, se si vuole riportare come un'unica colonna, si è obbligati a passare tramite un

campo calcolato come somma dei due campi più uno spazio in mezzo.

Oppure, di cui l'esempio sotto riferito, abbiamo l'esigenza di definire un campo calcolato che deve corrispondere al prezzo di un articolo scontato se la percentuale dello sconto da applicare è diversa da zero.

Esempio:

- se la percentuale dello sconto è uguale 0 devo visualizzare il campo prezzo;
- se la percentuale dello sconto è diversa da 0 devo visualizzare il "prezzo-(%sconto)"

La definizione di un campo calcolato è fatta tramite la specifica '@COMPUTEDCOLUMN.

BC
<pre>' Definizione di un campo calcolato che deve corrispondere a: ' se lo sconto = 0 allora il campo calcolato è uguale al campo prezzo ' altrimenti il campo calcolato è uguale al campo prezzo - la percentuale di sconto PrezzoDefinitivo=0 '@COMPUTEDCOLUMN ESPR[IF BLDART.SCONTO=0 THEN BLDART.PREZZO _ ELSE BLDART.PREZZO- ((BLDART.PREZZO*BLDART.SCONTO)/100) [PrezzoDefinitivo] FMT[NUMERICO[8,2]]]</pre>

SQL
<pre>... CASE WHEN BLDArticoli.Sconto = 0 THEN BLDArticoli.Prezzo ELSE BLDArticoli.Prezzo - ((BLDArticoli.Prezzo * BLDArticoli.Sconto) / 100) END AS [PrezzoDefinitivo] ...</pre>

Definizione dei filtri (condizioni) su raggruppamenti

La definizione di una clausola *WHERE* (filtro) nel linguaggio SQL vieta l'utilizzo di formule di aggregazione al proprio interno.

Questo tipo di filtri devono essere indicati nella clausola *HAVING*.

Il linguaggio BC prevede una sola specifica di filtro per gestire entrambe le casistiche; la differenziazione avviene con l'uso di parametri diversi:

- '@WHERE ESPR[. . .] per definire una condizione di filtro
- '@WHERE ESPRHAVING[. . .] per definire una condizione di filtro su campo aggregato.

Volendo visualizzare solo gli articoli che hanno un prezzo definitivo non inferiore ai 50€ devo inserire un filtro sul campo calcolato [PrezzoDefinitivo] nel seguente modo:

BC
<pre>... ' SOGLIA_MINIMA_PREZZO#[15,2]=50 ' '@WHERE ESPRHAVING[SUM([PrezzoDefinitivo])>=SOGLIA_MINIMA_PREZZO] ' ...</pre>
SQL

```
...
'
HAVING (
    SUM(
        CASE
            WHEN BLDArticoli.Sconto = 0
            THEN BLDArticoli.Prezzo
            ELSE BLDArticoli.Prezzo - ((BLDArticoli.Prezzo * BLDArticoli.Sconto) / 100)
        END
    ) >= 50
)
'
...
```

Precisazioni sulla conversione in SQL dei filtri su raggruppamenti

Il BC ha un grande vantaggio rispetto al linguaggio SQL: permette di indicare l'alias nell'espressione di filtro (con notevole guadagno in sintesi), mentre SQL obbliga a replicare l'intera formula di raggruppamento già espressa nella sintassi di *SELECT*.

Per definire una condizione sui filtri di raggruppamento è possibile impiegare la '@WHERE.

Con la specifica '@WHERE si usa l'attributo *ESPRHAVING*.

Come sopra riportato dall'esempio è possibile notare come l'ambiente opera alcuni automatismi sulla traduzione dei filtri indicati in istruzioni SQL al fine di garantire la corretta sintassi; sono quindi inseriti in automatico per i campi presenti nel filtro su raggruppamenti le funzioni di raggruppamento come richiesto dal linguaggio SQL.

Questa conversione avviene secondo la seguente tabella:

Funzione di raggruppamento del campo	Operazione compiuta dall'ambiente
GROUP BY	Nessuna operazione di adeguamento
SUM	Si aggiunge al campo la funzione SUM
MAX	Si aggiunge al campo la funzione MAX
MIN	Si aggiunge al campo la funzione MIN
AVG	Si aggiunge al campo la funzione AVG

La conversione non avviene però in maniera uniforme, ma secondo i seguenti criteri:

- 1. Se il campo è indicato mediante il suo ALIAS, allora al campo è applicata sempre la conversione descritta dalla tabella precedente;
- 2. Se il campo è indicato mediante la notazione Descrittore.Nomebreve, allora la conversione è applicata solo se il campo è stato indicato all'interno dell'attributo FORMULA1 o FORMULA2 della '@DEFQUERYWHEREGRID.

Funzionamento delle tabelle di sessione in query

L'ambiente BC permette di definire e utilizzare una nuova tipologia di tabelle, le tabelle di sessione.

Le tabelle di sessione sono una funzionalità di SQL Server:

- Il loro nome inizia con il carattere # (tabelle temporanee di sessione) o ## (tabelle temporanee globali condivise)
- Sono create in modo analogo alle tabelle standard mediante l'operazione CREATE TABLE
- Sono "droppate" in automatico dal sql server quando la connessione si chiude 8 (per quelle globali condivise quando l'ultima connessione si chiude)

L'ambiente BC popola le tabelle di sessione con i dati presenti sulla memoria del programma corrente. Facendo un esempio, si supponga di avere la (solita) stringa bc che contiene un elenco di id articoli, frutto di una selezione operata dall'operatore. L'approccio attuale per operare un filtro dei soli articoli usa la funzione INLISTSTR:

```
BC
'@WHERE ESPR[INLISTSTR(Articoli.id, Selezionati?)>0]
```


L'istruzione BC precedente diventa in SQL

SQL
<code>SELECT ... FROM ... WHERE Articoli.Id IN (<elenco id articoli presenti in Selezionati?>)</code>

Se il numero di id articoli è basso, l'istruzione SQL è ancora “maneggiabile” e sufficientemente ottimizzata per le prestazioni. Se il numero di id articolo è invece alto, l'istruzione SQL diventa subito complessa. Difficilmente l'ottimizzatore di SQL Server sceglierà un piano che non implichi la scansione completa della tabella Articoli.

Si possono usare le tabelle di sessione per migliorare le prestazioni :

- Lato BC, prima di definire la condizione di filtro, si definisce all'interno della query una tabella di sessione:

BC
<code>'@DEFQUERYSESSIONTABLE NOME["Sele"] VALORI[Selezionati?] FMT[NUMERICO[1]]</code>

L'istruzione definisce una tabella di sessione, di nome “Sele”, popolata con il valore corrente della variabile Selezionati?. I dati sono di tipo numerico, come specificato dal parametro FMT.

Si può quindi utilizzare la tabella di sessione per definire il filtro:

BC
<code>'@WHERE ESPR[Articoli.id IN TAB(Sele)]</code>

Che produce l'istruzione SQL:

SQL
<code>SELECT ... FROM ... WHERE Articoli.Id IN (SELECT Codice from #Sele)</code>

Osservazioni:

- Con le tabelle di sessione è possibile utilizzare una nuova sintassi per la clausola IN, molto simile alla forma SQL: Articoli IN TAB(Sele)]
- L'istruzione SQL rimane “semplice” anche se il numero di articoli selezionati è elevato. Il filtro avviene usando una sotto query: Articoli.Id IN (SELECT Codice from #Sele)
- La tabella di sessione #Sele è popolata in automatico dall'ambiente nel contesto dell'esecuzione della istruzione GETQUERYDATA in maniera ottima. La struttura è fissa, in funzione della variabile Selezionati? e del formato dell'esempio precedente.

Le tabelle di sessione possono essere impiegate all'interno della query di definizione come se fossero tabelle a tutti gli effetti. Ad esempio si può decidere che la maniera ottima per effettuare il filtro non sia l'impiego della clausola IN ma di una INNER JOIN:

BC
<code>'@JOIN TABELLA[[Sele]] TABELLAPADRE[Articoli] REL[[Sele].Codice =Articoli.Id] OPERATOREJOIN["INNER"]</code>

La sintassi che si usa all'interno delle specifiche di definizione delle query è la stessa già definita per le CTE, ovvero si racchiude il nome della tabella tra parentesi quadre: [Sele]. L'effetto dell'istruzione BC precedente è:

SQL
<code>Select ... FROM Articoli INNER JOIN #Sele ON (#Sele.Codice = Articoli.Id) AS #Sele</code>

La tabella di sessione è definita in modo in automatico dall'ambiente:

- Ha una sola colonna
- Il tipo della colonna è definito dal formato
- Il nome del campo lato programmazione BC è Codice

I formati ammessi per definire il tipo colonna sono:

Formato	Tipo colonna
FMT[DATA6]	Data
FMT[DATA8]	Data
FMT[ANNO2]	Intero
FMT[ANNO4]	Intero
FMT[PERIOD04]	Periodo
FMT[PERIOD06]	Periodo
FMT[GIORNOMESE]	Intero
FMT[ORA]	Intero
FMT[ORA6]	Intero
FMT[IMPORTO]	Double
FMT[IMPORTOEURO]	Double
FMT[CODICE]	Double
FMT[NUMERICO[NN,DD]]	Long o Double
FMT[TESTO[nn]]	Blob

E' possibile definire anche tabelle di sessione con più colonne. Per farlo occorre specificare come “fonte dati” una collezione di classi o una struttura dinamica. Ad esempio:

```
BC

'@DEFDYNSTRUCT  STRUTTURA["UpdPrezzo"]  CAMPI[IdArticolo[5], Prezzo#[6,2]]

'@DEFQUERY  IDQUERY[IdQuery]
'@DEFQUERYSESSIONTABLE  IDQUERY[IdQuery]  NOME["NuoviPrezzi"]  STRUTTURA["UpdPrezzo"]
'@DEFQUERYFROM  IDQUERY[IdQuery]  TABELLA[Articoli]
'@DEFQUERYJOIN  IDQUERY[IdQuery]  TABELLA[["NuoviPrezzi"]]  TABELLAPADRE[Articoli]  REL[["NuoviPrezzi"].IDARTICOLO=Articoli.ID]  OPERATOREJOIN["INNER"]
'@DEFQUERYCOLUMN  IDQUERY[IdQuery]  ESPR[["NuoviPrezzi"].PREZZO AS [NuovoPrezzo]]

'@UPDATEDBDBDATA  TABELLA[Articoli]  QUERY[IdQuery]  ASSEGNA[Prezzo=ALIAS["NuovoPrezzo"]]
```

Nell’esempio il programma impiega la struttura dinamica “UpdPrezzo” per effettuare una variazione multipla dei prezzi degli articoli. La struttura è’ definita con 2 colonne, IdArticolo e Prezzo, ovvero la chiave della tabella che si vuole aggiornare ed il campo che è oggetto della variazione.

Impiega quindi una query per aggiornare in un colpo i prezzi. La query è definita nel seguente modo:

- Definizione della tabella di sessione “NuoviPrezzi”, prendendo struttura e dati dalla struttura dinamica “UpdPrezzo”
- Definizione della tabelle della query: Articoli in Join con la tabella di sessione “NuoviPrezzi”

Per creare una tabella di sessione a partire da una collezione usare il parametro COLLEZIONE:

```
BC

DIM coll[CLASSE[Articoli] COLLEZIONE]

'@DEFQUERYSESSIONTABLE  IDQUERY[IdQuery]  NOME["ArticoliDaSpostare"]  COLLEZIONE[coll]
```

Nota per il camel case: strutture dinamiche e oggetti possono essere definiti in modo "cammellato", ma il traduttore scarica comunque i campi di queste entità in maiuscolo. Questo si riflette sulla definizione della tabella di sessione, che avrà i propri campi in maiuscolo. occorre quindi indicare il campo della tabella di sessione in maiuscolo nella fomulazione della propria query

Funzionamento delle tabelle di sessione

Le tabelle di sessione definite mediante la specifica ‘@DEFQUERYTABLESESSION sono gestite nel seguente modo:

- Quando il programma BC esegue la specifica ‘@DEFQUERYTABLESESSION l’ambiente immagazzina nella query struttura e dati necessari per la successione creazione/ popolamento della tabella di sessione
- Quando il programma BC esegue per la prima volta l’istruzione GETQUERYDATA, prima di inviare al server il comando SELECT esegue le seguenti operazioni preliminari per le tabelle di sessioni eventualmente presenti
- Assegna un nome univoco alla tabella di sessione (appende un prefisso numerico)
- Crea la tabella di sessione create
- Popola la tabella di sessione

- Quando il programma BC ha completamente consumato i dati della GETQUERYDATA l'ambiente dropa (cancella) la tabella di sessione
- Se il programma effettua un posizionamento della getquerydata (parametro POSIZIONA) l'ambiente dropa e ricrea la tabella. I dati rimangono comunque gli stessi (è un operazione da ottimizzare)

Esempi di utilizzo delle tabelle di sessione

Utilizzo di una tabella di sessione in una query di autocompletamento

Le query di auto-completamento sono normalmente utilizzate per "decodificare" dati di una tabella, ad esempio la descrizione. In alcuni casi si vuole però utilizzare dei dati in memoria.

E' possibile utilizzare le tabelle di sessione per questo scopo. Nell'esempio che segue, la seguente collezione:

```
DIM list_of_auts[CLASSE[AIGCLASS4] COLLEZIONE]
DIM aut[CLASSE[AIGCLASS4]]

aut.Clear():aut.Codice = 1:aut.autore = "Tolkien":aut.sottopubblicazione = "H":aut.editore
= 1:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 2:aut.autore = "Christie":aut.sottopubblicazione = "H":aut.editor
e = 2:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 3:aut.autore = "Orwell":aut.sottopubblicazione = "H":aut.editore
= 3:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 4:aut.autore = "Eco":aut.sottopubblicazione = "H":aut.editore = 1
:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 5:aut.autore = "Dickens":aut.sottopubblicazione = "H":aut.editore
= 2:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 6:aut.autore = "Manzoni":aut.sottopubblicazione = "H":aut.editore
= 3:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 7:aut.autore = "D'Annunzio":aut.sottopubblicazione = "H":aut.edi
tore = 4:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 8:aut.autore = "King":aut.sottopubblicazione = "H":aut.editore =
3:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 9:aut.autore = "Leopardi":aut.sottopubblicazione = "H":aut.editor
e = 2:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 10:aut.autore = "Rowling":aut.sottopubblicazione = "H":aut.editore
= 1:list_of_auts.Add(aut)
```

Dovrà essere usata per una query di auto-completamento per decodificare l'autore. Per farlo la query

```
QUERY[5]=0
'@DEFQUERY IDQUERY[QUERY]

'@DEFQUERYSESSIONTABLE IDQUERY[QUERY] NOME["AUTS"] COLLEZIONE[list_of_auts]

' Tabella BTF_AUTORE
'@DEFQUERYFROM IDQUERY[QUERY] TABELLA[ [AUTS] ]

'@DEFQUERYCOLUMN IDQUERY[QUERY] _
    ESPR[ [AUTS].CODICE AS [VAR]] ETICHETTA["Codice"] VAR[AUCODAUT$] _
    ESPR[ [AUTS].AUTORE AS [Autore]] ETICHETTA["Autore"] VAR[AUCOGNOME$] N
OOUTPUT _ ' Cognome

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[QUERY] _
    ESPR[FNS$( [AUTS].CODICE) + " -
" + [AUTS].AUTORE AS [VARVIDEO] ETICHETTA["Autore"] VAR[CGN$]]

' Filtri (Where)
'@DEFQUERYWHERE IDQUERY[QUERY] ESPR[ _
    MATCH( [VAR], "%#TD%")>0 _
    OR MATCH( [VARVIDEO], "%#TD%")>0 ]
```

Nella query si impiegano le medesime convenzioni usate per la gestione dell'auto-completamento, ovvero gli alias VAR e VARVIDEO.

L'utilizzo della query di auto completamento rimane il medesimo:

https://supporto.sistemi.com/5108/1/927075/full_print_view

```
'@DEFEDIT VAR[VAR1$] ID["v1"] RET[1,1] POS[1,1] DIM[1,60] FMT[TESTO]
AUTOCOMPLETAMENTO[IDQUERY[QUERY]] _
DECODVIDEO[ VARVIDEO[VAR2$] FMTVIDEO[TESTO] ] _ ' DECOD[GoSub
BLANK_ACTION] _
GRID[COL[3] RIGA[1] RIGADIN[1] COLDIN[1] ]
```

Le tabelle di sessione dell'auto-completamento sono soggette ad una gestione speciale rispetto alle tabelle di sessione ordinarie. La query di auto-completamento "vive" su una connessione a parte rispetto a quella utilizzata dal programma principale. Inoltre l'auto-completamento è eseguito in un "thread" diverso da quello principale:

- Quando una videata è visualizzata per la prima volta, l'ambiente si predispone per la gestione delle eventuali tabelle di sessione che i campi con auto-completamento potrebbero impiegare
- Quando un campo di auto-completamento viene attivato, ovvero digitando qualcosa, prima di eseguire la query di auto-completamento, avviene la creazione della tabella di sessione e del suo popolamento.
- Quando la videata viene chiusa, tutte le tabelle di sessione create per i campi di auto-completamento vengono "droppate"

Alcune considerazioni:

- Possono esserci anche più tabelle di sessione nella query di autocompletamento
- Due campi di edit possono condividere la stessa query di auto-completamento. In questo caso anche le tabelle di sessione sono in comune
- Due videate non condividono mai le eventuali tabelle di sessione dell'auto completamento.

Aggiornamento dei dati della tabella di sessione

I dati della tabella di sessione impiegata nella query di autocompletamento può essere aggiornati ricreando la query e quindi passando nuovi valori alla DEFQUERYSESSIONTABLE e usando la specifica SETEDIT, come in esempio riportato:

```
list_of_auts.Clear()

aut.Clear():aut.Codice = 1:aut.autore = "new-
Tolkien":aut.sottopubblicazione = "W":aut.editore = 1:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 2:aut.autore = "new-
Christie":aut.sottopubblicazione = "W":aut.editore = 2:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 3:aut.autore = "new-
Orwell":aut.sottopubblicazione = "W":aut.editore = 3:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 4:aut.autore = "new-
Eco":aut.sottopubblicazione = "W":aut.editore = 1:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 5:aut.autore = "new-
Dickens":aut.sottopubblicazione = "W":aut.editore = 2:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 6:aut.autore = "new-
Manzoni":aut.sottopubblicazione = "W":aut.editore = 3:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 7:aut.autore = "new-
D'Annunzio":aut.sottopubblicazione = "W":aut.editore = 4:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 8:aut.autore = "new-
King":aut.sottopubblicazione = "W":aut.editore = 3:list_of_auts.Add(aut)
aut.Clear():aut.Codice = 9:aut.autore = "new-
Leopardi":aut.sottopubblicazione = "W":aut.editore = 2:list_of_auts.Add(aut)
aut.Clear():aut.Codice =10:aut.autore = "new-
Rowling":aut.sottopubblicazione = "W":aut.editore = 1:list_of_auts.Add(aut)

GoSub QUERY

'@SETEDIT IDCONTROLLO["v1"] AUTOCOMPLETAMENTO[IDQUERY[query]]

RETURN
```

Utilizzo di una tabella di sessione in una CTE

Le tabelle di sessione possono essere usate anche all'interno di una CTE. Per farlo la tabella di sessione va definita sulla query principale (in realtà sempre) e poi può essere impiegata nelle CTE come in qualsiasi altro punto della query.:

```
'@DEFQUERY IDQUERY[IdQryOgg] ELIMINADUPLICATI[1]
'@DEFQUERY IDQUERY[IdQryOggCTE1] IDQUERYPADRE[IdQryOgg] CTE[NOME["OggComponenti"] PARTE[#FISSA]]
'@DEFQUERY IDQUERY[IdQryOggCTE2] IDQUERYPADRE[IdQryOgg] CTE[NOME["OggComponenti"] PARTE[#RICORSI
VA]]
```

' Definizione della tabella di sessione mappa su collezione

'@DEFQUERYSESSIONTABLE IDQUERY[IdQryOgg] NOME["ElencoCheckin"] COLLEZIONE[ElencoOggetti]

Una volta definita la tabella di sessione, questa potrà essere utilizzata nella definizione della CTE, in questo caso della parte fissa della CTE:

'@DEFQUERYFROM IDQUERY[IdQryOggCTE1] TABELLA[AGREL]

'@DEFQUERYJOIN IDQUERY[IdQryOggCTE1] TABELLA
[[ElencoCheckin]] TABELLAPADRE[AGREL] REL[[ElencoCheckin].AREA=AGREL.ARE And [ElencoCheckin].PRODOTTO=AGREL.PRJ And [ElencoCheckin].PROGETTO=AGREL.PRJ And _
[ElencoCheckin].
TIPOOGGETTO=AGREL.TIB And [ElencoCheckin].CODICEOGGETTO=AGREL.NOB] OPERATOREJOIN["INNER"]

L'aspetto importante è che la tabella di sessione sia definita tramite DEFQUERYSESSIONTABLE prima del suo utilizzo. Questo permette all'ambiente di effettuare gli opportuni controlli , che il traduttore non può fare perché non conosce la struttura della tabella di sessione.

La tabella di sessione può essere usata sia nella CTE che nella query principale, anche contemporaneamente. Il suo popolamento avviene contestualmente alla GETQUERYDATA sulla query principale.

Definizione di un campo della query con clausola di raggruppamento

Le funzioni di aggregazione si utilizzano quando si vuole calcolare un totale (oppure una media, o valore massimo/minimo) di alcuni dati della query, raggruppati per un preciso insieme.

L'esempio mette in relazione le tabelle BLTESTATA, BLDRIGA e BLDART.
In ogni campo, all'interno del parametro ESPR viene specificata la funzione di aggregazione da applicare.

Esempio - Definizione di campo con clausola di raggruppamento:

BC
'@COLUMN ESPR[GROUPBY(BLDTTESTATA.IDD) AS [IdDocumento]] _ ESPR[GROUPBY(BLDTTESTATA.NUMDOC) AS [NumDocumento]] _ ESPR[SUM(BLDRIGA.IMPTOT) AS [TotaleImporto]] _ ESPR[GROUPBY(BLDART.DES) AS [DesArticolo]]

Il risultato in esecuzione

	IdDocumento	NumDocumento	TotaleImporto	DesArticolo
1	2	1	3500.0000	Articolo 001
2	3	2	5543.0000	Articolo 002
3	3	2	2500.0000	Articolo 003
4	4	3	1923.0000	Articolo 005
5	5	4	628.0000	Articolo 004
6	5	4	337.5000	Articolo 006
7	6	5	115.0000	Articolo 009
8	7	1	1000.0000	Articolo 001
9	8	6	1560.0000	Articolo 004
10	9	7	1275.0000	Articolo 010
11	10	8	1105.0000	Articolo 007
12	10	8	425.0000	Articolo 009
13	11	9	2500.0000	Articolo 015
14	12	10	2760.0000	Articolo 003
15	13	1	1360.0000	Articolo 011
16	13	1	4556.0000	Articolo 012
17	14	2	1035.0000	Articolo 014
18	14	2	780.0000	Articolo 015
19	15	3	2300.0000	Articolo 002
20	15	3	782.0000	Articolo 010
21	16	4	1960.0000	Articolo 004
22	16	4	1020.0000	Articolo 013
23	17	5	450.0000	Articolo 005
24	18	6	1380.0000	Articolo 001
25	18	6	4152.0000	Articolo 007

Il corrispondente SQL

I campi che fanno parte del raggruppamento (GROUPBY) vengono riutilizzati al fondo della stringa SQL per creare il vero e proprio raggruppamento in fase di estrazione dati. Il campo RilevanzaMagazzino, invece, non fa parte del raggruppamento, ma su di esso viene applicata la funzione di aggregazione SUM e viene scritta direttamente nella clausola SELECT.

SQL

```
SELECT BLDTestate.IDD AS [IdDocumento],
       BLDTestate.NrDoc AS [NumDocumento],
       SUM(BLDRighe.ImportoUdC) AS [TotaleImporto],
       BLDArticoli.DesArt AS [DesArticolo]
FROM BLDTestate
INNER JOIN BLDRighe ON BLDRighe.DBGruppo=BLDTestate.DBGruppo AND BLDRighe.IDD=BLDTestate.IDD
INNER JOIN BLDArticoli ON BLDArticoli.DBGruppo=BLDRighe.DBGruppo AND BLDArticoli.CodArt=BLDRighe.CodArt
WHERE BLDTestate.DBGruppo='GG'
GROUP BY BLDTestate.IDD, BLDTestate.NrDoc, BLDArticoli.DesArt
ORDER BY BLDTestate.IDD
```

Riferimenti

 **Query e Subquery**- Guida all'utilizzo

Per avere ulteriori dettagli all'utilizzo delle query e subquery consultare le seguenti guide:

- Guida allo sviluppo in BC, sezione "[Le query in BC](#)";
- Guida allo sviluppo in BC, sezione "[Le subquery in BC](#)"
- Guida allo sviluppo in BC, sezione "[Le common table expressions \(CTEs\) in BC](#)"

Utilizzo di INT all'interno delle specifiche per la creazione di query

INT è una funzione che effettua l'arrotondamento per difetto di un'espressione numerica.

Funzione INT- Modalità di utilizzo della funzione di arrotondamento per difetto di un numero

Funzione	Sintassi	Utilizzo
INT	INT(espr)	La funzione INT effettua l'arrotondamento di una espressione numerica

L'esempio che segue descrive come estrarre una data da una stringa definendo tale data come campo di tipo date.

Esempio:

BC

→ <data e ora>="2014092510:34"

→ nel formato AAAAMMGGHH:MM

' estrazione Anno

ANNO_CALC[4]=0

'@COMPUTEDCOLUMN

ESPR[IF [<data e ora><>" THEN _

INT(LEFT\$([<data e ora>],4)) _

ELSE 0 AS[AnnoCalc] VAR[ANNO_CALC] _

FMT[NUMERICO[4]]

'

' estrazione Mese

MESE_CALC[2]=0

'@COMPUTEDCOLUMN

ESPR[IF [<data e ora><>" THEN _

INT(MID\$([DataOraOpeCalc],5,2)) _

ELSE 0 AS[MeseCalc]

VAR[MESE_CALC] _

FMT[NUMERICO[2]]

'

' estrazione Giorno

GG_CALC[2]=0

'@COMPUTEDCOLUMN

ESPR[IF [DataOraOpeCalc]<>" THEN _

INT(MID\$([<data e ora>],7,2)) _

ELSE 0 AS[GiornoCalc] VAR[GG_CALC] _

FMT[NUMERICO[2]]

'

' creazione data Data

DATA_CALC&=0

'@COMPUTEDCOLUMN

ESPR[IF [<data e ora><>" THEN _

FNDATE([AnnoCalc],[MeseCalc],[GiornoCalc]) _

ELSE DATA_ZERO& AS[DataCalc] VAR[DATA_CALC&] _

FMT[DATA8]]

→ sono definiti tre campi calcolati che contengono l'anno, il mese e il giorno e poi un quarto campo calcolato con la data composta.

Nell'esempio di cui sopra la data composta viene valorizzata come: "25/09/2014"

Il corrispondente SQL

SQL

```
DECLARE @DATAEORA AS VARCHAR(100)
SET @DATAEORA='2014092510:34'
...
SELECT ...
    CASE WHEN ( @DATAEORA ) <> ''
        THEN CASE WHEN ISNUMERIC(LEFT((@DATAEORA ), 4)) > 0
            THEN CONVERT(INTEGER, LEFT((@DATAEORA ), 4))
            ELSE 0
        END
        ELSE 0
    END AS [AnnoCalc] ,
CASE WHEN (@DATAEORA ) <> ''
    THEN CASE WHEN ISNUMERIC(SUBSTRING((@DATAEORA ), 5, 2)) > 0
        THEN CONVERT(INTEGER, SUBSTRING((@DATAEORA ),
            5, 2))
        ELSE 0
    END
    ELSE 0
END AS [MeseCalc] ,
CASE WHEN (@DATAEORA ) <> ''
    THEN CASE WHEN ISNUMERIC(SUBSTRING((@DATAEORA ), 7, 2)) > 0
        THEN CONVERT(INTEGER, SUBSTRING((@DATAEORA ),
            7, 2))
        ELSE 0
    END
    ELSE 0
END AS [GiornoCalc] ,
CASE WHEN (@DATAEORA ) <> ''
    THEN CONVERT(DATETIME, CONVERT(VARCHAR(4), ( CASE
        WHEN (@DATAEORA ) <> ''
        THEN CASE
            WHEN ISNUMERIC(LEFT((@DATAEORA ),
                4)) > 0
            THEN CONVERT(INTEGER,
                LEFT((@DATAEORA ), 4))
            ELSE 0
        END
        ELSE 0
        END )) + '/'
    + CONVERT(VARCHAR(2), ( CASE WHEN (@DATAEORA ) <> ''
        THEN CASE WHEN ISNUMERIC(SUBSTRING((@DATAEORA ),
            5, 2)) > 0
        THEN CONVERT(INTEGER,
            SUBSTRING((@DATAEORA ), 5, 2))
            ELSE 0
        END
        ELSE 0
        END )) + '/'
    + CONVERT(VARCHAR(2), ( CASE WHEN (@DATAEORA ) <> ''
        THEN CASE WHEN ISNUMERIC(SUBSTRING((@DATAEORA ),
            7, 2)) > 0
        THEN CONVERT(INTEGER,
            SUBSTRING((@DATAEORA ), 7, 2))
            ELSE 0
        END
        ELSE 0
        END ))), 120)
    ELSE {D '1800-01-01'}
END AS [DataCalc]
FROM ...
WHERE ...
```

⚠ La funzione SQL effettua una conversione su campi di tipi Integer invece che su campi di tipo Decimal

Clausola CONTAINS

L'ambiente BC offre un supporto minimo di gestione del motore di indicizzazione Full Text Search, permettendo di indicare nella specifica '@DEFQUERYCONDGRID, il criterio 13-CONTAINS.

La clausola CONTAINS permette di ricercare su colonne di tipo testo, parole e frasi, in modo esatto o vago:

Una parola o una frase

Un prefisso di una parola o di una frase

Una parola accanto ad una parola specifica

Una parola generata per flessione, ad esempio le parole guida, guidare e guidando derivanti dalla radice guida

Sinonimo di una parola specifica utilizzando il thesaurus. Le parole alluminio e acciaio possono essere sinonimi della parola acciaio

Per poter impiegare la clausola CONTAINS è necessario soddisfare i seguenti requisiti:

Sia installato il pacchetto Full Text Search

Sia Attivo il servizio di indicizzazione Full Text Search

Sia definito un catalogo degli indici full text search

La colonna su cui si impiega la clausola Full Text Search sia coperta da un indice Full Text Search

Questi requisiti non sono ad oggi supportati direttamente dall'ambiente BC, pertanto si deve operare mediante gli strumenti standard messi a disposizione da Microsoft (SQL Management Studio).

L'ambiente prima di impiegare la clausola CONTAINS verifica quindi le condizioni:

Sul server sia installato il motore di indicizzazione Full Text Search

Sulla colonna alfanumerica sia presente un indice Full Text Search

Se le condizioni su indicate non sono soddisfatte, trasforma la clausola CONTAINS in una clausola LIKE.

Esempi di istruzioni SQL

Alcuni esempi di ricerche con la clausola CONTAINS:

Istruzione SQL	Descrizione
SELECT Name, ListPrice FROM Production.Product WHERE ListPrice = 80.99 AND CONTAINS(Name, 'Mountain');	tutti i prodotti il cui prezzo è \$80.99 e che includono la parola "Mountain"
SELECT Name FROM Production.Product WHERE CONTAINS(Name, ' "Mountain" OR "Road" ')	tutti i prodotti che contengono la frase "Mountain" o "Road"
SELECT Name FROM Production.Product WHERE CONTAINS(Name, ' "Chain*" ')	tutti i nomi di prodotto della colonna Name contenenti almeno una parola che inizia con il prefisso chain
SELECT Name FROM Production.Product WHERE CONTAINS(Name, '"chain*" OR "full*"')	tutte le descrizioni di categoria contenenti stringhe con il prefisso "chain" o "full"
SELECT Description FROM Production.ProductDescription WHERE CONTAINS(Description, 'bike NEAR performance')	tutti i nomi di prodotto in cui la parola bike è vicina alla parola performance
SELECT Description FROM Production.ProductDescription WHERE CONTAINS(Description, ' FORMSOF (INFLECTIONAL, ride) ')	tutti i prodotti con parole derivate da ride, ad esempio riding, ridden e così via
SELECT Description FROM Production.ProductDescription WHERE CONTAINS(Description, 'ISABOUT (performance weight (.8), comfortable weight (.4), smooth weight (.2) ')	tutti i nomi di prodotto contenenti la parola performance, comfortable o smooth. Ogni parola viene ponderata in modo diverso.

Esempio

Il seguente esempio impiega il criterio CONTAINS sul campo RagioneSociale1 per cercare la parola ROSSI.

Se sul database impiegato non è attivo il full text search engine oppure sul campo RagioneSociale1 non è presente alcun indice Full text search, l'ambiente trasforma la clausola CONTAINS in LIKE:

BC

```
IDQUERY[5]=0
'
'@DEFQUERY IDQUERY[IDQUERY] ERRORE
'

'@DEFQUERYFROM IDQUERY[IDQUERY] TABELLA[ANAGNR]
'@DEFQUERYPARTITION IDQUERY[IDQUERY] TABELLA["ANAGNR"] CAMPO["AGN"]
'@DEFQUERYCONDGRID IDQUERY[IDQUERY] TABELLA["ANAGNR"] CAMPO["RS1"]
CRITERIO[13] VALORE1["ROSSI"]
```

Condizioni non bilanciate

Nello sviluppo di query con condizioni dinamiche e complesse è possibile incontrare la necessità di indicare specifiche @WHERE la cui espressione definita non ha parentesi bilanciate o inizia con un operatore logico.

Esempio

```
BC

IF FL_COND_OR=0 AND FL_KEY_OR=0 THEN ' Normale
'@WHERE ESPR[DEMO1.VALIDO>=FIL_VALNDA AND
DEMO1.VALIDO<=FIL_VALNA]
ENDIF

IF FL_COND_OR=0 AND FL_KEY_OR=1 THEN ' In OR con la Chiave di ricerca
'@WHERE ESPR[(((DEMO1.VALIDO>=FIL_VALNDA AND
DEMO1.VALIDO<=FIL_VALNA))]
ENDIF

IF FL_COND_OR=1 THEN ' In OR con gli altri filtri
'@WHERE ESPR[OR (DEMO1.VALIDO>=FIL_VALNDA AND DEMO1.VALIDO
<=FIL_VALNA)] ' da/a
ENDIF
```

Questa rappresentazione rende la '@WHERE di difficile lettura e rende superfluo, in questo ambito, il controllo di coerenza dei parametri effettuato dal traduttore; risulta inoltre difficile ricercare situazioni simili all'interno dei sorgenti.

Per superare il problema è possibile utilizzare le seguenti parole chiave in sostituzione degli elementi non bilanciati

Valore	Descrizione
#SQLAND	Aggiunta dell'operatore logico AND all'interno di una condizione
#SQLOR	Aggiunta dell'operatore logico OR all'interno di una condizione
#SQLPAROP	Aggiunta di una parentesi aperta all'interno di una condizione
#SQLPARCL	Aggiunta di una parentesi chiusa all'interno di una condizione

utilizzando le parole chiave, le precedenti condizioni possono diventare

Esempio

```
BC
```

```
IF FL_COND_OR=0 AND FL_KEY_OR=0 THEN ' Normale
  '@WHERE ESPR[DEMO1.VALIDO>=FIL_VALNDA AND
DEMO1.VALIDO<=FIL_VALNA]
ENDIF

IF FL_COND_OR=0 AND FL_KEY_OR=1 THEN ' In OR con la Chiave di ricerca
  '@WHERE ESPR[#SQLPAROP (DEMO1.VALIDO>=FIL_VALNDA AND
DEMO1.VALIDO<=FIL_VALNA)]
ENDIF

IF FL_COND_OR=1 THEN      ' In OR con gli altri filtri
  '@WHERE ESPR[#SQLOR (DEMO1.VALIDO>=FIL_VALNDA AND DEMO1.VALIDO
<=FIL_VALNA)] ' da/a
ENDIF
```

Funzioni di windowing

Le funzioni di windowing permettono di eseguire dei calcoli su insieme di righe suddivise per chiave di rottura ed ordinate. Le funzioni sono suddivise in Funzioni di Aggregazione e Funzioni di Ranking.

Funzione aggregazione	Descrizione
COUNT	Calcola il numero di elementi
MIN	Determina il valore minimo
MAX	Determina il valore massimo
SUM	(solo campi numerici) Calcola la somma di tutti i valori
AVG	(solo campi numerici) Calcola la media di tutti i valori
STDEV	Deviazione standard
STDEVP	Deviazione standard sull'intera popolazione
VAR	Varianza
VARP	Varianza sull'intera popolazione

Funzione ranking	Descrizione
ROW_NUMBER()	Numerazione delle righe
RANK()	Rango della riga all'interno del gruppo (prima, seconda, ecc...). Due righe hanno lo stesso rango se per la chiave di ordinamento sono uguali
DENSE_RANK()	Come RANK, ma non permette il salto nella numerazione dei ranghi se due righe hanno lo stesso rango
NTILE(n)	Calcolo il numero di gruppi per un insieme di righe suddivise e ordinate. La numerazione parte da 1.

Le funzioni di Windowing se usate in sotto query (tabelle derivate) richiedono l'applicazione manuale dei filtri su gruppo e dominio se necessario

Esempi

BC

```
'@COLUMN ESPR[BLDART.IdMagPrincipale AS [f12] ]
'@COLUMN ESPR[BLDART.CostoStandard AS [f13] ]

' Definizione di una colonna con row_number()
'@COMPUTEDCOLUMN ESPR[ROW_NUMBER() AS [f1]] _
    PARTIZIONE[CRITERIO[BLDART.IdMagPrincipale]] ORDINAMENTO[BFT_ARTICOLI.CostoStandard]
    FMT[NUMERICO[9]]]

' Definizione di una colonna con MIN
'@COMPUTEDCOLUMN ESPR[MIN(BLDART.Prezzo) AS [f2] _
    PARTIZIONE[CRITERIO[BLDART.IdMagPrincipale]] FMT[NUMERICO[9]]]

' Definizione di una colonna con MAX
'@COMPUTEDCOLUMN ESPR[MAX(BLDART.Prezzo) AS [f3] _
    PARTIZIONE[CRITERIO[BLDART.IdMagPrincipale]] FMT[NUMERICO[9]]]

' Definizione di una colonna con AVG
'@COMPUTEDCOLUMN ESPR[AVG(BLDART.Prezzo) AS [f4] _
    PARTIZIONE[CRITERIO[BLDART.IdMagPrincipale]] [NUMERICO[9]]]

' Definizione di una colonna con COUNT()
'@COMPUTEDCOLUMN ESPR[COUNT(BLDART.Prezzo) AS [f5] _
    PARTIZIONE[CRITERIO[BLDART.IdMagPrincipale]] FMT[NUMERICO[9]]]

' Definizione di una colonna con SUM
'@COMPUTEDCOLUMN ESPR[SUM(BLDART.Prezzo) AS [f6] _
    PARTIZIONE[CRITERIO[BLDART.IdMagPrincipale]] FMT[NUMERICO[9]]]

' Definizione di una colonna con NTILE(n)
'@COMPUTEDCOLUMN ESPR[NTILE(3) AS [f7] _
    PARTIZIONE[CRITERIO[BLDART.IdMagPrincipale] ORDINAMENTO[BLDART.Prezzo /DESC] ]
    FMT[NUMERICO[9]]]

' Definizione di una colonna con RANK
'@COMPUTEDCOLUMN ESPR[RANK() AS [f8] _
    PARTIZIONE[CRITERIO[BLDART.IdMagPrincipale, BLDART.IdUdM] ORDINAMENTO[BLDART.Prezzo] ]
    FMT[NUMERICO[9]]]

' Definizione di una colonna con DENSE_RANK
'@COMPUTEDCOLUMN ESPR[DENSE_RANK() AS [f9] _
    PARTIZIONE[CRITERIO[BLDART.IdMagPrincipale, BLDART.IdUdM] ORDINAMENTO[BLDART.Prezzo]]
    FMT[NUMERICO[9]]]

' Definizione di una colonna con row_number() e alias
'@COMPUTEDCOLUMN ESPR[ROW_NUMBER() AS [f11]] _
    PARTIZIONE[CRITERIO[[f12] ORDINAMENTO[[f13]/DESC]] FMT[NUMERICO[9]]]
```

Riferimenti

['@COMPUTEDCOLUMN - Definizione campi calcolati](#)

Definizione di una query di servizio

Definizione tabella di servizio come tabella principale

La query è definita in una sub esterna e viene richiamata come tabella principale della query:

BC

```
' Definizione della query di servizio in una sub esterna
IDSrv[5]=0
CALL DefinizioneQueryEsterna(IDSrv [INOUT])

' Definizione della query principale in cui usare la query di servizio
IDMain=0

'@DEFQUERY IDQUERY[IDMain] NOME["IDMain"]
'@DEFQUERYTABLESERVICE IDQUERY[IDMain] FIGLIO[QUERYSERVICE[IDSrv] ALIAS["TEST1"]]
'@DEFQUERYJOIN IDQUERY[IDMain] TABELLA[BTFTOR02_EXT] TABELLAPADRE[[TEST1]] REL[BTFTOR02_EXT.IDEDI=
[TEST1].EDIT]

'@DEFQUERYPADRE IDQUERY[IDMain] ESPR[[TEST1].EDIT AS [CAMPO1] ]
'@DEFQUERYPADRE IDQUERY[IDMain] ESPR[BTFTOR02_EXT.EXT AS [CAMPO2] ]
'@DEFQUERYWHERE IDQUERY[IDMain] ESPR[BTFTOR02_EXT.NPR=6]
```

Definizione tabella di servizio come tabella in join

La query è definita in una sub esterna e viene richiamata in join con la tabella principale:

```
BC

' Definizione della query da usare come servizio
IDSrv[5]=0
'@DEFQUERY IDQUERY[IDSrv] NOME["IDMain"]
CALL DefinizioneQueryEsterna(IDSrv [INOUT])

' Definizione della query principale in cui usare la query di servizio
IDMain =0

'@DEFQUERY IDQUERY[IDMain] NOME["BFT_S_DEFQUERYTABLESERVICE_IDMAIN_250"]
'@DEFQUERYFROM IDQUERY[IDMain] TABELLA[BTFTOR01]
'@DEFQUERYTABLESERVICE IDQUERY[IDMain] FIGLIO[QUERYSERVICE[IDSrv] ALIAS["TEST1"]] PADRE[BTFTOR01] REL[[TEST1].EDIT=BTFTOR01.IDEDI]
'@DEFQUERYJOIN IDQUERY[IDMain] TABELLA[BTFTOR02_EXT] TABELLAPADRE[[TEST1]] REL[BTFTOR02_EXT.IDEDI=[TEST1].EDIT]

'@DEFQUERYPADRE IDQUERY[IDMain] ESPR[[TEST1].EDIT AS [CAMPO1] ]
'@DEFQUERYPADRE IDQUERY[IDMain] ESPR[BTFTOR02_EXT.EXT AS [CAMPO2] ]
'@DEFQUERYWHERE IDQUERY[IDMain] ESPR[BTFTOR02_EXT.NPR=6]
```

Strumenti a supporto della scrittura delle query

La scrittura delle query è supportata mediante:

- **template** in fase di creazione di una query, che consente di avere immediatamente disponibile come codice la struttura base per definire una query. Questa struttura base, poi, è da completare con l'indicazione delle tabelle, dei campi, dei filtri, ecc.;
- **wizard** in fase di editazione del sorgente, per la definizione visuale della query;
- **convertitore** delle query alla sintassi semplificata, che consente di aggiornare i sorgenti che utilizzano le query adattandone le specifiche '@xxxQUERYxxx alla nuova sintassi.

Riferimenti

[Wizard delle query](#)

[Conversione query in forma semplificata](#)

Wizard delle query

Il wizard delle query è uno strumento che permette di costruire e riportare in un sorgente BC le specifiche per la definizione di una query, tramite l'interfaccia grafica di Reporter.

Per generare una query il programmatore deve procedere con la sua definizione tramite diverse specifiche (alcune opzionali):

- '@DEFQUERY: per definire la query che si sta procedendo a creare.
- '@JOIN: per definire le n tabelle (con le rispettive relazioni) coinvolte nella query.
- '@COLUMN: per definire tutti i campi da utilizzare nella query.
- '@COMPUTEDCOLUMN: per definire tutti i campi calcolati da utilizzare nella query.
- '@WHERE: per definire i condizionamenti/filtri sulla query e sul raggruppamento.
- '@ORDERBY: per definire gli ordinamenti della query.
- '@DEFQUERYGROUP: per definire i raggruppamenti applicabili in fase di lettura (opzionale).

L'interfaccia grafica si astrae da questa definizione puntuale mediante specifiche, e permette di sfruttare le informazioni messe a disposizione dal prodotto, mediante i dizionari dati (relazioni, descrizioni, ecc.), per facilitare la creazione di quella che sarà, al termine dell'operazione, la query nel programma BC.

Soprattutto nel caso di query complesse, nelle quali intervengo più di una tabella, è impensabile, nella maggior parte dei casi, riuscire a ricordarsi tutti i parametri di tutte le specifiche, tutte le tabelle, i campi e tutte le relazioni.

Il wizard permette di eliminare problemi di questo tipo facilitando la navigazione tra le tabelle del database, offrendo le relazioni di partenza definite in fase di creazione dei dfx (dando la possibilità, inoltre, di crearne delle nuove), di studiare le composizioni delle tabelle (visualizzando le descrizioni di quest'ultime e di ogni campo definito).

Il wizard delle query utilizza la stessa interfaccia di Reporter disegnatore.

Consente sia di creare una nuova query, sia di aprirne una esistente.

Una volta composta la query con le sue tabelle, campi, filtri, eventuali ordinamenti, occorre salvarla



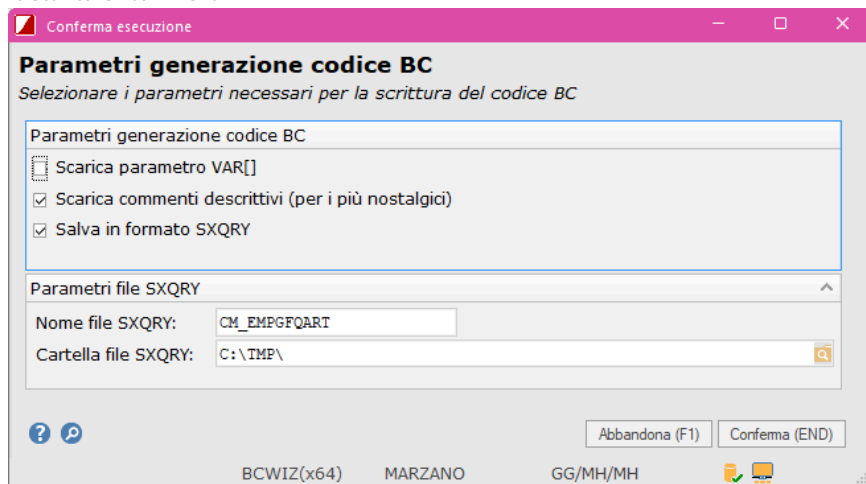
per far sì che sia prodotto il codice BC da inserire sul sorgente.

A partire da BC/X v.31.4, il wizard consente di scaricare la query BC con specifiche diverse, a seconda della versione di traduttore correntemente in uso per il sorgente

fino al traduttore 31.1x, la query BC viene scaricata con le "vecchie" specifiche (@DEFQUERYTABLE, @DEFQUERYFIELD_LV etc)

a partire dal traduttore 31.2, la query BC viene scaricata con le nuove specifiche (@DEFQUERYJOIN, DEFQUERYCOLUMN etc)

In fase di salvataggio, è inoltre possibile definire alcune opzioni di composizione del codice BC, per utilizzare ad esempio il parametro VAR o scaricare i commenti



Riferimenti

[Strumenti a supporto della scrittura delle query](#)

[Come si utilizza](#)

[Quando si utilizza](#)

[Funzionalità](#)

[Limitazioni](#)

Funzionalità

Il wizard prevede varie funzionalità, alcune condizionano l'interfaccia utente altre, collocate nella sezione "Avanzate", condizionano il risultato della query.

Segue l'elenco delle funzionalità previste:

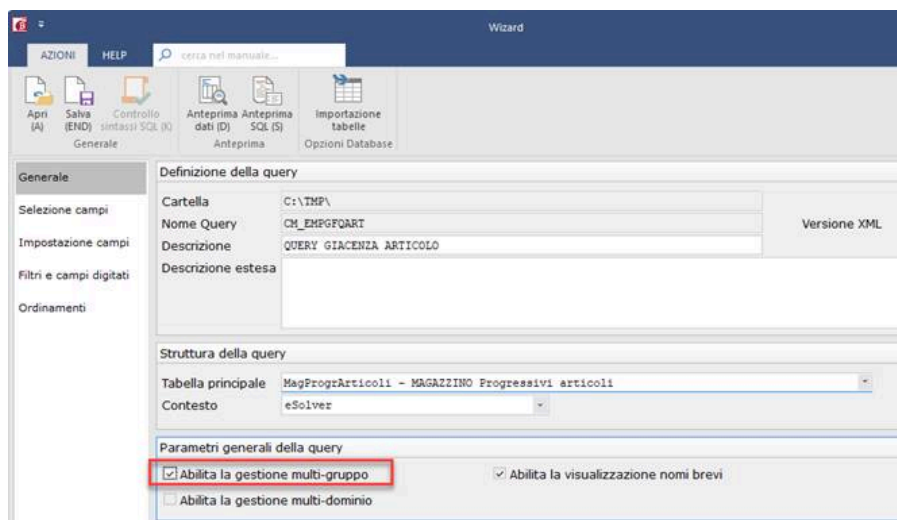
- interfaccia:
 - abilita la gestione della query MULTIGRUPPO
 - abilita la visualizzazione dell'albero delle relazioni nella selezione dei campi della query
 - abilita la visualizzazione dei nomi brevi nella costruzione della query
- avanzate: Distinct (Applica alla selezione dei campi la clausola DISTINCT)
- salvataggio della query
- modifica di una query in un sorgente richiamando una query precedentemente salvata.

Abilita la gestione della query MULTIGRUPPO

In genere l'esecuzione di una query su tabelle di gruppo opera una selezione implicita sui dati utilizzando la variabile di ambiente GR\$ (gruppo dati in esecuzione). Questo significa che è possibile ottenere come risultato i dati di un gruppo per volta perché la selezione sulla colonna "Gruppo archivi" (nella definizione della tabella corrisponde alla colonna speciale DBGruppo) non è disponibile.

In alcuni casi però serve ottenere come risultato i dati di più gruppi e per fare questo è necessario intervenire nella definizione della query abilitando la funzione MULTIGRUPPO.

Abilitata la funzione, il wizard rende disponibile il campo *GR* (DbGruppo), come campo della query, che si potrà utilizzare come campo normale sia per assegnarlo ad una variabile ('@COLUMN) che utilizzarlo come filtro ('@WHERE).



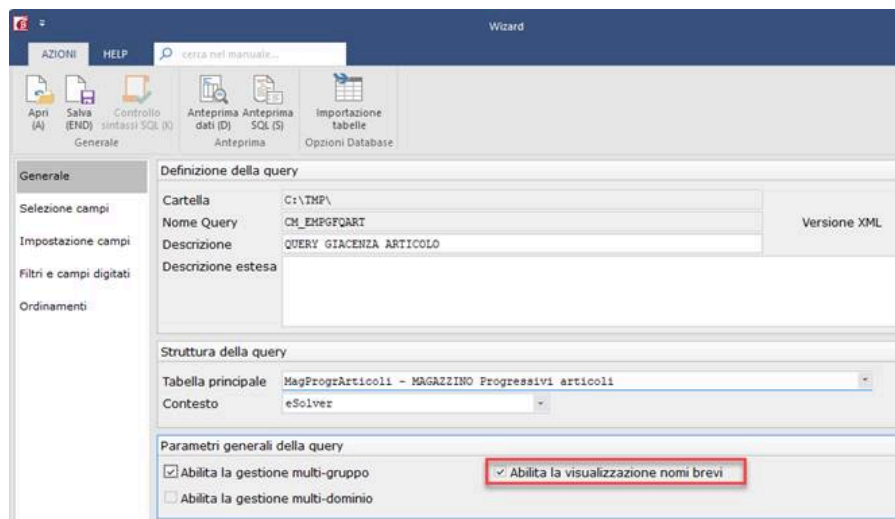
Abilita la visualizzazione nomi brevi

Tutti gli strumenti di creazione di una query (come SQL Server Manager) utilizzano, per la navigazione tra tabelle e campi, la nomenclatura fisica dell'oggetto, quello che noi definiamo Nome esteso.

Il Wizard delle query fornisce la possibilità di utilizzare sia i nomi estesi che la combinazione Nomi brevi/Nomi estesi.

Il nome breve corrisponde sia al nome interno (nome del descrittore) della tabella che al nome interno del campo (nome del campo nel descrittore) facilitando il compito ai programmatori che hanno più familiarità con questa nomenclatura.

L'opzione è abilitata d'ufficio.



Salvataggio della query

Quando si preme il pulsante <Salva (END)> viene richiamata la funzione di salvataggio del wizard.

Compito di questa funzione è tradurre le selezioni operate utilizzando l'interfaccia grafica in istruzioni del linguaggio BC che verranno collocate nel sorgente BC nella posizione sottostante il cursore al momento in cui è stata premuta la combinazione <Ctrl+W> (richiamo del wizard).

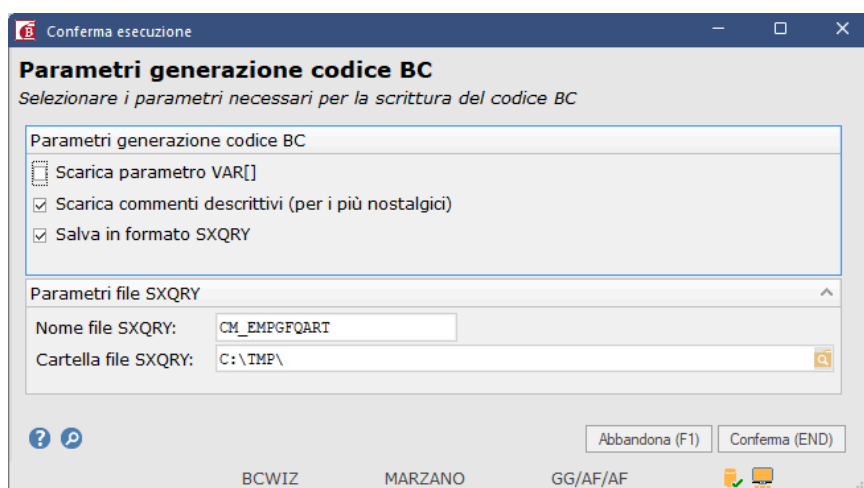
Il salvataggio della query consente di:

1. riportare l'elenco delle specifiche '@DEFQUERYxxx nel sorgente BC (il lavoro svolto nel wizard viene perso);
2. riportare l'elenco delle specifiche '@DEFQUERYxxx nel sorgente BC e salvare il lavoro svolto nella sessione del wizard in un file esterno con estensione <nome query>.SXQRY (il lavoro svolto nel wizard viene preservato).

La differenza è sostanziale e consiste in:

- nel **caso 1**, il sorgente BC contiene le specifiche di definizione della query, ma se devo apportare una modifica lo devo fare agendo sul sorgente BC perché se si volesse farlo con il wizard bisognerebbe ripetere tutto da capo;
- nel **caso 2**, il sorgente BC contiene le specifiche di definizione della query, ma se devo apportare una modifica lo posso fare richiamando il wizard. Cancello la porzione di sorgente precedente, richiamo il wizard e apro il file esterno <nome query>.SXQRY salvato in precedenza.

A tal fine, viene presentata la videata di opzioni seguente:



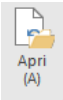
Dove l'opzione "Salva in formato SXQRY" è proprio destinata al salvataggio del file esterno.

Modifica di una query in un sorgente richiamando un query precedentemente salvata

Se si deve modificare una query in un sorgente sfruttando le funzionalità offerte dal wizard è necessario che al momento del salvataggio si sia scelta la strada del salvataggio del file della query (un file con estensione .SXQRY) e che ci sia corrispondenza tra il contenuto del sorgente BC e la query salvata (se nel frattempo il sorgente BC è stato modificato non c'è garanzia che il file SXQRY contenga l'ultima versione della query salvata).

È possibile procedere come segue:

- 1. editate il sorgente da modificare;
- 2. premete <Ctrl+W> (richiamo del wizard);
- 3. confermate o variate la Direttiva dei descrittori (se richiesta);
- 4. premete <Costruisci Query (F7)>;



- 5. quando viene proposta la videata principale del wizard premete il pulsante ;
- 6. selezionate la query salvata da quelle proposte nell'elenco.

A questo punto è possibile eseguire le variazioni desiderate e ripetere il salvataggio della query.
La query può essere salvata nuovamente su file esterno .SXQRY (sovrascrivendo quello precedente o creandone uno nuovo).
Le specifiche del linguaggio BC vengono aggiunte al sorgente in editazione, quelle precedenti devono essere rimosse manualmente.

Funzionalità di dettaglio Wizard delle query.

Funzionalità	Descrizione
TABELLE	Per definire le tabelle che fanno parte della query, è possibile utilizzare tutti i descrittori presenti all'interno della direttiva dfx richiesta in avvio del programma.
RELAZIONI	Le relazioni che legano due tabelle di una query possono essere di due tipi: 1 - Automatica 2 - Dinamica. La relazione automatica è la relazione che viene definita all'interno del descrittore in fase di creazione di quest'ultimo e visibile nel wizard (selezione campi) nel riquadro a sinistra perché collegata alla tabella principale definita. La relazione dinamica è una relazione costruita sul momento dal programmatore stesso, modificabile e personalizzabile all'interno del wizard.
DISTINCT	Utilizzo clausola DISTINCT per l'eliminazione delle righe duplicate all'interno del resul-set finale della query.
CAMPI CALCOLATI	È possibile definire la creazione di campi calcolati complessi con l'utilizzo di funzioni, costanti e altri campi della tabella.
SELEZIONE CAMPI	È possibile selezionare qualsiasi campo di qualsiasi tabella relazionata con la tabella definita come principale.
SELEZIONE FILTRI	È possibile definire, uno qualsiasi dei campi della tabella, come campo filtro o come filtro per raggruppamento.
RAGGRUPPAMENTI	È possibile attribuire, ad ogni campo, una clausola di raggruppamento: SUM, COUNT, AVG, MIN, MAX e GROUPBY.
FILTRI	È possibile definire filtri semplice (Criterio = valore) o filtri complessi utilizzando formule definite con funzioni, campi o costanti.
ORDINAMENTI/ RAGGRUPPAMENTI	È possibile creare più di un ordinamento (e quindi livelli di rottura) a seconda delle esigenze del programmatore.

Riferimenti

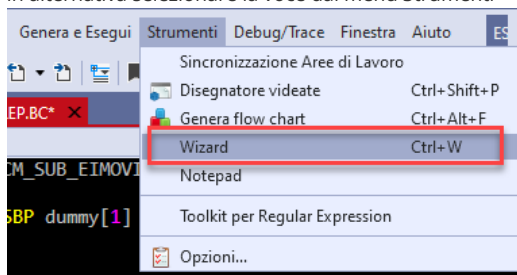
- [Wizard delle query](#)
- [Come si utilizza](#)
- [Quando si utilizza](#)
- [Limitazioni](#)
- [Sintassi](#)

Come si utilizza

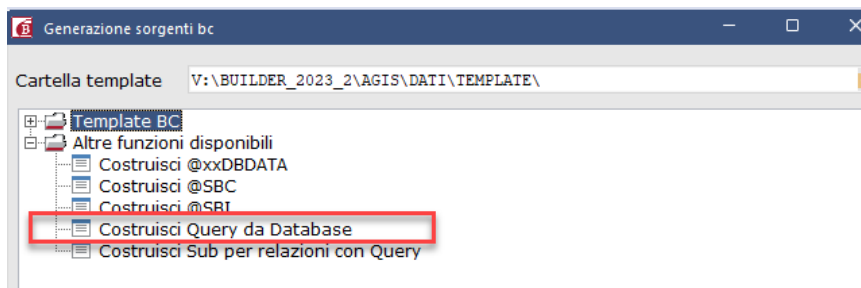
Richiamo del wizard dall'interno dell'editor BCDevelop

- Per richiamare il wizard delle query si procede come segue:
- 1. editare il sorgente BC interessato
 - 2. posizionarsi nella parte del sorgente in cui si vuole vengano inserite le specifiche BC relative alla definizione della query;

3. premere la combinazione di tasti <Ctrl+W> (W è sinonimo di Wizard) che provvede al richiamo del programma BCWiz... o in alternativa selezionare la voce dal menu Strumenti



4. quando compare la videata, selezionare l'opzione Costruisci query da database



premere il pulsante <Costruisci Query (F7)>.

Si ritroverà nella videata principale del wizard in cui potrà procedere con la generazione della query.

Processo di creazione una query

Il passo successivo al richiamo del wizard nel sorgente è la creazione della query mediante interfaccia grafica.

Si hanno due possibilità:

- **partenza da zero:** la query deve essere creata compilando in sequenza le videata "Generale", "Selezione campi", "Definizione opzioni", "Filtri", ecc.;
- **variazione o recupero query da file:** la query è già stata salvata in precedenza: si preme il bottone  per richiamare la query salvata e procedere con le sole modifiche necessarie.

Una volta composta graficamente la query è possibile procedere come segue:

1. cliccare su  per verificare il comando SQL generato in anteprima;
2. il bottone  consente di sperimentare in anteprima il risultato dell'esecuzione (viene visualizzata una griglia con l'elenco dei record elaborati);
3. cliccare su  per salvare la query e riportare nel sorgente le specifiche BC generate.

Riferimenti

[Wizard delle query](#)

[Quando si utilizza](#)

[Funzionalità](#)

[Limitazioni](#)

[Sintassi](#)

Quando si utilizza

Il wizard delle query è utile, per non dire indispensabile, ogni qual volta si vuole procedere con la creazione di una query più o meno complessa. Per contro, nel caso si debba generare una query semplice, per la quale bisogna scrivere poche e semplici istruzioni, il wizard diventa uno strumento opzionale.

Sono punti a favore del wizard:

- **l'interfaccia grafica:** in cui vengono evidenziate le relazioni tra tabelle, campi calcolati, filtri con formule;
- **l'astrazione dalle specifiche del linguaggio BC:** non si obbliga il programmatore a ricerche sia nel Manuale di riferimento del linguaggio BC (BCRIF) sia nei descrittori per i nomi delle tabelle, dei campi, ecc.

Processo di creazione - Creazione di una query:

Caso	Descrizione	Wizard
Query semplice	- Query da una tabella, senza relazioni. - Query tra due tabelle con relazioni semplici.	Opzionale
Query complessa	- Query tra più tabelle con relazioni complesse. - Query con diversi campi e campi calcolati. - Query con condizionamenti su query e su raggruppamento. - Query con condizionamenti con formule	Consigliato
Query derivate da file	- Query salvate precedentemente su file SXQRY o derivate da Report	Consigliato

Riferimenti

[Wizard delle query](#)

[Come si utilizza](#)

[Funzionalità](#)

[Limitazioni](#)

[Sintassi](#)

Limitazioni

Le limitazioni del wizard delle query riguardano due aspetti:

- Reverse Engineering da specifiche nel sorgente BC.

Reverse Engineering

Non è possibile, partendo dal sorgente BC, risalire allo schema grafico all'interno del wizard delle query. Per sopperire a questa limitazione il wizard offre la possibilità di salvare su file la query che si sta andando a generare e a richiamarla in un tempo successivo per avere almeno una base di partenza (escludendo le modifiche fatte direttamente sul sorgente) per implementare la query precedentemente scaricata.

Riferimenti

[Wizard delle query](#)

[Come si utilizza](#)

[Quando si utilizza](#)

[Funzionalità](#)

[Sintassi](#)

Sintassi

L'esempio permette di visualizzare il sorgente scaricato dal wizard, in base ad una query che coinvolge 4 tabelle.

Sorgente BC ottenuto dal Wizard delle Query

Le tabelle coinvolte sono:

Tabella	Descrizione
EMMAG	Movimenti di magazzino
EARTIC	Articoli: Anagrafiche
ECFA	Clienti/fornitori: anagrafiche

ANAGNR	Anagrafiche: dati generali
--------	----------------------------

Sorgente complessivo scaricato dal wizard:

BC

```

IDQUERY[5]=0
'@DEFQUERY IDQUERY[IDQUERY]

' Tabella EMMAG
'@DEFQUERYFROM IDQUERY[IDQUERY] TABELLA[EMMAG]

' Relazione tra EARTIC (EQArtAnagrafiche) - Articoli: Anagrafiche
' e EMMAG (EQMovMag) - Movimenti: Magazzini
'@DEFQUERYJOIN IDQUERY[IDQUERY] TABELLA[EARTIC] TABELLAPADRE[EMMAG] _
REL[EARTIC.DIT = EMMAG.DIT AND EARTIC.COD = EMMAG.ESPR[ [MACROCOD] ]]

' Relazione tra ECFA (EQCliForAn) - Clienti/fornitori: Anagrafiche
' e EMMAG (EQMovMag) - Movimenti: Magazzini
'@DEFQUERYJOIN IDQUERY[IDQUERY] TABELLA[ECFA] TABELLAPADRE[EMMAG] _
REL[ECFA.DIT = EMMAG.DIT AND
ECFA.ICCF = EMMAG.ICCF AND ECFA.IPART = EMMAG.IPART]

' Relazione tra ANAGNR (EQAnDatiGenerali) - Anagrafiche: Dati generali
' e ECFA (EQCliForAn) - Clienti/fornitori: Anagrafiche
'@DEFQUERYJOIN IDQUERY[IDQUERY] TABELLA[ANAGNR] TABELLAPADRE[ECFA] _
REL[ANAGNR.AGN = ECFA.AGN]

'@DEFQUERYCOLUMN IDQUERY[IDQUERY] _
ESPR[EMMAG.IDD AS [IDDDocumento]] VAR[MMAIDD] _
ESPR[EMMAG.DIT AS [CodDitta]] VAR[MMADIT] _
ESPR[EMMAG.DRG AS [RegisData]] VAR[MMADRG&] _
ESPR[EMMAG.NRG AS [RegisNumero]] VAR[MMANRG] _
ESPR[EMMAG.CAR AS [CodArticolo]] VAR[MMACAR$] _

MACROCOD$[5]="'"
'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDQUERY] _
ESPR[LEFT$(EMMAG.CAR,5) AS [MACROCOD] VAR[MACROCOD$]]

'@DEFQUERYCOLUMN IDQUERY[IDQUERY] _
ESPR[EARTIC.DES AS [DesArticolo]] VAR[ARDES$] _
ESPR[ECFA.AGN AS [CodAnagrafica]] VAR[CFAGN] _
ESPR[ANAGNR.RS1 AS [RagioneSociale1]] VAR[ANRS1$] _
ESPR[ANAGNR.RS2 AS [RagioneSociale2]] VAR[ANRS2$]

' Filtri (Where)
'@DEFQUERYWHERE IDQUERY[IDQUERY] _
ESPR[BETWEEN(EMMAG.DIT,0,99999)>0] _
ESPR[ AND BETWEEN(EMMAG.DRG,0,31122099)>0 ]

' Ordinamento
'@DEFQUERYORDERBY IDQUERY[IDQUERY] IND[1] _

```

ESPR[EARTIC.DES/ASC]

Definizione iniziale della query

La specifica @DEFQUERY inizializza la query che si sta andando a definire.

BC
IDQUERY[5]=0
'@DEFQUERY IDQUERY[IDQUERY]

Definizione tabelle coinvolte

La prima DEFQUERYFROM stabilisce la tabella principale della query.

Le successive DEFQUERYJOIN, invece, definiscono le nuove tabelle e le relazioni con quella a cui sono legate.

Tabella	Tabella Padre	Relazione	Descrizione
EMMAG			Tabella principale della query
EARTIC	EMMAG	EARTIC.DIT = EMMAG.DIT AND EARTIC.COD = [[MACROCOD]]	EARTIC collegata direttamente alla tabella principale
ECFA	EMMAG	ECFA.DIT = EMMAG.DIT AND ECFA.ICCF = EMMAG.ICCF AND ECFA.IPART = EMMAG.IPART	ECFA collegata direttamente alla tabella principale
ANAGNR	ECFA	ANAGNR.AGN = ECFA.AGN	La tabella ANAGNR ha una relazione di secondo livello con la tabella ECFA. Di secondo livello perché ECFA ha sua volta è relazionata alla tabella principale

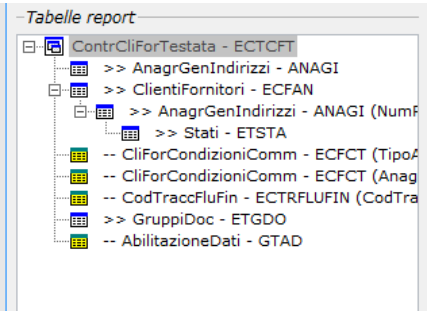
Tabella - EARTIC (Figlio) collegata alla tabella principale EMMAG (Padre) attraverso la relazione:

EARTIC.DIT = EMMAG.DIT e COD = Campo calcolato MACROCOD

BC
' Tabella EMMAG '@DEFQUERYFROM IDQUERY[IDQUERY] TABELLA[EMMAG]
' Relazione tra EARTIC (EQArtAnagrafiche) - Articoli: Anagrafiche ' e EMMAG (EQMovMag) - Movimenti: Magazzini '@DEFQUERYJOIN IDQUERY[IDQUERY] TABELLA[EARTIC] TABELLAPADRE[EMMAG] _ REL[EARTIC.DIT = EMMAG.DIT AND EARTIC.COD = EMMAG.ESPR[[MACROCOD]]]
' Relazione tra ECFA (EQCliForAn) - Clienti/fornitori: Anagrafiche ' e EMMAG (EQMovMag) - Movimenti: Magazzini '@DEFQUERYJOIN IDQUERY[IDQUERY] TABELLA[ECFA] TABELLAPADRE[EMMAG] _ REL[ECFA.DIT = EMMAG.DIT AND ECFA.ICCF = EMMAG.ICCF AND ECFA.IPART = EMMAG.IPART]
' Relazione tra ANAGNR (EQAnDatiGenerali) - Anagrafiche: Dati generali ' e ECFA (EQCliForAn) - Clienti/fornitori: Anagrafiche '@DEFQUERYJOIN IDQUERY[IDQUERY] TABELLA[ANAGNR] TABELLAPADRE[ECFA] _ REL[ANAGNR.AGN = ECFA.AGN]

Corrisponde nel wizard

L'albero permette la visualizzazione delle relazioni inserite all'interno della query:



Definizione campi utilizzati

Il terzo blocco riguarda la definizione dei campi utilizzati all'interno della query. Oltre ai campi della tabella vengono definiti anche i campi calcolati o digitati:

Il campo MACROCOD corrisponde ai primi 5 caratteri del campo EMMAG.CAR

BC
'@DEFQUERYCOLUMN IDQUERY[IDQUERY] _ ESPR[EMMAG.IDD AS [IDDocumento]] VAR[MMAIDD] _ ESPR[EMMAG.DIT AS [CodDitta]] VAR[MMADIT] _ ESPR[EMMAG.DRG AS [RegisData]] VAR[MMADRG&] _ ESPR[EMMAG.NRG AS [RegisNumero]] VAR[MMANRG] _ ESPR[EMMAG.CAR AS [CodArticolo]] VAR[MMACAR\$] MACROCOD\$[5]="" '@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDQUERY] _ ESPR[LEFT\$(EMMAG.CAR,5) AS [MACROCOD] VAR[MACROCOD\$] '@DEFQUERYCOLUMN IDQUERY[IDQUERY] _ ESPR[EARTIC.DES AS [DesArticolo]] VAR[ARDES\$] _ ESPR[ECFA.AGN AS [CodAnagrafica]] VAR[CFAGN] _ ESPR[ANAGNR.RS1 AS [RagioneSociale1]] VAR[ANRS1\$] _ ESPR[ANAGNR.RS2 AS [RagioneSociale2]] VAR[ANRS2\$]

Corrisponde nel wizard

La griglia visualizza l'elenco dei campi utilizzati all'interno della query.

Pos	Tabella	Nome Campo	Nome Breve	Selezione	Descrizione
136	CltForCondizioniComm	CodCondisPagamento	CPG	☐	CODICE CONDIZIONE DI PAGAMENTO
137	CltForCondizioniComm	CltIdListinoPriorita	IDLISPR	☐	IDENTIFICATORE LISTINO PRIORITARIO
138	CltForCondizioniComm	CltIdListinoOrdinari	IDLISORD	☐	IDENTIFICATORE LISTINO ORDINARIO
139	CltForCondizioniComm	CltIdListinoTariffar	IDLISTAR	☐	IDENTIFICATORE LISTINO TARIFFARIO
140	CltForCondizioniComm	CltIdPoliticaSconti	IDPOLSC	☐	IDENTIFICATORE POLITICA SCONTI
141	*Campo calcolato*	CondPagFattCanoni_C	CondPagFattCar	☒	Condizione Di Pagamento Per Fatturazione canoni
142	*Campo calcolato*	CondPagFattPrest_C	CondPagFattPre	☒	Condizione Di Pagamento Per Fatturazione Prestazi
143	*Campo calcolato*	ListinoPrioritario_C	ListinoPriorit	☒	Listino Prioritario: Identific
144	*Campo calcolato*	ListinoOrdinario_C	ListinoOrdinar	☒	Listino Ordinario: Identificatore
145	*Campo calcolato*	ListinoTariffario_C	ListinoTariffe	☒	Tariffario: Identificatore
146	*Campo calcolato*	IdPoliticaSconti_C	IdPoliticaScor	☒	Identificatore Politica Sconti
147	ClientiFornitori	ANFT_CessioneCltPriv	FCCLIPRIV	☐	CESSIONE A CLIENTE PRIVATO (0-NO;1-SI)
148	ClientiFornitori	ANFT_RilTraccFlussiF	TRFLUFIN	☐	RILEVANTE PER TRACCIABILITA' FLUSSI FINANZIARI (0-NO;1-SI)
149	ClientiFornitori	ANFT_EscludiDeCommO	FESCOPIVA	☐	ESCLUDE DA COMUNICAZIONE OPERAZIONI IVA (0-NO;1-SI)

Definizione condizionamenti

Le condizione definite nella query vengono interamente scaricate all'interno di un'unica DEFQUERYWHERE. Ogni condizione è legata da un operatore AND o OR.

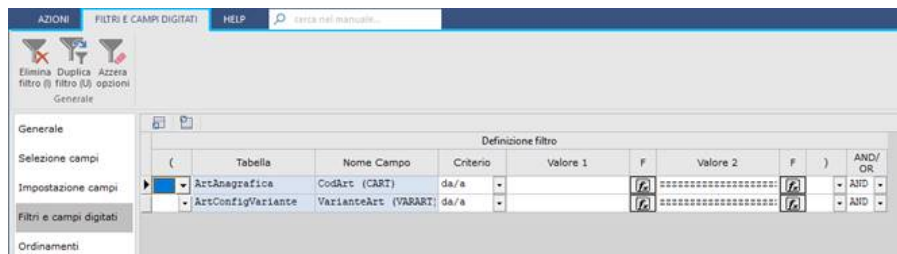
BC

```
' Filtri (Where)

'@DEFQUERYWHERE IDQUERY[IDQUERY] _
    ESPR[BETWEEN(EMMAG.DIT,0,99999)>0 ] _
    ESPR[ AND BETWEEN(EMMAG.DRG,0,31122099)>0 ]
```

Corrisponde nel wizard

La griglia visualizza l'elenco dei filtri impostati all'interno della query.



Definizione ordinamenti

All'interno del parametro ESPR della DEFQUERYORDERBY, è possibile elencare tutti i campi che fanno parte dell'ordinamento.

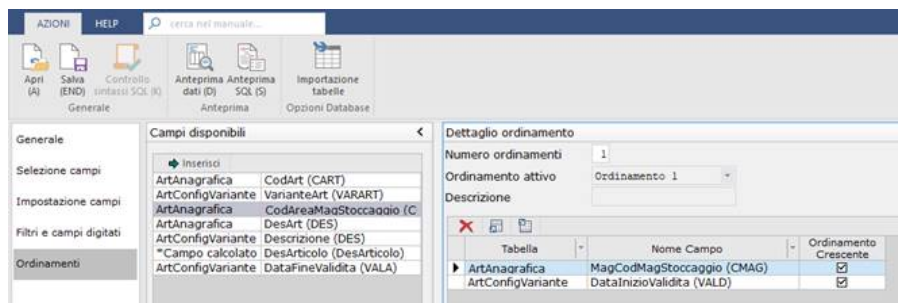
BC

```
' Ordinamento

'@DEFQUERYORDERBY IDQUERY IND[1] _
    ESPR[EARTIC.DES/ASC]
```

Corrisponde nel wizard

La griglia visualizza l'elenco dei campi selezionati per ordinare i dati estratti.



Riferimenti

[Wizard delle query](#)

[Come si utilizza](#)

[Quando si utilizza](#)

[Funzionalità](#)

[Limitazioni](#)

Le subquery in BC

Cos'è una subquery

Le subquery consentono di nidificare query all'interno di un'altra, nonché per recuperare dati che sarebbe impossibile costruire senza ricorrere ad una elaborazione che proceda a passaggi successivi, facendo eventualmente uso di tabelle temporanee.

Esistono due tipi di subquery:

- subquery correlate, cioè una query che dipende e utilizza riferimenti della query esterna;
- subquery non correlate, cioè query indipendenti dalla query esterna

La dipendenza si evince nell'utilizzo di campi della query esterna nella formulazione della sotto query, ad esempio per definirne i filtri

Entrambi i tipi di subquery possono restituire:

- un unico valore (scalare), da usare all'interno di una espressione (**Scalar subqueries**)
- più di un valore, da usare dove si può indicare una tabella (**Table subqueries**)

Le subquery sono a tutti gli effetti delle SELECT utilizzate all'interno di un'altra SELECT e nella sintassi del linguaggio SQL devono essere espresse sempre all'interno di parentesi tonde in modo da differenziarsi dalla query esterna.

Scalar subqueries

Sono query in grado di restituire un unico valore (detto appunto valore scalare), la si può usare in qualsiasi punto sia possibile utilizzare il nome di una colonna. Esempi di impiego

- Calcolo di un valore, tipicamente aggregato, dipendente da uno o più valori dal record corrente della query esterna.
- Filtro di record

Utilizzando una Scalar subquery se si tenta di estrarre da una tabella più valore SQL produce un errore come :

"Subquery returned more than 1 value" oppure "Operand should contain 1 column(s)".

Table subqueries

Con questa categoria si identificano interrogazioni annidate nella clausola FROM o JOIN. Ritornano genericamente più righe e più colonne.

Definizione

In BC le sottoquery, come vederemo in questi esempi, sono definite mediante un blocco:

BC
<pre>'@DEFSUBQUERY QUERY[SottoQuery] QUERYPADRE[Query] '@ENDQUERY</pre>

- **DEFSUBQUERY** : definisce l'inizio della definizione della sottoquery. I parametri essenziali sono:
 - QUERY : definisce l'identificatore della sottoquery, che verrà impiegato successivamente per riferire la sottoquery nella definizione della query principale
 - QUERYPADRE è l'identificatore della query padre, che può essere omissso se il blocco `@DEFSUBQUERY/@ENDQUERY` è contenuto in un altro blocco `@DEFSUBQUERY/@ENDQUERY`
- **ENDQUERY** : definisce la fine della definizione della sottoquery

All'interno del blocco `@DEFSUBQUERY/@ENDQUERY` si procede alla definizione della sotto query usando le stesse specifiche impiegate per la query: `@FROM`, `@JOIN...`:

BC

```
'@DEFSUBQUERY QUERY[TotaleRigheArt]

'@FROM TABELLA[BLDTESTATA]
'@JOIN TABELLA[BLDRIGA] TABELLAPADRE[BLDTESTATA] OPERATOREJOIN["INNER"] REL[BLDRIGA.IDD=BLDTESTATA.IDD]

'@COMPUTEDCOLUMN ESPR[COUNTDISTINCT(BLDRIGA.IDDOC) AS [Totale]]

'@WHERE ESPR[BLDRIGA.CODART=BLDART.COD ]

'@ENDQUERY
```

Impiego

Subquery scalare

Una sottoquery definita mediante @DEFSUBQUERY/@ENDQUERY e che soddisfa i requisiti:

- Torna una sola colonna
- Torna una sola riga

Può essere impiegata nella definizione di una espressione della query principale, usando la pseudo-funzione SUBQ passando come parametro l'id della sottoquery. L'ambiente BC espande la pseudo funzione SUBQ con la sua definizione SQL

```
BC - BLDOBJQRY_SUBQUERY_SCA_DEFINIZIONE

'@DEFQUERY QUERY[Query] NOME["BLDOBJQRY_SUBQUERY_SCA_DEFINIZIONE_15"]

'@FROM TABELLA[BLDART]

'@DEFSUBQUERY QUERY[TotaleRigheArt]

'@FROM TABELLA[BLDTESTATA]
'@JOIN TABELLA[BLDRIGA] TABELLAPADRE[BLDTESTATA] OPERATOREJOIN["INNER"] REL[BLDRIGA.IDD=BLDTESTATA.IDD]

'@COMPUTEDCOLUMN ESPR[COUNTDISTINCT(BLDRIGA.IDDOC) AS [Totale]]

'@WHERE ESPR[BLDRIGA.CODART=BLDART.COD ]

'@ENDQUERY

'@COLUMN ESPR[BLDART.COD AS [Codice]] _
          ESPR[BLDART.DES AS [Descrizione]]

'@COMPUTEDCOLUMN ESPR[SUBQ(TotaleRigheArt) AS [NumeroDoc] FMT[NUMERICO[6]]]
'@WHERE ESPR[SUBQ(TotaleRigheArt)>3]

'@ENDQUERY
```

Il codice è strutturato nel seguente modo:

- Definizione query
 - Definizione tabella from per la query principale
 - Definizione sottoquery **TotaleRigheArt**
 - Definizione tabella from/join della sottoquery
 - Definizione conteggio del numero di fatture che menzionano l'articolo
 - Utilizzo della sotto query per la definizione del recordset della query. In particolare la definizione del campo calcolato **[NumeroDoc]** impiega la funzione **SUBQ** cui è stata passata l'identificatore della sottoquery **TotaleRigheArt**

La sottoquery scalare è correlata in quanto nella sua definizione usa il campo codice articolo nel filtro, campo proveniente dalla query principale. Come regola generale per ogni campo presente in una sottoquery, l'ambiente:

- 1. Ricerca la tabella del campo nella sottoquery
- 2. Se non è presente ricerca la tabella nella query che contiene la sottoquery
- 3. Se non presente procede alla query padre

 In generale, quando si utilizzano i campi della subquery all'interno della query è necessario utilizzare il campo calcolato.

Il codice precedente produce il seguente codice SQL:

SQL

```
SELECT BLDArticoli.CodArt AS [Codice],
       BLDArticoli.DesArt AS [Descrizione],

       (SELECT COUNT(DISTINCT BLDRighe.IDD) AS [Totale]
        FROM BLDTestate AS [BLDTestate]
        INNER JOIN BLDRighe AS [BLDRighe] ON ((BLDRighe.DBGruppo=BLDTestate.DBGruppo))
        WHERE ((BLDRighe.CodArt=BLDArticoli.CodArt))
              AND (BLDTestate.DBGruppo='GG')) AS [NumeroDoc]
FROM BLDArticoli AS [BLDArticoli]
WHERE (((
        (SELECT COUNT(DISTINCT BLDRighe.IDD) AS [Totale]
         FROM BLDTestate AS [BLDTestate]
         INNER JOIN BLDRighe AS [BLDRighe] ON
              ((BLDRighe.DBGruppo=BLDTestate.DBGruppo))
         WHERE ((BLDRighe.CodArt=BLDArticoli.CodArt))
                AND (BLDTestate.DBGruppo='GG'))>3))
AND (BLDArticoli.DBGruppo='GG')
```

Dove sono evidenziate le due occorrenze della sottoquery

Subquery tabellare

Una sottoquery definita mediante @DEFSUBQUERY/@ENDQUERY e che soddisfa i requisiti:

- Torna una o più colonne
- Torna 0 o più righe

Può essere impiegata nella definizione di una tabella derivata, detta anche sottoquery tabellare, usando all'interno del parametro TABELLA la pseudo-funzione SUBQ e passando come parametro l'id della sottoquery. Il parametro TABELLA va poi completato con il sottoparamento AS per darle un nome..

Riassumendo per le sottoquery tabellari il parametro TABELLA assume la forma:

TABELLA[SUBQ(idsottoquery) AS [NomeSottoquery]

Questo funzionamento vale per le specifiche FROM e JOIN

BC - BLDOBJQRY_SUBQUERY_TAB_DEFINIZIONE

```
'@DEFQUERY QUERY[Query] NOME["BLDOBJQRY_SUBQUERY_TAB_DEFINIZIONE_15"]

'@DEFSUBQUERY QUERY[QScadenze]

'@FROM TABELLA[BLDTESTATA]
'@JOIN TABELLA[BLDPIEDE] TABELLAPADRE[BLDTESTATA] OPERATOREJOIN["INNER"] REL[BLDPIEDE.IDDOC=BLDTESTATA.IDDOC]
'@JOIN TABELLA[BLDCPG] TABELLAPADRE[BLDTESTATA] OPERATOREJOIN["INNER"] REL[BLDCPG.COD=BLDTESTATA.CODPAG]
'@JOIN TABELLA[BLDSDE] TABELLAPADRE[BLDCPG] OPERATOREJOIN["LEFT"] REL[BLDSDE.COD=BLDCPG.COD]
'@WHERE ESPR[BLDTESTATA.GRDOC="FTV"]

'@COLUMN ESPR[BLDTESTATA.INTEST AS [Cliente]] _
            ESPR[BLDTESTATA.IDDOC AS [Documento]]

'@COMPUTEDCOLUMN ESPR[FNDAYDATEADD(BLDTESTATA.DatDoc,BLDSDE.GG) AS [DataScadenza] FMT[DATA8]] _
            ESPR[BLDPiede.ValTot*BLDSDE.PerImp/100 AS [Importo] FMT[NUMERICO[7,2]]]

'@ENDQUERY

'@FROM TABELLA[BLDCLIFOR]
'@JOIN TABELLA[SUBQ(QScadenze) as [Scadenze]] TABELLAPADRE[BLDCLIFOR] OPERATOREJOIN["INNER"] REL[[Scadenze].Cliente=BLDCLIFOR.COD]

'@COLUMN ESPR[BLDCLIFOR.COD AS [Cliente]]
'@COMPUTEDCOLUMN ESPR[[Scadenze].DataScadenza AS [Scadenza] FMT[DATA8]] _
            ESPR[[Scadenze].Importo AS [Importo] FMT[NUMERICO[8,2]]]

'@WHERE ESPR[[Cliente]=1 AND [Importo]>0]

'@ORDERBY ESPR[[Scadenza]]

'@ENDQUERY
```

```
SQL

SELECT BLDClientiFornitori.CodCliFor AS [Cliente],
       Scadenze.DataScadenza AS [Scadenza],
       Scadenze.Importo AS [Importo]
FROM BLDClientiFornitori AS [BLDClientiFornitori]
INNER JOIN
    (SELECT BLDTestate.DBGruppo AS [DBGruppo],
            BLDTestate.CodCliFor AS [Cliente],
            BLDTestate.IDD AS [Documento],
            DATEADD(DAY, ISNULL(BLDScadenzeCPag.GgMesiScad, 0), BLDTestate.DataDoc) AS [DataScadenza],
            BLDPiede.TotDocumentoVal*ISNULL(BLDScadenzeCPag.PercImpScad, 0.00)/100 AS [Importo]
    FROM BLDTestate AS [BLDTestate]
    INNER JOIN BLDPiede AS [BLDPiede] ON (BLDPiede.IDD=BLDTestate.IDD
                                         AND (BLDPiede.DBGruppo=BLDTestate.DBGruppo))
    INNER JOIN BLDCondPagamento AS [BLDCondPagamento] ON
(BLDCondPagamento.CodCondPagamento=BLDTestate.CodCondPag
                                         AND
(BLDCondPagamento.DBGruppo=BLDTestate.DBGruppo))
    LEFT OUTER JOIN BLDScadenzeCPag AS [BLDScadenzeCPag] ON
(BLDScadenzeCPag.CodCondPad=BLDCondPagamento.CodCondPagamento
                                         AND
(BLDScadenzeCPag.DBGruppo=BLDCondPagamento.DBGruppo))
    WHERE (BLDTestate.GruppoDoc='FTV')) AS [Scadenze] ON
(Scadenze.Cliente=BLDClientiFornitori.CodCliFor
                                         AND
(Scadenze.DBGruppo=BLDClientiFornitori.DBGruppo))
WHERE (((BLDClientiFornitori.CodCliFor)=1
        AND (Scadenze.Importo)>0.00))
        AND (BLDClientiFornitori.DBGruppo='')
ORDER BY [Scadenza] ASC
```

Il codice mette in evidenza la sottoquery usata come tabella derivata.

Subquery nella clausola IN

Una sottoquery definita mediante *@DEFSUBQUERY/@ENDQUERY* e che soddisfa i requisiti:

- Torna una colonna
- Torna 0 o più righe

Può essere impiegata nella definizione di una espressione **IN**, normalmente presente nella clausola WHERE SQL. La sintassi BC da impiegare è :

Campo **IN (SUBQ(idsottoquery))**

BC - BLD OBJQRY SUBQUERY IN DEFINIZIONE
<pre>'@PGQUERY BLD OBJQRY SUBQUERY IN DEFINIZIONE STRICT[4] ' Sottoquery nella clausola IN - Definizione query '@SBP Query[BCQUERY] [INOUT] '@DEFQUERY QUERY[Query] NOME["BLD OBJQRY SUBQUERY IN DEFINIZIONE_15"] '@FROM TABELLA[BLDCLIFOR] '@DEFSUBQUERY QUERY[Compratori] ELIMINADUPLICATI '@FROM TABELLA[BLDTESTATA] '@JOIN TABELLA[BLDRIGA] TABELLAPADRE[BLDTESTATA] OPERATOREJOIN["INNER"] REL[BLDRIGA.IDD=BLDTESTATA.IDD] '@COLUMN ESPR[BLDTESTATA.INTEST AS [Cliente]] '@ENDQUERY '@COLUMN ESPR[BLDCLIFOR.COD AS [Codice]] '@COMPUTEDCOLUMN ESPR[BLDCLIFOR.RS1+" "+BLDCLIFOR.RS1 AS[RagioneSociale] FMT[TESTO[50]]] '@WHERE ESPR[BLDCLIFOR.COD IN(SUBQ(Compratori))]] '@ENDQUERY End</pre>

Viene prodotto questo codice SQL:

SQL
<pre>SELECT BLDClientiFornitori.CodCliFor AS [Codice], BLDClientiFornitori.RagioneSociale1+' '+BLDClientiFornitori.RagioneSociale1 AS [RagioneSociale] FROM BLDClientiFornitori AS [BLDClientiFornitori] WHERE ((BLDClientiFornitori.CodCliFor IN ((SELECT DISTINCT BLDTestate.CodCliFor AS [Cliente] FROM BLDTestate AS [BLDTestate] INNER JOIN BLDRighe AS [BLDRighe] ON ((BLDRighe.DBGruppo=BLDTestate.DBGruppo)) WHERE (BLDTestate.DBGruppo='GG'))</pre>

Subquery annidate

E' possibile definire sottoquery annidate in modo arbitrario, cioè è possibile definire sottoquery all'interno di altre sottoquery. È sufficiente annidare le specifiche '@DEFSUBQUERY/ENDQUERY le une dentro le altre:

BC - BLD OBJQRY SUBQUERY NESTED DEFINIZIONE
--

```
'@DEFQUERY QUERY[Query] NOME["BLDOBJQRY_SUBQUERY_NESTED_DEFINIZIONE_15"]

'@DEFSUBQUERY QUERY[Nested]

'@FROM TABELLA[BLDART]

'@DEFSUBQUERY QUERY[TotaleRigheArt]

'@FROM TABELLA[BLDTESTATA]
'@JOIN TABELLA[BLDRIGA] TABELLAPADRE[BLDTESTATA] OPERATOREJOIN["INNER"] REL[BLDRIGA.IDD=BLDTESTATA.IDD]

'@COMPUTEDCOLUMN ESPR[COUNTDISTINCT(BLDRIGA.IDDOC) AS [Totale]]

'@WHERE ESPR[BLDRIGA.CODART=BLDART.COD ]

'@ENDQUERY

'@COLUMN ESPR[BLDART.COD AS [Codice]] _
        ESPR[BLDART.DES AS [Descrizione]]

'@COMPUTEDCOLUMN ESPR[SUBQ(TotaleRigheArt) AS [NumeroDoc] FMT[NUMERICO[6]]]
'@WHERE ESPR[SUBQ(TotaleRigheArt)>3]

'@ENDQUERY

'@FROM TABELLA[SUBQ(Nested) AS [Interna]]
'@COMPUTEDCOLUMN ESPR[[Interna].Descrizione AS [Descrizione] FMT[TESTO[50]]] _
        ESPR[[Interna].NumeroDoc AS [NDOC] FMT[NUMERICO[6]]]

'@ENDQUERY
```

Viene prodotto il seguente codice SQL:

```
SQL

SELECT Interna.Descrizione AS [Descrizione],
        Interna.NumeroDoc AS [NDOC]
FROM
    (SELECT BLDArticoli.DBGruppo AS [DBGruppo],
            BLDArticoli.CodArt AS [Codice],
            BLDArticoli.DesArt AS [Descrizione],

            (SELECT COUNT(DISTINCT BLDRighe.IDD) AS [Totale]
             FROM BLDTestate AS [BLDTestate]
             INNER JOIN BLDRighe AS [BLDRighe] ON ((BLDRighe.DBGruppo=BLDTestate.DBGruppo))
             WHERE ((BLDRighe.CodArt=BLDArticoli.CodArt))
                  AND (BLDTestate.DBGruppo='GG')) AS [NumeroDoc]
      FROM BLDArticoli AS [BLDArticoli]
      WHERE (
          (SELECT COUNT(DISTINCT BLDRighe.IDD) AS [Totale]
           FROM BLDTestate AS [BLDTestate]
           INNER JOIN BLDRighe AS [BLDRighe] ON
              ((BLDRighe.DBGruppo=BLDTestate.DBGruppo))
           WHERE ((BLDRighe.CodArt=BLDArticoli.CodArt))
                  AND (BLDTestate.DBGruppo='GG'))>3)) AS [Interna]
      WHERE (Interna.DBGruppo='GG')
```

Utilizzo avanzato delle subquery correlate

Nel caso in cui la subquery correlata è definita in un sorgente diverso da quello della query principale, è necessario passare sulla *@SBP* della sub di definizione il campo (o i campi) della query principale utilizzati per la correlazione.

In questo caso, ciò che viene passato non è il "valore" del campo, ma la "definizione" del campo della tabella principale. Questo significa che abbiamo bisogno di utilizzare un tipo particolare di dato che ci permette di convertire una espressione (<NOME_TABELLA.CAMPO>) in un dato base (STRING, INT, DOUBLE, ecc).

Nascono dei tipi nuovi che permettono di definire delle espressioni che si risolvono in un tipo dato base. Questi sono:

Tipo espressione	Tipo base
BCQUERY_ESPRDATE	DATE
BCQUERY_ESPRDOUBLE	DOUBLE
BCQUERY_ESPRINT	INT
BCQUERY_ESPRPERIOD	PERIOD
BCQUERY_ESPRSTRING	STRING

L'utilizzo delle espressioni all'interno dei sorgenti avviene tramite due metodi statici legati alle query:

```
- QUERY.GetExpression( expression )  
- QUERY.SetExpression( expression )
```

Esempio

prendiamo una query che estragga il numero totale delle righe per articolo (esempio CBAS_93) e questo numero deve essere maggiore di 3:

SQL

```
SELECT  
    BLDArticoli.CodArt AS [Codice]  
    ,BLDArticoli.DesArt AS [Descrizione]  
    ,(SELECT COUNT(DISTINCT BLDRighe.IDD) AS [Totale]  
    FROM BLDTestate AS [BLDTestate]  
    INNER JOIN BLDRighe AS [BLDRighe]  
        ON ((BLDRighe.DBGruppo = BLDTestate.DBGruppo))  
    WHERE ((BLDRighe.CodArt = BLDArticoli.CodArt)) AND (BLDTestate.DBGruppo = 'GG'))  
    AS [NumeroDoc]  
FROM BLDArticoli AS [BLDArticoli]  
WHERE (((SELECT COUNT(DISTINCT BLDRighe.IDD) AS [Totale]  
    FROM BLDTestate AS [BLDTestate]  
    INNER JOIN BLDRighe AS [BLDRighe]  
        ON ((BLDRighe.DBGruppo = BLDTestate.DBGruppo))  
    WHERE ((BLDRighe.CodArt = BLDArticoli.CodArt))  
    AND (BLDTestate.DBGruppo = 'GG'))  
> 3))  
AND (BLDArticoli.DBGruppo = 'GG')
```

La subquery che calcola il numero di righe per articolo è correlata con il campo COD della tabella BLDART.

Per rendere più dinamica la costruzione della query, possiamo pensare di dividerla in due:

- query principale: elementi dell'articolo
- subquery: calcola il numero delle righe per articolo.

In BC si traduce con due oggetti query separati in due oggetti distinti:

- il primo definisce la parte degli articoli e che richiama il secondo
- il secondo definisce la subquery.

Il primo oggetto query:

BC - BLDOBJQRY_SUBQUERY_RICHIAMO_DEFINIZIONE

```
'@PGQUERY BLD OBJQRY_SUBQUERY_RICHIAMO_DEFINIZIONE STRICT[4] ' SUBQUERY Richiamo - Definizione query

'@SBP Query[BCQUERY] [INOUT]

'@DEFQUERY QUERY[Query] NOME["BLD OBJQRY_SUBQUERY_RICHIAMO_DEFINIZIONE_15"]

'@FROM TABELLA[BLDART]

DIM totaleRigheArt[QUERY[BLD OBJQRY_SUBQUERY_DEF(Query [INOUT], QUERY.SetExpression(BLDART.COD) [INOUT] )]]

'@COLUMN ESPR[BLDART.COD AS [Codice]] _
          ESPR[BLDART.DES AS [Descrizione]]

'@COMPUTEDCOLUMN ESPR[SUBQ(totaleRigheArt) AS [NumeroDoc] FMT[NUMERICO[6]]]

'@WHERE ESPR[SUBQ(totaleRigheArt)>3]

'@ENDQUERY

End
```

Il secondo oggetto query:

BC - BLD OBJQRY_SUBQUERY_RICHIAMO_DEFINIZIONE

```
'@PGQUERY BLD OBJQRY_SUBQUERY_DEF_DEFINIZIONE STRICT[4] ' SUBQUERY Definizione - Definizione query

'@SBP TotaleRigheArt[BCQUERY] [INOUT] _ ' SubQuery
QueryPadre[BCQUERY] [INOUT] _ ' Query principale
CodArticolo[BCQUERY_ESPRSTRING] [INOUT] ' Campo COD della tabella BLDART

'@DEFSUBQUERY QUERY[TotaleRigheArt] QUERYPADRE[QueryPadre]

'@FROM TABELLA[BLDTESTATA]
'@JOIN TABELLA[BLDRIGA] TABELLAPADRE[BLDTESTATA] OPERATOREJOIN["INNER"] REL[BLDRIGA.IDD=BLDTESTATA.IDD]

'@COMPUTEDCOLUMN ESPR[COUNTDISTINCT(BLDRIGA.IDDOC) AS [Totale] FMT[NUMERICO[5]]]

'@WHERE ESPR[BLDRIGA.CODART=QUERY.GetExpression( CodArticolo )]

'@ENDQUERY

End
```

La query principale legge il codice e la descrizione dell'articolo. Il numero totale delle righe viene calcolato dalla subquery.

La subquery, essendo separata dalla query principale, riceve:

- l'oggetto query principale (BCQUERY)
- il codice articolo per effettuare la correlazione (BCQUERY_ESPRSTRING).

Il codice articolo viene passato come "espressione" nel senso che non viene passato il valore del campo della tabella, ma direttamente il campo della tabella. Nel nostro caso, il risultato dell'espressione (il codice articolo) è una stringa, quindi abbiamo usato BCQUERY_ESPRSTRING.

All'interno dei sorgenti, per leggere o scrivere i valori delle espressioni vengono usati i due metodi statici delle query: GetExpression e SetExpression.

Nel nostro esempio, la subquery deve leggere l'espressione (il campo COD di BLDART) che gli viene passata dal parametro CodArticolo di tipo BCQUERY_ESPRSTRING e per farlo usa la GetExpression. La query principale deve passare l'espressione e quindi usa la SetExpression.

Le common table expressions (CTEs) in BC

Cos'è una common table expression

La common table expression (CTE) è uno strumento di programmazione SQL che permette di semplificare query complesse. Sono comandi SELECT cui è stato dato un nome e che possono poi essere impiegate nella formulazione della query principale. In questo senso sono funzionalmente equivalenti alle viste.

Esistono 2 tipologie di CTE:

- Non ricorsive, equivalenti alla definizione di una o più viste temporanee, valide per il periodo di esecuzione della query;
- Ricorsive, che permettono di interrogare strutture gerarchiche complesse.

Sintassi SQL

La CTE si specifica usando la keyword WITH:

```
WITH <NomeCTE> ( <NomiColonna 1, NomeColonna 2, NomeColonna n> ) AS
(
    <QUERY DI DEFINIZIONE CTE>
),
....

<NomeCTE n> ( <NomiColonna 1, NomeColonna 2, NomeColonna n > ) AS
(
    <QUERY DI DEFINIZIONE CTE N>
)
SELECT <QueryPrincipale>
```

- WITH è la keyword che introduce la definizione di una o più CTE (ognuna separata da virgole);
- Ad ogni CTE si associa un nome (<NomeCTE >) che verrà usato per riferirsi nel codice SQL come se fosse una vista;

Tra parentesi tonde vi è la definizione della query di definizione del recordset della CTE.

Caratteristiche principali di una CTE

- Non è possibile annidare CTE. La keyword WITH può comparire solo all'inizio della query principale
- Posso definire più CTE
- Una CTE può riferirsi a una CTE definita prima nel comando SQL
- Una CTE può riferirsi anche a se stessa: è come sono definite le CTE ricorsive
- Valgono in genere le stesse limitazioni delle tabelle derivate: ad esempio non si può specificare la clausola ORDER BY, a meno che non sia presente anche la clausola TOP.

Definizione in BC

La definizione di una CTE avviene tramite la coppia di specifiche @DEFQUERYCTE /@ENDQUERY:

BC
<pre>'@DEFQUERY QUERY[Query] NOME["BLDOBJQRY_CTE_DEFINIZIONE_15"] '@DEFQUERYCTE QUERY[QScadenze] ALIAS["Scadenze"] '@ENDQUERY . . . '@ENDQUERY</pre>

- **DEFQUERYCTE** : definisce l'inizio della definizione della CTE. I parametri essenziali sono:
 - **QUERY** : definisce l'identificatore della query CTE
 - **QUERYPADRE** è l'identificatore della query padre, che può essere omesso innestato in un altro blocco
 - **ALIAS** : è il nome attribuito alla CTE che verrà utilizzato per riferirsi alla CTE nella query principale
- **ENDQUERY** : definisce la fine della definizione della CTE

All'interno del blocco *@DEFQUERYCTE*/*@ENDQUERY* si procede alla definizione della sotto query usando le stesse specifiche impiegate per la query: *@FROM*, *@JOIN*...:

BC
<pre>'@DEFQUERYCTE QUERY[QScadenze] ALIAS["Scadenze"] '@FROM TABELLA[BLDTESTATA] '@JOIN TABELLA[BLDPIDE] TABELLAPADRE[BLDTESTATA] OPERATOREJOIN["INNER"] REL[BLDPIDE.IDDOC=BLDTESTATA.IDDOC] '@JOIN TABELLA[BLDCPG] TABELLAPADRE[BLDTESTATA] OPERATOREJOIN["INNER"] REL[BLDCPG.COD=BLDTESTATA.CODPAG] '@JOIN TABELLA[BLDSDE] TABELLAPADRE[BLDCPG] OPERATOREJOIN["LEFT"] REL[BLDSDE.COD=BLDCPG.COD] '@WHERE ESPR[BLDTESTATA.GRDOC="FTV"] '@COLUMN ESPR[BLDTESTATA.INTEST AS [Cliente]] _ ESPR[BLDTESTATA.IDDOC AS [Documento]] '@COMPUTEDCOLUMN ESPR[FNDAYDATEADD (BLDTESTATA.DatDoc,BLDSDE.GG) AS [DataScadenza] FMT[DATA8]] _ ESPR[BLDPiede.ValTot*BLDSDE.PerImp/100 AS [Importo] FMT[NUMERICO[7,2]]] '@ENDQUERY</pre>

Impiego

Una volta definita una CTE può essere impiegata in un parametro TABELLA di *@FROM*/*@JOIN* racchiudendo il suo ALIAS tra parentesi quadre

TABELLA [[NomeCTE]]

BC - BLDOBJQRY_CTE_DEFINIZIONE
<pre>'@DEFQUERY QUERY[Query] NOME["BLDOBJQRY_CTE_DEFINIZIONE_15"] '@DEFQUERYCTE QUERY[QScadenze] ALIAS["Scadenze"] '@FROM TABELLA[BLDTESTATA] '@JOIN TABELLA[BLDPIDE] TABELLAPADRE[BLDTESTATA] OPERATOREJOIN["INNER"] REL[BLDPIDE.IDDOC=BLDTESTATA.IDDOC] '@JOIN TABELLA[BLDCPG] TABELLAPADRE[BLDTESTATA] OPERATOREJOIN["INNER"] REL[BLDCPG.COD=BLDTESTATA.CODPAG] '@JOIN TABELLA[BLDSDE] TABELLAPADRE[BLDCPG] OPERATOREJOIN["LEFT"] REL[BLDSDE.COD=BLDCPG.COD] '@WHERE ESPR[BLDTESTATA.GRDOC="FTV"] '@COLUMN ESPR[BLDTESTATA.INTEST AS [Cliente]] _ ESPR[BLDTESTATA.IDDOC AS [Documento]] '@COMPUTEDCOLUMN ESPR[FNDAYDATEADD (BLDTESTATA.DatDoc,BLDSDE.GG) AS [DataScadenza] FMT[DATA8]] _ ESPR[BLDPiede.ValTot*BLDSDE.PerImp/100 AS [Importo] FMT[NUMERICO[7,2]]] '@ENDQUERY '@FROM TABELLA[BLDCLIFOR] '@JOIN TABELLA[[Scadenze]] TABELLAPADRE[BLDCLIFOR] OPERATOREJOIN["INNER"] REL[[Scadenze].Cliente=BLDCLIFOR.COD] '@COLUMN ESPR[BLDCLIFOR.COD AS [Cliente]] '@COMPUTEDCOLUMN ESPR[[Scadenze].DataScadenza AS [Scadenza] FMT[DATA8]] _ ESPR[[Scadenze].Importo AS [Importo] FMT[NUMERICO[8,2]]] '@WHERE ESPR[[Cliente]=1 AND [Importo]>0] '@ORDERBY ESPR[[Scadenza]]</pre>

⚠ Anche per CTE, come per le subquery, quando si deve usare un campo nella query è necessario usare un campo calcolato.

Viene prodotto il seguente codice SQL:

SQL

```
WITH Scadenze (Cliente, Documento, DataScadenza, Importo)
AS
(SELECT
    BLDTestate.CodCliFor AS [Cliente]
    ,BLDTestate.IDD AS [Documento]
    ,DATEADD(DAY, ISNULL(BLDScadenzeCPag.GgMesiScad, 0), BLDTestate.DataDoc) AS [DataScadenza]
    ,BLDPiede.TotDocumentoVal * ISNULL(BLDScadenzeCPag.PercImpScad, 0.00) / 100 AS [Importo]
FROM BLDTestate AS [BLDTestate]
INNER JOIN BLDPiede AS [BLDPiede]
    ON (BLDPiede.IDD = BLDTestate.IDD
    AND (BLDPiede.DBGruppo = BLDTestate.DBGruppo))
INNER JOIN BLDCondPagamento AS [BLDCondPagamento]
    ON (BLDCondPagamento.CodCondPagamento = BLDTestate.CodCondPag
    AND (BLDCondPagamento.DBGruppo = BLDTestate.DBGruppo))
LEFT OUTER JOIN BLDScadenzeCPag AS [BLDScadenzeCPag]
    ON (BLDScadenzeCPag.CodCondPag = BLDCondPagamento.CodCondPagamento
    AND (BLDScadenzeCPag.DBGruppo = BLDCondPagamento.DBGruppo))
WHERE ((BLDTestate.GruppoDoc = 'FTV'))
AND (BLDTestate.DBGruppo = 'GG'))
SELECT
    BLDClientiFornitori.CodCliFor AS [Cliente]
    ,Scadenze.DataScadenza AS [Scadenza]
    ,Scadenze.Importo AS [Importo]
FROM BLDClientiFornitori AS [BLDClientiFornitori]
INNER JOIN Scadenze AS [Scadenze]
    ON (Scadenze.Cliente = BLDClientiFornitori.CodCliFor)
WHERE (((BLDClientiFornitori.CodCliFor) = 1
AND (Scadenze.Importo) > 0.00))
AND (BLDClientiFornitori.DBGruppo = 'GG'))
ORDER BY [Scadenza] ASC
```

Struttura CTE ricorsiva

Sintassi SQL

La sintassi SQL di una CTE ricorsiva estende quella non ricorsiva:

```
WITH <NomeCTE> ( <NomiColonna 1, NomeColonna 2, NomeColonna n> ) AS
(
    <QUERY DI DEFINIZIONE CTE - PARTE FISSA>
    UNION ALL
    <QUERY DI DEFINIZIONE CTE - PARTE RICORSIVA>
)

SELECT <QueryPrincipale>
```

- La query di definizione prevede due parti: una fissa e d una ricorsiva. Le due parti sono due query (o subquery).
- Le due parti sono separate dalla clausola UNION ALL.
- La parte ricorsiva usa il nome della CTE per innescare la ricorsione.

Caratteristiche principali di una CTE ricorsiva

Per utilizzare una CTE ricorsiva è bene ricordare alcune regole base:

- Il numero delle colonne nei membri ricorsivi e non ricorsivi deve essere lo stesso.
- Il tipo di dati di una colonna nel membro ricorsivo deve essere lo stesso del tipo di dati della colonna corrispondente nel membro non ricorsivo.
- La clausola FROM di un membro ricorsivo deve fare riferimento solo una volta all'espressione <NomeCTE>.
- Gli elementi seguenti non sono consentiti nella definizione <QUERY DI DEFINIZIONE CTE - PARTE RICORSIVA>:
 - SELECT DISTINCT

- GROUP BY
- HAVING
- Aggregazione scalare
- LEFT, RIGHT, OUTER JOIN (è consentito solo INNER JOIN)
- Sottoquery
- Una CTE ricorsiva formulata in modo non corretto può provocare un ciclo infinito. Ad esempio, se la definizione della query del membro ricorsivo restituisce gli stessi valori per entrambe le colonne padre e figlio, si crea un ciclo infinito.
- Non è possibile utilizzare una vista che contiene una CTE ricorsiva per aggiornare i dati.

Funzionamento CTE ricorsiva

I passi logici eseguiti sono:

1. Esecuzione della parte fissa e produzione del primo recordset
2. Esecuzione della parte ricorsiva che deve riferirsi alla CTE stessa. La parte ricorsiva esamina il recordset prodotto dalla parte fissa o dalla precedente ricorsione e produce un nuovo recordset
3. Se la parte ricorsiva ha prodotto un recordset non vuoto viene eseguito nuovamente il passo 2.

Il risultato finale è la somma di tutti i recordset prodotti:

- dalla parte fissa
- dalle n ricorsioni

Nell'esempio avremo quindi:

Passo	Recordset esaminato dalla parte ricorsiva	Recordset prodotto dalla parte fissa/ ricorsiva	Recordset finale
0		n 1	n 1
1	n 1	n 2	n 1 2
2	n 2	n 3	n 1 2 3
...			
50	n 50	n	n 1 2 3 ... 50

Sintassi BC

La scrittura in BC di una CTE ricorsiva si caratterizza dalla presenza nella definizione del sotto parametro PARTE del parametro CTE che specifica quale delle due è la parte ricorsiva.

Esempio

SQL

```
/* Esempio di una CTE ricorsiva
   che prevede la definizione di due CTE:
   - la prima definizione è la parte fissa
   - la seconda la seconda è la parte ricorsiva
*/

WITH Organigramma (IdPadre, Nominativo, Livello, Posizione, Sort)
AS
(SELECT
    BLDImpiegati.IDImpiegato AS [IdPadre]
    ,CAST(BLDImpiegati.Nome + ' ' + BLDImpiegati.Cognome AS VARCHAR(80)) AS [Nominativo]
    ,1 AS [Livello]
    ,CAST(LTRIM(RTRIM(BLDImpiegati.Nome)) + ' ' + LTRIM(RTRIM(BLDImpiegati.Cognome)) AS VARCHAR(255)) AS [Posizione]
    ,CAST(CONVERT(VARCHAR(30), BLDImpiegati.IDImpiegato) AS VARCHAR(255)) AS [Sort]
FROM BLDImpiegati AS [BLDImpiegati]
WHERE ((BLDImpiegati.IDResponsabile = 0))
AND (BLDImpiegati.DBGruppo = 'GG')
UNION ALL
SELECT
    BLDImpiegati.IDImpiegato AS [IdPadre]
    ,CAST(BLDImpiegati.Nome + ' ' + BLDImpiegati.Cognome AS VARCHAR(80)) AS [Nominativo]
    ,Organigramma_1.Livello + 1 AS [Livello]
    ,CAST(REPLICATE(' ', Organigramma_1.Livello) + LTRIM(RTRIM(BLDImpiegati.Nome)) + ' ' +
LTRIM(RTRIM(BLDImpiegati.Cognome)) AS VARCHAR(255)) AS [Posizione]
    ,CAST(Organigramma_1.Sort + CONVERT(VARCHAR(30), BLDImpiegati.IDImpiegato) AS VARCHAR(255)) AS [Sort]
FROM BLDImpiegati AS [BLDImpiegati]
INNER JOIN Organigramma AS [Organigramma_1]
    ON (BLDImpiegati.IDResponsabile = Organigramma_1.IdPadre)
WHERE (BLDImpiegati.DBGruppo = 'GG'))
SELECT
    Nominativo AS [Nominativo]
    ,Posizione AS [Posizione]
    ,Livello AS [Livello]
FROM Organigramma AS [Organigramma]
ORDER BY Sort ASC;
```

BC - BLDOBJQRY_CTE_RECURSIVE_DEFINIZIONE

```
'@PGQUERY BLD OBJQRY_CTE_RECURSIVE_DEFINIZIONE STRICT[4] ' CTE ricorsiva - Definizione query

'@SBF Query[BCQUERY] [INOUT]

'@DEFQUERY QUERY[Query] NOME["BLOBJQRY_CTE_RECURSIVE_DEFINIZIONE_15"]

'@DEFQUERYCTE QUERY[ParteFissa] ALIAS["Organigramma"] PARTE[#FISSA]

'@FROM TABELLA[BLDIMPIEGATI]

'@COLUMN ESPR[BLDIMPIEGATI.IDImpiegato AS [IdPadre]]
'@COMPUTEDCOLUMN ESPR[BLDIMPIEGATI.Nome + " " + BLDIMPIEGATI.Cognome AS [Nominativo] FMT[TESTO[80]]]
'@COMPUTEDCOLUMN ESPR[1 AS [Livello] FMT[NUMERICO[2]]]
'@COMPUTEDCOLUMN ESPR[FNTRSD$(BLDIMPIEGATI.Nome) + "
" + FNTRSD$(BLDIMPIEGATI.Cognome) AS [Posizione] FMT[TESTO[255]]]
'@COMPUTEDCOLUMN ESPR[FNS$(BLDIMPIEGATI.IDImpiegato) AS [Sort] FMT[TESTO[255]]]

'@WHERE ESPR[BLDIMPIEGATI.IDResponsabile=0]

'@ENDQUERY

'@DEFQUERYCTE QUERY[ParteRicorsiva] ALIAS["Organigramma"] PARTE[#RICORSIVA]

'@FROM TABELLA[BLDIMPIEGATI]
'@JOIN TABELLA[[Organigramma]] TABELLAPADRE[BLDIMPIEGATI] OPERATOREJOIN["INNER"] REL[BLDIMPIEGATI.IDResponsabile=
[Organigramma].IdPadre]

'@COLUMN ESPR[BLDIMPIEGATI.IDImpiegato AS [IdPadre]]
'@COMPUTEDCOLUMN ESPR[BLDIMPIEGATI.Nome + " " + BLDIMPIEGATI.Cognome AS [Nominativo] FMT[TESTO[80]]]
'@COMPUTEDCOLUMN ESPR[[Organigramma].Livello + 1 AS [Livello] FMT[NUMERICO[2]]]
'@COMPUTEDCOLUMN ESPR[STRING$([Organigramma].Livello, "| ") + FNTRSD$(BLDIMPIEGATI.Nome) + "
" + FNTRSD$(BLDIMPIEGATI.Cognome) AS [Posizione] FMT[TESTO[255]]]
'@COMPUTEDCOLUMN ESPR[[Organigramma].Sort + FNS$(BLDIMPIEGATI.IDImpiegato) AS [Sort] FMT[TESTO[255]]]

'@ENDQUERY

'@FROM TABELLA[[Organigramma]]

'@COMPUTEDCOLUMN ESPR[[Organigramma].Nominativo AS [Nominativo] FMT[TESTO[80]]]
'@COMPUTEDCOLUMN ESPR[[Organigramma].Posizione AS [Posizione] FMT[TESTO[255]]]
'@COMPUTEDCOLUMN ESPR[[Organigramma].Livello AS [Livello] FMT[NUMERICO[2]]]
'@COMPUTEDCOLUMN ESPR[[Organigramma].Sort AS [Sort] FMT[NUMERICO[5]] NOOUTPUT]

'@ORDERBY ESPR[[Sort]]

'@ENDQUERY

End
```

Risultato:

	Nominativo	Posizione	Livello
1	Mario Rossi	Mario Rossi	1
2	Paolo Verdi	Paolo Verdi	2
3	Davide Fucsia	Davide Fucsia	3
4	Maria Rosa	Maria Rosa	4
5	Giovanni Bianchi	Giovanni Bianchi	3
6	Michele Gialli	Michele Gialli	4
7	Linda Blu	Linda Blu	4
8	Sara Viola	Sara Viola	3
9	Laura Arancione	Laura Arancione	4

Utilizzo CTE con clausola UNION

In BC è necessario seguire una sintassi ben precisa per poter gestire le CTE con più query in union. La sinstassi SQL prevede che tutte le CTE vengano definite ad inizio query, in modo che siano tutte disponibili per l'utilizzo all'interno dello statement.

```
WITH <NomeCTE> ( <NomiColonna 1, NomeColonna 2, NomeColonna n> ) AS
(
    <QUERY DI DEFINIZIONE CTE>

```

```
),
....

<NomeCTE n> ( <NomiColonna 1, NomeColonna 2, NomeColonna n > ) AS
(
    <QUERY DI DEFINIZIONE CTE N>
)
SELECT <QueryPrincipale1>
UNION [ALL]
SELECT <QueryPrincipale2>
UNION [ALL]
SELECT <QueryPrincipaleN>
```

In BC è necessario definire prima tutti gli identificativi delle query e poi esplicitare nella UNION prima le query con CTE e poi le altre.

SQL

```
/* Esempio di due CTE presenti in entrambe le
query messe in UNION.*/

/* Prima CTE: estrae tutti gli uffici con più di un impiegato*/
WITH Impiegati (IdUfficio, ContaUfficio)
AS (SELECT IDUfficio AS [IdUfficio],
          COUNT(IDImpiegato) AS [ContaUfficio]
     FROM BFT_Impiegati AS [BFT_Impiegati]
     GROUP BY IdUfficio
     HAVING (COUNT((IDImpiegato)) > 1)),

/* Seconda CTE: estrarre tutti gli uffici con un solo impiegato*/
Impiegati2 (IdUfficio, ContaUfficio)
AS (SELECT IDUfficio AS [IdUfficio],
          COUNT(IDImpiegato) AS [ContaUfficio]
     FROM BFT_Impiegati AS [BFT_Impiegati]
     GROUP BY IdUfficio
     HAVING (COUNT((IDImpiegato)) = 1))

/* Prima query con la prima CTE*/
SELECT BFT_Uffici.Descrizione AS [DescrizioneUfficio],
       Impiegati.ContaUfficio AS [NumeroImpiegati]
FROM BFT_Uffici AS [BFT_Uffici]
     INNER JOIN Impiegati AS [Impiegati]
       ON (Impiegati.IdUfficio = BFT_Uffici.IDUfficio)

/* Clausola di UNION*/
UNION

/* Seconda query con la seconda CTE*/
SELECT BFT_Uffici.Descrizione AS [DescrizioneUfficio],
       Impiegati2.ContaUfficio AS [NumeroImpiegati]
FROM BFT_Uffici AS [BFT_Uffici]
     INNER JOIN Impiegati2 AS [Impiegati2]
       ON (Impiegati2.IdUfficio = BFT_Uffici.IDUfficio)

/* Clausola di UNION*/
UNION

/* Terza query senza CTE*/
SELECT 'Ufficio Fittizio' AS [DescrizioneUfficio],
       12 AS [NumeroImpiegati]
FROM BFT_Uffici AS [BFT_Uffici]

/* Ordinamento sulla UNION*/
ORDER BY [DescrizioneUfficio] ASC;
```

BC

```

'@PGQUERY BLD OBJQRY_CTE_UNION_DEFINIZIONE STRICT[4] ' CTE con UNION - Definizione query

'@SBP Query[BCQUERY] [INOUT]

'@DEFQUERY QUERY[Query] NOME["BLDOBJQRY_CTE_UNION_DEFINIZIONE_15"]

' Definizione della prima query
'@DEFQUERY QUERY[PrimaQuery] NOME["BLDOBJQRY_CTE_PrimaQuery"]

' Definizione della prima CTE
'@DEFQUERYCTE QUERY[Impiegati] ALIAS["Impiegati"]

'@FROM TABELLA[BLDIMPIEGATI]

'@COLUMN ESPR[GROUPBY (BLDIMPIEGATI.IDUfficio) AS [IdUfficio]] _
        ESPR[COUNT (BLDIMPIEGATI.IDImpiegato) AS [ContaUfficio]]

' Uffici con più di un impiegato
'@WHERE ESPRHAVING[ [ContaUfficio]>1]

'@ENDQUERY

'@FROM TABELLA[BLDUFFICI]

'@JOIN TABELLA[[Impiegati]] TABELLAPADRE[BLDUFFICI] _
        REL[[Impiegati].IdUfficio = BLDUFFICI.IDUfficio] _
        OPERATOREJOIN["INNER"]

'@COLUMN ESPR[BLDUFFICI.Descrizione AS [DescrizioneUfficio]]
'@COMPUTEDCOLUMN ESPR[[Impiegati].ContaUfficio AS [NumeroImpiegati] FMT[NUMERICO[9]]]

'@ENDQUERY

' Definizione della seconda query
'@DEFQUERY QUERY[SecondaQuery] NOME["BLDOBJQRY_CTE_SecondaQuery"]

' Definizione della seconda CTE
'@DEFQUERYCTE QUERY[Impiegati2] ALIAS["Impiegati2"]

'@FROM TABELLA[BLDIMPIEGATI]

'@COLUMN ESPR[GROUPBY (BLDIMPIEGATI.IDUfficio) AS [IdUfficio]] _
        ESPR[COUNT (BLDIMPIEGATI.IDImpiegato) AS [ContaUfficio]]

' Uffici con un impiegato
'@WHERE ESPRHAVING[ [ContaUfficio]=1]

'@ENDQUERY

'@FROM TABELLA[BLDUFFICI]

'@JOIN TABELLA[[Impiegati2]] TABELLAPADRE[BLDUFFICI] _
        REL[[Impiegati2].IdUfficio = BLDUFFICI.IDUfficio] _
        OPERATOREJOIN["INNER"]

'@COLUMN ESPR[BLDUFFICI.Descrizione AS [DescrizioneUfficio]]
'@COMPUTEDCOLUMN ESPR[[Impiegati2].ContaUfficio AS [NumeroImpiegati] FMT[NUMERICO[9]]]

'@ENDQUERY

' Definizione della terza query
'@DEFQUERY QUERY[TerzaQuery] NOME["BLDOBJQRY_CTE_TerzaQuery"]

'@FROM TABELLA[BLDUFFICI]

'@COMPUTEDCOLUMN ESPR["Ufficio Fittizio" AS [DescrizioneUfficio] FMT[TESTO[50]]]
'@COMPUTEDCOLUMN ESPR[12 AS [NumeroImpiegati] FMT[NUMERICO[9]]]

'@ENDQUERY

'@DEFQUERYSET ESPR[QRY(PrimaQuery) UNION QRY(SecondaQuery) UNION QRY(TerzaQuery)]

'@ENDQUERY

```

End

Risultato:

	DescrizioneUfficio	NumeroImpiegati
1	Direzione	1
2	Ufficio Commerciale	6
3	Ufficio Fittizio	12
4	Ufficio Marketing	2

Analisi e ottimizzazione delle query

Concetti di base

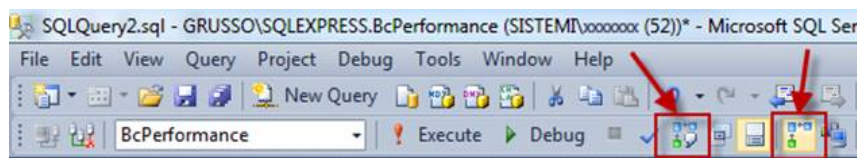
Ogni volta che eseguiamo una query, il motore SQL cerca di capire quale sia il percorso migliore per elaborare i dati. Ovviamente se le 'strade' iniziano a essere numerose il motore non le controlla tutte, ma inizia a considerare solo quelle che secondo le sue statistiche sono più performanti.

Per fare un esempio è come se chiedessimo ad un navigatore la strada più veloce per andare da Torino a Milano, le soluzioni sono molteplici, ma sa che l'autostrada consente una velocità maggiore rispetto a una statale.

Pertanto con i 'pesi' delle statistiche verrà identificato il piano di esecuzione più idoneo secondo le statistiche presenti sul DB.

Piano di esecuzione

Il piano di esecuzione non è un concetto astratto, ma può essere visualizzato nell'SQL Management Studio attraverso i seguenti tasti:



- Il primo bottone (Ctrl+L) consente di visualizzare il piano di esecuzione stimato, cioè quello che teoricamente dovrebbe fare;
- Il secondo bottone (Ctrl+M) consente di visualizzazione il piano di esecuzione effettivo, cioè quello eseguito dalla query.

Ovviamente è consigliato usare il tasto del piano di esecuzione effettivo.

Il piano di esecuzione si legge da destra verso sinistra e consente di capire come una query lavora per restituire i dati richiesti, da esso è possibile apprendere:

- Il dettaglio delle operazioni eseguite
- I tempi di elaborazione
- Su quali indici si sta lavorando
- Il quantitativo di righe lette/stimate
- Il costo per la CPU

Proviamo a leggere un piano di esecuzione a fronte delle seguenti tabelle Customer e Zone

Customer	
IdCustomer	
Ragsoc	
RegisterDate	
IdZone	

Zone	
IdZone	
ZoneDes	
RegisterDate	

Viene eseguita la seguente query per ricavare i clienti di una determinata zona creata in una certa data:

```
-- Per eseguire la query come se fosse la prima volta
```

```
DBCC FREEPROCCACHE
```

```
DBCC DROPCLEANBUFFERS
```

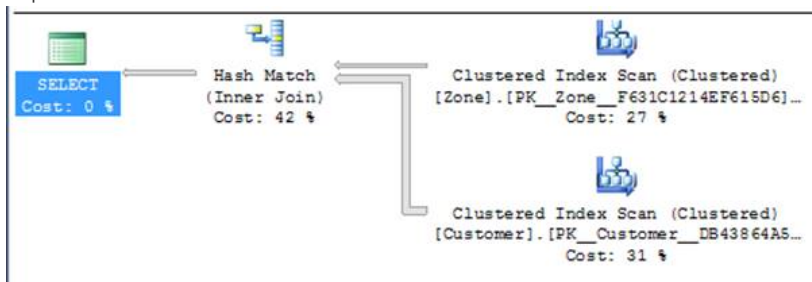
```
go
```

```

select C.IdCustomer, Z.ZoneDes from Customer as C
join Zone as Z on Z.IdZone=C.IdZone
where Z.RegisterDate= '19990820'

```

Che produce questo piano di esecuzione:



Dal quale riusciamo a capire che viene eseguito un Index Scan della tabella Customer e Zone e gli indici utilizzati sono quelli della chiave primaria di ogni tabella (PK_Customer., PK_Zone...).

Tutte le operazioni riportano il peso in percentuale riferito all'intero processo di elaborazione e lo spessore delle frecce è proporzionale al numero di righe elaborate.

Analizziamo alcune operazioni nel dettaglio per comprendere che cosa stiamo vedendo, nell'esempio sono presenti delle operazioni di:

- **Index Scan:** Indica che l'indice viene interamente letto per trovare gli elementi. In genere viene eseguita se non vi è un indice che soddisfa le condizioni della WHERE della query. È da considerarsi un'operazione costosa.
- **Hash Match:** Indica che viene creato il codice hash per ogni riga delle tabelle coinvolte nella join e successivamente viene eseguita un'operazione di confronto per associare i record. È da considerarsi un'operazione costosa.

Inoltre potrebbero esserci anche altre operazioni come:

- **Table Scan:** Indica che la tabella viene interamente letta per trovare gli elementi, viene eseguita quando la tabella non presenta indici al suo interno. È da considerarsi un'operazione costosa.
- **Index Seek:** Indica che viene letto in modo diretto il record ricercato secondo le chiavi previste nella WHERE o in JOIN, inoltre se la query lo richiede e l'indice ne è provvisto restituisce i dati secondo l'ordinamento. È da considerarsi un'operazione prestante.
- **Merge join:** Indica che le tabelle oggetto di JOIN sono ordinate secondo la chiave della Join e su di esse viene eseguito un confronto per associare i dati. È da considerarsi un'operazione prestante.

Per l'elenco completo di tutte le operazioni consultare il seguente link: <http://technet.microsoft.com/it-it/library/ms191158.aspx>

Posizionandoci sulle varie icone potremo visualizzare il dettaglio delle operazioni:

Clustered Index Scan (Clustered)	
Scanning a clustered index, entirely or only a range.	
Physical Operation	Clustered Index Scan
Logical Operation	Clustered Index Scan
Actual Execution Mode	Row
Estimated Execution Mode	Row
Actual Number of Rows	30337
Actual Number of Batches	0
Estimated I/O Cost	0.164606
Estimated Operator Cost	0.198134 (31%)
Estimated CPU Cost	0.0335277
Estimated Subtree Cost	0.198134
Estimated Number of Executions	1
Number of Executions	1
Estimated Number of Rows	30337
Estimated Row Size	15 B
Actual Rebinds	0
Actual Rewinds	0
Ordered	False
Node ID	2
Object	
[BcPerformance].[dbo].[Customer].	
[PK_Customer_DB43864A51F42706] [C]	
Output List	
[BcPerformance].[dbo].[Customer].IdCustomer;	
[BcPerformance].[dbo].[Customer].IdZone	

Dal quale poter ricavare il costo della CPU, un dato importante da tenere in considerazione e il numero delle righe estratte da confrontare con il numero delle righe stimate.

È da tenere in considerazione che il piano di esecuzione di una query viene salvato in memoria per poter essere riutilizzato a fronte di query con la stessa struttura.

Una volta compreso il funzionamento del piano di esecuzione, iniziamo a capire come intervenire sul DB per migliorare le prestazioni a partire dalla progettazione della base dati.

I dati

L'unità base per l'archiviazione dei dati in SQL è la pagina, il motore SQL legge e scrive per pagine. Una pagina è grande 8K e pertanto più righe della mia tabella riuscirò far stare in una pagina, migliori saranno le prestazioni di lettura.

Fatta questa precisazione, ecco alcune regole da seguire per progettare una corretta base dati:

- Utilizziamo per i campi della tabella il tipo di dato più piccolo adatto alle nostre esigenze, in modo da poter far stare più righe possibili in una pagina;
- Suddividiamo le tabelle con molte colonne, in modo da lasciare nella principale i dati più usati e nella/e secondaria/e i dati meno usati;
- Evitiamo la duplicazione dei dati

Di seguito viene riportata una tabella inerente i tipi di dato che possiamo definire in BC e il corrispondente tipo di dato SQL. Il tipo di dato SQL può variare in base alla versione di SQL Server in cui viene installata la base dati:

TIPO BC	MSQL 2000	MSQL 2005	MSQL 2008	MSQL 2008 R2	MSQL 2010
A	DATETIME	DATETIME	DATE	DATE	DATE
C	VARCHAR	VARCHAR	VARCHAR	VARCHAR	VARCHAR
D	DECIMAL	DECIMAL	DECIMAL	DECIMAL	DECIMAL
E	INTEGER	INTEGER	INTEGER	INTEGER	INTEGER
I	SMALLINT	SMALLINT	SMALLINT	SMALLINT	SMALLINT
L	INTEGER	INTEGER	INTEGER	INTEGER	INTEGER
N	SMALLINT TINYINY	SMALLINT TINYINY	SMALLINT TINYINY	SMALLINT TINYINY	SMALLINT TINYINY
S	DOUBLE	DOUBLE	DOUBLE	DOUBLE	DOUBLE
B	TEXT	VARCHAR(MAX)	VARCHAR(MAX)	VARCHAR(MAX)	VARCHAR(MAX)

Per l'occupazione in byte delle varie tipologie di dati, consultare il seguente link: <http://msdn.microsoft.com/it-it/library/ms187752.aspx>

Gli indici

Attraverso la creazione di indici è possibile migliorare sensibilmente le prestazioni del nostro database.

In una tabella possono esistere:

- un indice Clustered;
- "tanti" indici NonClustered (999)

Composti al massimo da 16 colonne, la differenza tra questi due indici è che quello cluster rappresenta l'ordinamento fisico di tutti i record della tabella, in quello non cluster sono presenti solo i campi dell'indice e i restanti campi vengono recuperati facendo riferimento all'indirizzo fisico degli altri campi.

Pertanto se ne deriva che per leggere tutti i campi di una tabella con l'indice Clustered viene eseguita una sola operazione, con l'indice NonClustered vengono eseguite due operazioni: la lettura dei campi dell'indice e poi ci sposta sugli altri campi della tabella per completarla.. (lettura in 'due tempi')

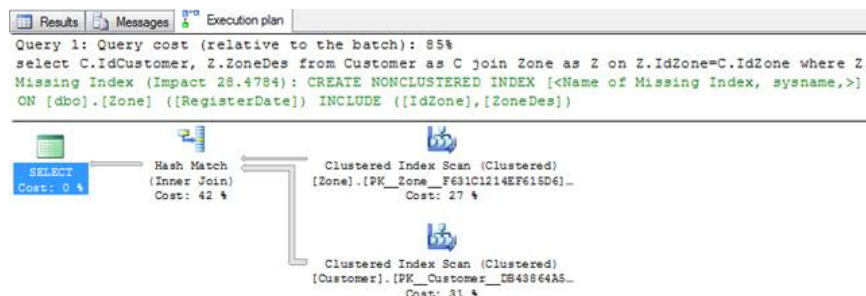
Quando visualizziamo il piano di esecuzione di una query, SQL cerca di darci eventuali suggerimenti sulla creazione di indici.

Considerando la precedente query su clienti e zone:

```
select C.IdCustomer, Z.ZoneDes from Customer as C
join Zone as Z on Z.IdZone=C.IdZone
where Z.RegisterDate= '19990820'
```

Il piano di esecuzione ci suggerisce di aggiungere alla nostra tabella Zone, un indice composto da:

- RegisterDate
- IdZone
- ZoneDes



Il motivo per cui ci fa questa richiesta è dovuto al fatto che nella *WHERE* viene utilizzato un filtro per il campo RegisterDate della tabella Zone. Mentre ci chiede di aggiungere a quest'indice anche i campi IdZone e ZoneDes perchè vengono utilizzati nella *SELECT* e nella *JOIN*.

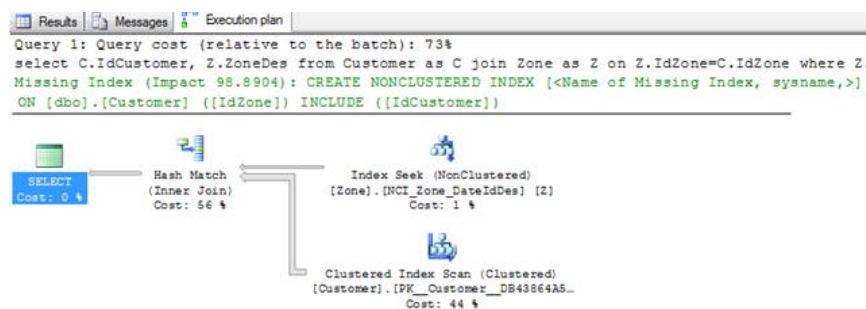
Procediamo alla creazione dell'indice:

```
create nonclustered index NCI_Zone_DateIdDes on Zone(RegisterDate,IdZone,ZoneDes)
```

Aggiungendoli all'indice NonClustered si evita di eseguire una lettura in 'due tempi' per l'indice, in quanto tutti i campi sono presenti direttamente sull'indice.

Eseguendo nuovamente la query il piano di esecuzione sostituisce il precedente IndexScan con un più performante Index Seek, portando dal 27% al 1% il peso della lettura della tabella Zone.

Ma ci viene dato un altro suggerimento:



In questo caso ci richiede di aggiungere un indice NonClustered sulla tabella Customer con i seguenti campi:

- IdZone
- IdCustomer

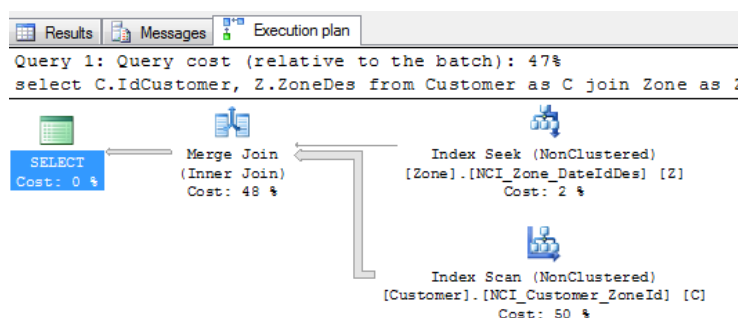
Il motivo per cui ci chiede di aggiungere questo indice è dovuto al fatto che la Join tra la tabella Customer e Zone avviene per IdZone e il motivo per cui ci fa aggiungere anche IdCustomer è dovuto alla presenza del campo nella select.

Anche in questo caso suggerisce di mettere nella chiave il campo IdCustomer per evitare una lettura in 'due tempi' dell'indice.

Procediamo alla creazione dell'indice:

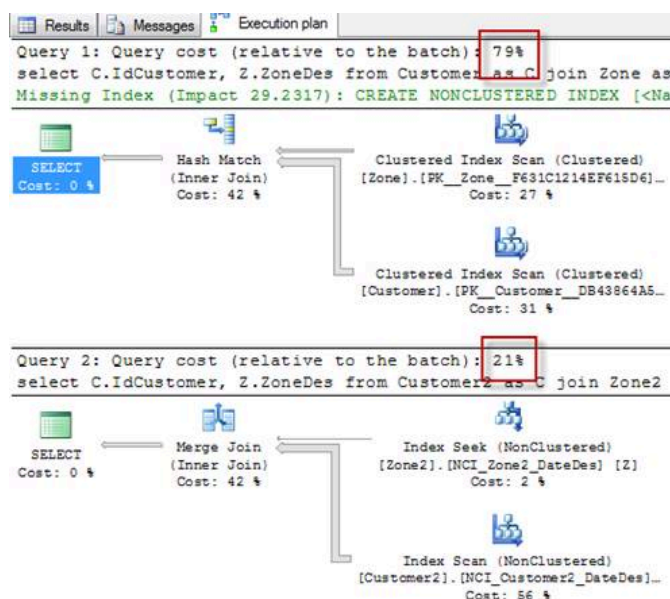
```
create nonclustered index NCI_Customer_ZoneId on Customer(IdZone,IdCustomer)
```

e, rieseguendo la query:



possiamo notare che è scomparsa l'operazione di Hash Match, sostituita da una più performante Merge Join.

Se creiamo due tabelle gemelle di zone e customer con all'interno gli indici suggeriti da SQL potremo mettere a confronto le due architetture e capire quanto abbiamo guadagnato in prestazioni:



Con l'aggiunta degli indici siamo riusciti a migliorare la performance delle query dell'80% circa.

La tecnica utilizzata da SQL è quella di suggerire un indice NonClustered, che oltre a soddisfare le clausole della *WHERE* o della *JOIN* permette di evitare la lettura in 'due tempi' dell'indice aggiungendo anche le colonne della tabella presenti nella *SELECT*. Questa tecnica viene denominata "indice di copertura", ed è altamente performante se la nostra tabella è soggetta ad una determinata query o gruppo di query con struttura e dati facilmente identificabili.

La creazione di indici non è un'operazione 'indolore' per SQL, implica dei costi di manutenzione ogni volta che si esegue un'aggiunta o un'eliminazione sulla tabella perché l'indice è soggetto a riordinamento.

Tenendo presente queste logiche, per riuscire a condurre delle buone scelte sulla progettazione degli indici, occorre avere le idee chiare sul principale utilizzo delle proprie tabelle. Tabelle soggette a molti inserimenti non dovrebbero avere molti indici in quanto il costo di scrittura diventerebbe 'pesante'. Al contrario, su tabelle abbastanza 'statiche' e frequentemente utilizzate in lettura converrebbe avere un indice di copertura (ove possibile identificare utilizzi sistematici di determinati filtri/colonne).

Un'altra pratica comune per migliorare le prestazioni delle query è quella di dotare le FOREIGN KEY (le relazioni) delle tabelle, con un indice per supportarle. Vedi precedente esempio delle zone dei clienti relazionate con la tabella zone.

Scrittura delle WHERE

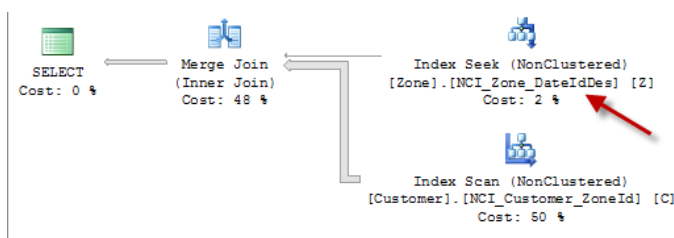
Quando scriviamo la clausola della *WHERE* all'interno delle nostre query occorre tenere presente che se vogliamo che venga utilizzato un indice della tabella, non dobbiamo applicare delle funzioni alla colonna che desideriamo filtrare.

Torniamo agli esempi precedenti, considerando di avere creato tutti gli indici proposti.

Se scriviamo questa query:

```
select C.IdCustomer, Z.IdZone from Customer as C
join Zone as Z on Z.IdZone=C.IdZone
where Z.RegisterDate='19990820'
```

E analizziamo il piano di esecuzione:

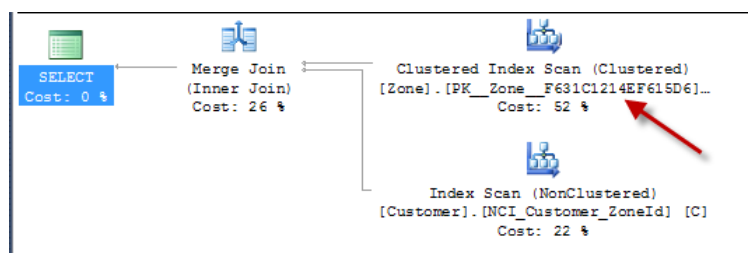


Ci rendiamo conto che stiamo utilizzando l'indice corretto, creato appositamente.

Se scriviamo invece la query nel seguente modo:

```
select C.IdCustomer, Z.IdZone from Customer as C
join Zone as Z on Z.IdZone=C.IdZone
where dateadd(day,1,Z.RegisterDate)='19990820'
```

E analizzando il piano di esecuzione:



Ci rendiamo conto che non riusciamo a sfruttare l'indice della tabella.

Pertanto una regola da tener presente è quella di non 'sporcare' le colonne con formule o campi calcolati, altrimenti non riusciremo a sfruttare gli indici per incrementare le prestazioni.

Le Join

Ogni volta che eseguiamo un'operazione di Join tra due tabelle dobbiamo sempre preferire, ove possibile, un'operazione di INNER, rispetto ad una LEFT o RIGHT e sicuramente cercare di evitare le CROSS.

Il motivo risiede principalmente nella possibilità di far elaborare meno righe possibili alla query, pertanto se un campo della tabella A è obbligatorio ed è relazionato col campo della tabella B, allora è consigliato usare una INNER JOIN, se il campo della tabella A non fosse obbligatorio, allora sarebbe necessaria un'operazione di LEFT.

Questa regola, per quanto banale, ci permette di tenere sotto controllo il numero delle righe elaborate da una query, pertanto quando eseguiamo un'operazione di Join analizziamo sempre che tipo di relazione stiamo creando.

Riepilogo

SQL legge e scrive per pagine di dati composte da 8K nel quale devono essere inseriti più record possibili. Per farlo occorre che le nostre tabelle siano sufficientemente 'snelle' ed eventualmente suddivise in più parti.

Il piano di esecuzione corrisponde al 'come' il motore SQL restituirà i dati richiesti e da esso è possibile capire:

- Il dettaglio delle operazioni eseguite
- I tempi di elaborazione
- Su quali indici si sta lavorando

Gli indici ci consentono di migliorare le prestazioni delle query, ma vanno 'dosati' in base al contesto di utilizzo della tabella.

E' sconsigliato creare molti indici su una tabella soggetta a frequenti inserimenti

Gli indici di copertura vengono creati per soddisfare esigenze di query che richiedono spesso gli stessi dati e filtri.

Per sfruttare gli indici è necessario non applicare funzioni alle colonne della *WHERE*.

E' sempre consigliato preferire, ove possibile, le INNER JOIN per relazionare due tabelle in una query

Esercitazioni di approfondimento (4)

Esercitazione "Query"

Requisiti per lo svolgimento delle esercitazioni

Per eseguire l'esercitazione, è necessario aver installato i sorgenti del corso Builder e creare un nuovo progetto di personalizzazione (es. "TUT_AP") all'interno dell'area/prodotto installata (se già creato, questa parte può essere saltata). È necessario l'uso della guida e una buona conoscenza degli argomenti: main, moduli e sub, strutturazione del codice, variabili, variabili di ambiente e passaggio parametri.

Durata prevista dell'esercitazione 4h.

Modalità di esecuzione dell'esercitazione

Creare un main in cui andare ad inserire i moduli dei singoli esercizi.

Il nome del main deve includere il riferimento all'esercitazione.

Il nome del modulo deve includere il riferimento dell'esercitazione e dell'esercizio che si sta svolgendo.

Il nome della sub deve includere il riferimento dell'esercitazione e dell'esercizio.

Il nome della dll in cui inserire le sub sviluppate deve contenere il riferimento dell'esercitazione.

Se non esplicitamente richiesto, il modulo deve contenere esclusivamente la sub che svolge l'esercizio.

Nel caso in cui l'esercizio richieda di aggiungere delle funzionalità all'esercizio precedente, si dovrà procedere con la duplicazione di tutti gli oggetti che compongono il programma (sub, classi, tipi dato) il quale avranno nel nome l'indicazione del nuovo esercizio.

 Tutti gli esercizi devono essere gestiti con gli oggetti query.

Esempi

Nome del main: ESE01 si riferisce al main della prima esercitazione

Nome del modulo: MOD01_02 si riferisce al secondo esercizio della prima esercitazione

Nome della sub: SU01_03_** si riferisce all'esercizio 3 della prima esercitazione

Nome della DLL: DLL01 si riferisce alla prima esercitazione

Esercizio 1

Creare un nuovo programma in modo da soddisfare le seguenti caratteristiche:

- definire una query che estragga le seguenti informazioni dalle tabelle BLDART e BLDFAMIGLIA:
 - codice articolo
 - descrizione articolo
 - codice famiglia
 - descrizione famiglia
- la query deve essere ordinata per codice articolo decrescente e codice famiglia crescente;
- leggere la query ed esporre con un messaggio a video i valori del codice articolo, codice famiglia e descrizione famiglia;
- verificare il comando SQL prodotto attivando il BCProfiler per tracciare la Query.

Esercizio 2

Prendere come riferimento l'esercizio 1 e creare un nuovo programma in modo da soddisfare le seguenti caratteristiche:

- introdurre un campo calcolato che computi il prezzo scontato effettuato sul singolo articolo (utilizzare i campi BLDART.PREZZO e BLDART.SCONTO);
- leggere la query ed esporre con un messaggio a video i valori del codice articolo, codice famiglia, descrizione famiglia e il nuovo campo calcolato;
- verificare il comando SQL prodotto attivando il BCProfiler per tracciare la Query.

Esercizio 3

Creare un nuovo programma in modo da soddisfare le seguenti caratteristiche:

- definire una query che estragga le seguenti informazioni dalle tabelle BLDART e BLDFAMIGLIA:
 - codice famiglia (BLDART)
 - descrizione famiglia (BLDFAMIGLIA)
 - un campo calcolato che computi la sommatoria della Quantità in magazzino (BLDART);
- i record devono essere aggregati per codice famiglia e per descrizione famiglia;
- la query deve essere ordinata per codice famiglia crescente;
- leggere la query ed esporre con un messaggio a video i valori del codice famiglia, descrizione famiglia e la quantità in magazzino;
- verificare il comando SQL prodotto attivando il BCProfiler per tracciare la Query.

Esercizio 4

Prendere come riferimento l'esercizio 3 e creare un nuovo programma in modo da soddisfare le seguenti caratteristiche:

- devono essere selezionate SOLO le famiglie comprese tra codice "02" e "03";
- leggere la query ed esporre con un messaggio a video i valori del codice famiglia, descrizione famiglia e la quantità in magazzino;
- verificare il comando SQL prodotto attivando il BCProfiler per tracciare la Query.

Esercizio 5

Creare un nuovo programma in modo da soddisfare le seguenti caratteristiche:

- definire una query che estragga le seguenti informazioni dalle tabelle BLDART e BLDRIGA:
 - codice articolo
 - descrizione articolo

- sommatoria della quantità ordinata (alias TotQtaOrd)
- devono essere estratti SOLO gli articoli che presentano una quantità totale ordinata (TotQtaOrd) maggiore di 1.000;
- devono essere ordinati in maniera decrescente per la quantità totale;
- leggere la query ed esporre con un messaggio a video i valori dei campi estratti;
- verificare il comando SQL prodotto attivando il BCProfiler per tracciare la Query.

Esercizio 6

Creare un nuovo programma in modo da soddisfare le seguenti caratteristiche:

- definire una query che estragga le seguenti informazioni dalla tabella BLDART:
 - codice articolo
 - descrizione articolo
 - Prezzo
 - campo calcolato "MediaPrezzo" con la media del prezzo di TUTTI gli articoli
 - campo calcolato "DifferenzaPrezzo" con la differenza tra il Prezzo dell'articolo e il campo calcolato "MediaPrezzo".

Nel caso di differenza negativa, azzerare il valore.

- devono essere estratti gli articoli della famiglia '01';
- la media di prezzo di tutti gli articoli deve essere calcolata con una subquery;
- leggere la query ed esporre con un messaggio a video tutti i campi estratti;
- verificare il comando SQL prodotto attivando il BCProfiler per tracciare la Query.

Esercizio 7

Creare un nuovo programma in modo da soddisfare le seguenti caratteristiche:

- definire una query che estragga le seguenti informazioni dalle tabelle BLDFAMIGLIA e BLDART:
 - codice famiglia
 - descrizione famiglia
- devono essere estratte le SOLE famiglie che abbiano degli articoli con il codice marca valorizzato;
- il filtro deve essere eseguito con una subquery;
- leggere la query ed esporre con un messaggio a video i valori del codice famiglia e descrizione famiglia;
- verificare il comando SQL prodotto attivando il BCProfiler per tracciare la Query.

Esercizio 8

Prendere come l'esercizio precedente e creare un nuovo programma in modo da soddisfare le seguenti caratteristiche:

- sostituire la subquery con una CTE;
- verificare il comando SQL prodotto attivando il BCProfiler per tracciare la Query.

Operazioni su DataBase

- [Gestione degli errori provenienti dal database SQL](#)
- [Operazioni di modifica dati automatica](#)
- [Gestione lock applicativo](#)
- [Gestione delle transazioni](#)
- [Mapping tipi dato tra BC e Microsoft SQL Server](#)
- [Cast sui tipi DATE e DATETIME](#)
- [Sostituzione dei nomi estesi delle tabelle](#)
- [Associazione del campo DbGruppo nella relazione tra tabelle](#)
- [Semplificazioni e ottimizzazioni nella clausola WHERE](#)

Gestione degli errori provenienti dal database SQL

- [Intercettazione dell'errore di deadlock](#)
- [Intercettazione dell'errore di connessione](#)

Intercettazione dell'errore di deadlock

Su Microsoft SQLServer se si eseguono istruzioni SQL senza che sia attiva una transazione, SQL Server apre (d'ufficio) una transazione costituita dalla singola istruzione SQL e quindi la chiude immediatamente dopo.

Tale transazione è detta automatica. L'errore di deadlock si può verificare quindi anche quando non ci sono transazioni esplicitamente aperte (si sono verificati dei casi di assistenza).

In questo caso l'ambiente gestisce l'errore di deadlock nel seguente modo:

Viene ritentata l'unica istruzione SQL che compone la transazione per un massimo prefissato(5). Tra un tentativo ed un altro si è messo un ritardo di 1 secondo (il processo è sospeso)

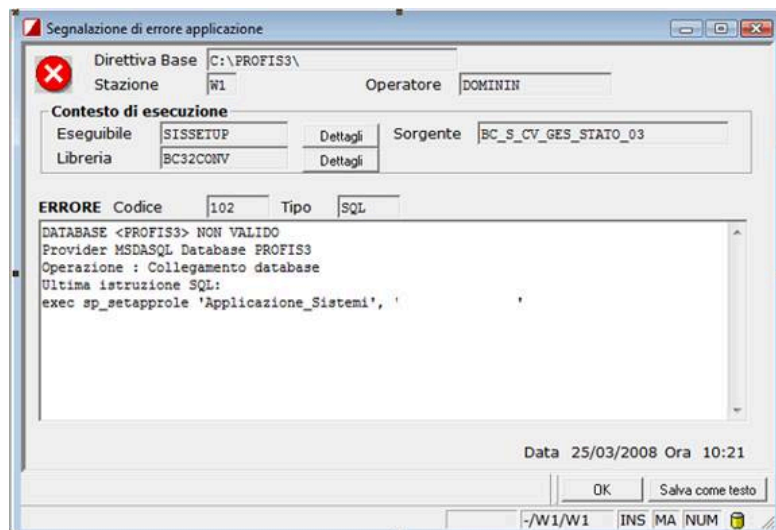
Se dopo i tentativi l'errore persiste, l'errore di deadlock è gestito nella maniera consueta, ovvero con emissione del messaggio di errore a video e con l'interruzione del processo.

Intercettazione dell'errore di connessione

Quando un programma Sistemi operante in ambiente SQL parte, la prima operazione di accesso ai dati apre una connessione verso il database. La connessione rappresenta un'entità logica nella quale client e server mantengono le informazioni necessarie alla comunicazione.

Normalmente la connessione è mantenuta fino al termine del programma. In taluni casi, se il programma esegue l'operazione @CL, la connessione può essere chiusa anticipatamente. La connessione viene ristabilita in automatico dall'ambiente alla successiva operazione di accesso ai dati.

In talune circostanze, se il tempo tra la chiusura della connessione e la successiva riapertura è molto ridotto, il client non riesce ad effettuare la connessione verso il database ed il programma termina con la seguente emissione di errore:



Nel messaggio riferisce che il database non è valido. Nella realtà per problemi di carico e di tempistiche, è il motore SQL che non riesce a stabilire la connessione in tempo che produce l'anomalia.

Per far fronte a questo comportamento anomalo, si è reintrodotta nell'ambiente un meccanismo di recupero della connessione, effettuando più tentativi (più precisamente 2):

Quando un programma deve stabilire la connessione verso il database, l'ambiente prova a collegarsi

Se la connessione fallisce, viene ritentata una seconda volta a 10 secondi di distanza dalla prima

Questo meccanismo **non interessa** il funzionamento della CHECKDBPARAM.

SQL Server - Errore di database non valido e opzione AUTOCLOSE

SQL Server prevede un'opzione di gestione dei file che costituiscono fisicamente la base dati tale da permettere operazioni di manutenzione su di essi (ad esempio backup,...). L'opzione in oggetto è l'opzione AUTO_CLOSE.

Letteralmente impone, se attiva, il rilascio di qualunque risorsa (file, memoria, ecc) da parte del motore quando l'ultimo utente chiude una connessione verso un database.

L'opzione è impostata diversamente in funzione della versione di SQL Server:

Attivata d'ufficio per le versioni Express Edition

Disattivata per le altre versioni.

Questa opzione ha importanza nel contesto delle applicazioni Sistemi perché durante la fase di Setup (ma potenzialmente su altre categorie di programmi) produce un errore di login fallito.

Lo scenario è il seguente:

il setup esegue delle operazioni verso il database di aggiornamento/adeguamento

Il setup chiude le connessioni verso il database (esegue la funzione @CL)

Il setup riapre la connessione al database (esegue una qualunque specifica accesso ai dati sul db).

L'errore appare nel punto 3. Sembrano fattori catalizzatori:

La velocità di passaggio dal punto 2 al punto 3, ovvero la richiesta rapida di passaggio dallo stato chiuso a quello aperto del database.

Il carico della macchina. ovvero poco prestante o vetusta.

La presenza di antivirus (l'antivirus potrebbe eseguire delle operazioni di "scan" sui file dati).

Il problema potrebbe potenzialmente verificarsi anche a programmi applicativi che facciano quindi una chiusura / apertura verso il database.

Operazioni di modifica dati automatica

- [Variazione automatica date/periodi](#)
- [Copia dati tra gruppi](#)
- [Eliminazione dati da gruppi](#)
- [Acquisizione dati da altro DB \(solo copia tabelle\)](#)
- [Acquisizione dati da altro DB \(copia tabelle e dizionari\)](#)

Variazione automatica date/periodi

eSOLVER ed i prodotti in generale distribuiscono delle tabelle DEMO da installare con il prodotto per fornire delle casistiche di utilizzo ai partner che fanno delle demo agli utenti.

Queste tabelle, e le relative tabelle, sono caricati dal gestionale con le date definite al momento dell'immissione dell'ordine, della fattura, della bolla, ecc...

eSOLVER ha una particolare gestione per cui i programmi di interrogazione visualizzano i dati relativi agli ultimi 30 giorni.

Questo in generale per limitare il peso delle interrogazioni laddove ci fossero molti dati.

Esempio di problematica:

- si caricano gli ordini al 25/2/2010 ai fini della demo
- queste tabelle sono valide dal 25/2/2010 sino al 31/12/2010
- alla data 14/4/2010, giorno in cui viene fatta la demo, le interrogazioni di eSOLVER visualizzano gli elenchi vuoti (perché impostano da prodotto il periodo 14/3/2010-14/4/2010)

L'esigenza è quindi, date le tabelle di eSOLVER ed in generale del prodotto, avere una funzione che analizzi tutte le tabelle presenti in esolver e individui i campi di tipo data o periodo avanzandoli nel tempo.

Per eseguire questa operazione, è necessario quindi predisporre un programma che:

1. definisca il numero di mesi con il quale si intende incrementare i campi data/periodo
2. qual è il dizionario dove sono presenti le tabelle
3. quali sono i campi da non incrementare, inserendoli in una struttura
4. e quindi avvii l'incremento attraverso il richiamo della sub di ambiente **BC32UTI. BC_S_VARIA_DATA_TABELLA.**

♥ La funzione di variazione gestisce oltre l'incremento anche il decremento del campo data permettendo l'utilizzo di un numero di mesi negativo.

Definizione delle strutture

La struttura che conterrà l'elenco dei campi da escludere durante l'operazione di incremento deve essere definita con i seguenti campi:

Campo	Descrizione
NOME_DFX\$[20]	Nome del DFX da non prendere in considerazione durante l'incremento\decremento della data.
CAMPO\$[20]	Campo appartenente ad un DFX (indicato nel campo NOME_DFX\$) che non verrà preso in considerazione durante l'operazione di incremento data. Nel caso in cui non sia valorizzato verrà esclusa tutta la tabella.

La struttura di output che conterrà l'elenco dei campi analizzati e incrementati è uguale a quella sopra indicata.

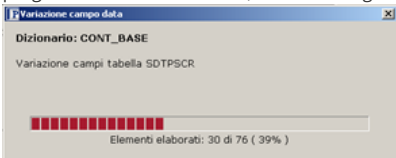
Richiamo della sub BC32UTI.BC_S_VARIA_DATI_TABELLA

Una volta predisposto la struttura contenente i campi delle tabelle da escludere, il programma deve richiamare la sub **BC_S_VARIA_DATA_TABELLA**, esportata dalla libreria di ambiente **BC32UTI.dll**.

Se non viene definita la struttura, l'operazione comprenderà tutte le tabelle presenti nel dizionario.

♥ La funzione di variazione agisce sul database attivo al momento dell'esecuzione del programma

La sub **BC_S_VARIA_DATA_TABELLA** dispone della seguente interfaccia:

Parametro	I/O	Descrizione	Note
MODALITA[1]	I	Modalità di richiamo	(0) Previsto per usi futuri
NOME_DIZ\$[20]	I	Nome del dizionario	Sigla del dizionario dati di cui si elaborano le tabelle
VID_PROG[1]	I	Visualizzazione progress bar	(0-No; 1-Si) - Se indicato a 1 viene visualizzata la progress bar di avanzamento, simile alla seguente: 
N_MESI[2]	I	Numero di mesi	Numero di mesi (può essere > 12, e negativo) che andranno ad incrementare il valore dei campi data presenti nelle tabelle
STRUT_INP\$[20]	I	Nome della struttura in input	Struttura contenente le tabelle e i campi da escludere
STRUT_OUT\$[20]	O	Nome della struttura in output	Struttura contenente le tabelle e i campi elaborati

Copia dati tra gruppi

Laddove si presenti la necessità di effettuare il trasferimento diretto dei dati contenuti nelle tabelle di un database di un gruppo specifico, si rende disponibile una funzione di copia dati di un elenco di tabelle da un gruppo di origine, a uno o più gruppi di destinazione.

Il presupposto è che preventivamente sia stata creata una lista di tabelle generate dalla funzione "Definizione elenchi predefiniti di tabelle", presente nelle "Altre operazioni" del BCFSQL.

Per eseguire questa operazione, è necessario quindi predisporre un programma che:

1. - imposti correttamente tutte le coordinate di ambiente (direttive, database, ...)
2. - individui il file contenente l'elenco predefinito di tabelle
3. - richiami la funzione di ambiente **BC32UTI.GR_S_BCF_SERVICE_COPIA_GRUPPI**

Richiamo della sub BC32UTI.GR_S_BCF_SERVICE_COPIA_GRUPPI

Una volta predisposto l'elenco delle tabelle da copiare, il programma deve richiamare la sub **GR_S_BCF_SERVICE_COPIA_GRUPPI**, esportata dalla libreria di ambiente **BC32UTI.dll**.

♥ La funzione di copia agisce sul database attivo al momento dell'esecuzione del programma

La sub **GR_S_BCF_SERVICE_COPIA_GRUPPI** dispone della seguente interfaccia:

Parametro	I/O	Descrizione	Note
FILE_ELENCO\$[20]	I	Nome file elenco tabelle	Nome del file contenente l'elenco delle tabelle
PATH_ELENCO\$[255]	I	Path collocazione file elenco tabelle	Directory del file contenente l'elenco delle tabelle
GRUPPO_ORIGINE\$[2]	I	Gruppo origine copia	Gruppo da cui effettuare la copia

GRUPPI_DESTINAZIONE?[]	I	Gruppo destinazione copia	Gruppi su cui effettuare la copia (indicare * per copiare su tutti i gruppi già esistenti). Attenzione a non indicare tra questi anche il gruppo di origine. Indicare l'elenco dei gruppi separati da virgola.
CREA_GRUPPIDEST[1]	I	Crea gruppi in destinazione se non presenti sulla tabella SiGruppi	Indica se creare i gruppi qualora non siano esistenti sulla tabella gruppi
PATH_BASE\$[100]	I	Direttiva base	Indicare la direttiva base

Eliminazione dati da gruppi

Laddove si presenti la necessità di effettuare l'eliminazione diretta dei dati contenuti nelle tabelle di un database di un gruppo specifico, si rende disponibile una funzione di eliminazione dati di un elenco di tabelle da uno o più gruppi. Il presupposto è che preventivamente sia stata creata una lista di tabelle generate dalla funzione "Definizione elenchi predefiniti di tabelle", presente nelle "Altre operazioni" del BCFSQL.

Per eseguire questa operazione, è necessario quindi predisporre un programma che:

1. imposti correttamente tutte le coordinate di ambiente (direttive, database, ...)
2. individui il file contenente l'elenco predefinito di tabelle
3. richiami la funzione di ambiente **BC32UTI.GR_S_BCF_SERVICE_DEL_GRUPPI**

Richiamo della sub **BC32UTI. GR_S_BCF_SERVICE_COPIA_GRUPPI**

Una volta predisposto l'elenco delle tabelle da eliminare, il programma deve richiamare la sub **GR_S_BCF_SERVICE_DEL_GRUPPI**, esportata dalla libreria di ambiente **BC32UTI.dll**.

♥ La funzione di eliminazione agisce sul database attivo al momento dell'esecuzione del programma

La sub **GR_S_BCF_SERVICE_DEL_GRUPPI**, dispone della seguente interfaccia:

Parametro	I/O	Descrizione	Note
FILE_ELENCO\$[20]	I	Nome file elenco tabelle	Nome del file contenente l'elenco delle tabelle
PATH_ELENCO\$[255]	I	Path collocazione file elenco tabelle	Directory del file contenente l'elenco delle tabelle
GRUPPO_ORIGINE\$[2]	I	Gruppo origine copia	Gruppo da cui effettuare la copia
GRUPPI_ELIMINAZIONE?[]	I	Gruppi da eliminare	Gruppi su cui effettuare l'eliminazione. Indicare * se si desidera eliminare i dati da TUTTI i gruppi presenti. Indicare l'elenco dei gruppi separati da virgola.
ELIMINA_GRUPPI[1]	I	Elimina gruppi selezionati dalla tabella SiGruppi (0-No;1-Si)	Indicare se eliminare i gruppi dalla tabella gruppi
PATH_BASE\$[100]	I	Direttiva base	Indicare la direttiva base

Acquisizione dati da altro DB (solo copia tabelle)

Laddove si presenti la necessità di effettuare il trasferimento diretto dei dati contenuti da un database ad un altro, si rende disponibile una funzione di copia dati di un elenco di tabelle da un database di origine, a uno di destinazione. Il presupposto è che preventivamente sia stata creata una lista di tabelle generate dalla funzione "Definizione elenchi predefiniti di tabelle", presente nelle "Altre operazioni" del BCFSQL.

Per eseguire questa operazione, è necessario quindi predisporre un programma che:

1. imposti correttamente tutte le coordinate dell'ambiente di origine dei dati (stringa di connessione al DB)
2. imposti correttamente tutte le coordinate dell'ambiente di destinazione dei dati (stringa di connessione al DB)
3. individui il file contenente l'elenco predefinito di tabelle
4. richiami la funzione di ambiente **BC32UTI. GR_S_BCF_SERVICE_COPIA_TABELLE**

Richiamo della sub BC32UTI. GR_S_BCF_SERVICE_COPIA_TABELLE

Una volta predisposto l'elenco delle tabelle da copiare, il programma deve richiamare la sub **GR_S_BCF_SERVICE_COPIA_TABELLE**, esportata dalla libreria di ambiente **BC32UTI.dll**.

♥ La funzione di copia agisce sul database attivo al momento dell'esecuzione del programma, come database di destinazione dei dati acquisiti

La sub **GR_S_BCF_SERVICE_COPIA_TABELLE** dispone della seguente interfaccia:

Parametro	I/O	Descrizione	Note
MODALITA[1]	I	Modalità: 0-Controllo dizionari; 1-Controllo e COPIA	Se si vuole preventivamente controllare solo l'allineamento dei dizionari dati sui 2 database, prima di effettuare la copia, si può utilizzare la modalità 0-Controllo
FILE_ELENCO[20]	I	Nome file elenco tabelle	Nome del file contenente l'elenco delle tabelle
PATH_ELENCO[255]	I	Path collocazione file elenco tabelle	Directory del file contenente l'elenco delle tabelle
STRINGA_CONN_ORIG[500]	I	Stringa di connessione DB di origine dei dati	Deve essere composta con la specifica '@SETCONNSTRING
STRINGA_CONN_DEST[500]	I	Stringa di connessione DB di destinazione dei dati	Deve essere composta con la specifica '@SETCONNSTRING
PATH_BASE[100]	I	Direttiva base	Indicare la direttiva base

♥ Le tabelle già presenti nel DB di destinazione vengono preventivamente svuotate e poi popolate prelevando i dati dal DB di origine.

Affinché la copia possa avvenire con successo, è necessario che la versione delle tabelle tra i due DB (e quindi dei dizionari dati) sia allineata.

Acquisizione dati da altro DB (copia tabelle e dizionari)

La funzione estende il comportamento della funzione simile "Acquisizione dati da altro DB (solo copia tabelle)".

Qualora si presenti la necessità di effettuare il trasferimento diretto dei dati contenuti da un database ad un altro, e nel database di destinazione alcuni dizionari non sono presenti la funzione permette anche di trasferire i dizionari inesistenti e nel dettaglio si preoccupa di:

1. Copiare i dizionari inesistenti sulla destinazione dall'origine
2. Creare le tabelle per i dizionari inesistenti

Il presupposto è che preventivamente sia stata creata una lista di tabelle generate dalla funzione "Definizione elenchi predefiniti di tabelle", presente nelle "Altre operazioni" del BCFSQL.

Se i dizionari esistenti e interessati dal trasferimento delle tabelle sono disallineati, la funzione non viene eseguita e viene emessa una videata di avviso.

Per il richiamo di questa funzione è necessario che il programma:

1. imposti le coordinate di origine dei dati (cartelle e database)
2. imposti le coordinate di destinazione dei dati (cartelle e database)
3. richiami la funzione **BC32UTI. BC_S_COPIA_TABELLE_DIZ**

Non è sufficiente la stringa di connessione al database (o solo le informazioni di collegamento al database) perché in caso di dizionari dati assenti viene eseguita fisicamente la copia tra le cartelle di origine e di destinazione.

La funzione può essere utilizzata in sostituzione di **GR_S_BCF_SERVICE_COPIA_TABELLE** in quanto il comportamento di base è lo stesso.

Richiamo della funzione BC32UTI. BC_S_COPIA_TABELLE_DIZ

La funzione **BC_S_COPIA_TABELLE_DIZ** dispone della seguente interfaccia:

Parametro	I/O	Descrizione	Note
-----------	-----	-------------	------

MOD_COPIA[1]	I	Modalità copia: 0-controllo dizionari; 1-Controllo e COPIA	Se si vuole preventivamente controllare solo l'allineamento dei dizionari dati sui 2 database, prima di effettuare la copia, si può utilizzare la modalità 0-Controllo
FL_CONTROLLO_DIZ[1]	I	Controllo dizionari: 0-Signala disallineati e inesistenti; 1-Signala solo disallineati (nella copia trasferisce anche i dizionari assenti)	Se i dizionari sulla destinazione devono essere perfettamente allineati a quelli di origine indicare la modalità 0. Se è permesso il trasferimento di dizionari inesistenti sulla destinazione impostare la modalità 1.
FILE_ELENCO\$[20]	I	Nome file elenco tabelle	Nome del file contenente l'elenco delle tabelle
PATH_ELENCO\$[255]	I	Direttiva di collocazione file elenco tabelle	Directory del file contenente l'elenco delle tabelle
SOURCE_BASE\$[100]	I	Direttiva base del dell'installazione origine	
SOURCE_DATI\$[100]	I	Direttiva dati dell'installazione origine	
SOURCE_PROG\$[100]	I	Direttiva programmi dell'installazione origine	
SOURCE_DB[1]	I	Sorgente dati del database origine	Indicare 1 (SQL Server) o ZDBSORGENTE
SOURCE_DBSERVER\$[40]	I	Server del database origine	
SOURCE_DBAUT[1]	I	Tipo autenticazione del database origine	0-SQL Server 1-Windows
SOURCE_DBUSER\$[40]	I	Utente server origine	Da indicare solo se il tipo autenticazione è SQL Server
SOURCE_DBPWD\$[40]	I	Password server origine	Da indicare solo se il tipo autenticazione è SQL Server
SOURCE_DBPROD\$[40]	I	Nome del database origine	
TARGET_DB[1]	I	Sorgente dati del database destinazione	Indicare 1 (SQL Server) o ZDBSORGENTE
TARGET_DBSERVER\$[40]	I	Server del database destinazione	
TARGET_DBAUT[1]	I	Tipo autenticazione del database destinazione	0-SQL Server 1-Windows
TARGET_DBUSER\$[40]	I	Utente server destinazione	Da indicare solo se il tipo autenticazione è SQL Server
TARGET_DBPWD\$[40]	I	Password server destinazione	Da indicare solo se il tipo autenticazione è SQL Server
TARGET_DBPROD\$[40]	I	Nome del database destinazione	
ERRORE_COPIA[2]	O	Errore durante la copia	0-Nessuno 1-Controllo: Dizionari incoerenti; 2-Copia dizionari non riuscita

♥ Le tabelle già presenti nel DB di destinazione vengono preventivamente svuotate e poi popolate prelevando i dati dal DB di origine.

Gestione lock applicativo

- [Possibili impieghi della specifica '@DEFAPPLOCK](#)

Possibili impieghi della specifica '@DEFAPPLOCK

Avviare un solo DAS

Per rendere certo che un solo programma (DAS) sia attivo in una installazione, il programma crea un lock di nome univoco per l'intera installazione:

BC

```
'
DAS[1]=0

'

'@DEFAPPLOCK NOME["DAS"] _
        GESTIONEDATI[GOSUB ELABORAZIONE] _
        GESTIONEERRORE[DAS=1] _
        TIMEOUT[0]

'

IF DAS=1 THEN

    '@PRB \IL DAS E' GIA' ATTIVO SU UN ALTRO PC\ T[]

ENDIF

'
```

il timeout a 0 significa che la specifica ritorna subito (richiamando GESTIONEERRORE) se il lock è impegnato;

il parametro TIPO non è specificato, si assume SESSIONE;

il parametro MODO non è specificato, si assume ESCLUSIVO

"Transazioni" di programma

In alcuni casi ci sono programmi che devono essere eseguiti come una sola istanza (ad esempio ricostruzione progressivi). E' uno scenario simile al precedente.

DAS[1]=0

Gestione delle transazioni

- [Impiego annidato della '@DEFTRANSACTION](#)

Impiego annidato della '@DEFTRANSACTION

La specifica '@DEFTRANSACTION è impiegata per gestire le transazioni in ambito BC/SQL. Ad esempio:

```
BC

'@DEFTRANSACTION GESTIONEDATI[GOSUB UPD]
        GESTIONEDEADLOCK[GOSUB AVVISODEADLOCK]

'

...

'

'=====
'@SR UPD
'=====

'

'@FINDDBDATA TABELLA[ECFAN] _
        TIPOREC[CF] _
        IND[0 \CF=1\CCF=COD_CLI\] _
        ERRORE[#NOKEY]

'

CFPROG=CFPROG+1

'

'@UPDATEDBDATA TABELLA[ECFAN] _
        TIPOREC[CF] _
        IND[0 \CF=1\CCF=COD_CLI\] _
        ERRORE[#NOKEY]

'

RETURN
```

permette di eseguire le operazioni presenti nella sub BC UPD all'interno di una transazione SQL. La sequenza di istruzioni SQL emesse è la seguente (se non ci sono errori o deadlock) :

BC
SET TRANSACTION ISOLATION LEVEL READ COMMITTED
BEGIN TRANSACTION
SELECT ... FROM ECFAN
UPDATE ECFAN SET (...)
COMMIT

La sub UPD dell'esempio precedente è un esempio semplice, dove è chiaro il punto di partenza della transazione. Talora capita che due istruzioni '@DEFTRANSACTION siano annidate, ovvero quando la sub di gestione dati della prima '@DEFTRANSACTION richiama (tipicamente in una sub esterna) una seconda '@DEFTRANSACTION.

Ricordando che in SQL non è possibile "annidare" le transazioni, il comportamento che si adotta in questi casi è:

La prima istruzione '@DEFTRANSACTION inizia la transazione. La sua esecuzione emette l'istruzione "BEGIN TRANSACTION" verso il database server;

Le operazioni di aggiornamento della prima '@DEFTRANSACTION avvengono detto la transazione aperta;

la seconda istruzione '@DEFTRANSACTION non apre una seconda transazione, anche se emette l'istruzione "BEGIN TRANSACTION". In questo caso il database server incrementa un proprio contatore, reperibile come variabile con la sintassi: @@TRANCOUNT. L'istruzione '@DEFTRANSACTION opera similmente al database server, cioè mantenendo un proprio contatore interno;

La fine della callback di aggiornamento della seconda istruzione '@DEFTRANSACTION, emette verso il database server l'istruzione SQL "IF @@TRANCOUNT > 0 COMMIT TRAN". Quest'istruzione, essendo eseguita dentro una transazione annidata, di fatto decrementa semplicemente il contatore @@TRANCOUNT di una unità. I dati della seconda '@DEFTRANSACTION di fatto non sono ancora stati "committati", né i lock sulle tabelle / record attivati dalla transazione in chiusura rimossi. L'ambiente opera in modo parallelo a quanto fatto dal database e decrementa il proprio contatore delle transazioni annidate;

La fine della callback di aggiornamento della prima istruzione '@DEFTRANSACTION esegue la seconda istruzione "IF @@TRANCOUNT > 0 COMMIT TRAN". In questo momento, dato che il contatore @@TRANCOUNT ritorna al valore 0, allora il database server "committa" i dati della prima e della seconda transazione;

Se, all'interno della procedura di aggiornamento dati della seconda '@DEFTRANSACTION avviene un deadlock, la transazione è abortita. In questo caso è eseguita la callback GESTIONEDEADLOCK della prima '@DEFTRANSACTION. La variabile del database server @@TRANCOUNT torna automaticamente al valore iniziale 0;

Se all'interno della procedura di aggiornamento dati della seconda '@DEFTRANSACTION, il programma esegue l'istruzione '@ROLLBACKTRANSACTION, l'ambiente emette l'istruzione "IF @@TRANCOUNT > 0 ROLLBACK TRAN". Quest'istruzione porta il contatore @@TRANCOUNT al valore 0 e la transazione viene abortita. La specifica '@DEFTRANSACTION non ha una callback di gestione del ROLLBACK. L'interruzione della transazione va gestita quindi da programma, anche per la prima '@DEFTRANSACTION.

Ripristino del livello di isolamento della transazione

Quando una transazione annidata si completa, l'ambiente, oltre a decrementare il contatore delle transazioni annidate (sia interno che del database server) esegue un'operazione di ripristino del livello di isolamento precedente (ovvero quello impostato per la '@DEFTRANSACTION esterna).

Ad esempio:

BC

```
'@DEFTRANSACTION GESTIONEDATI[GOSUB UPD]
    GESTIONEDEADLOCK[GOSUB AVVISODEADLOCK]
    LIVELLOISOLAMENTO[#REPEATABLE]

'
...
'

'=====
'@SR UPD
'=====

'

'@FINDDBDATA TABELLA[ECFAN] _
    TIPOREC[CF] _
    IND[0 \CF=1\CCF=COD_CLI\] _
    ERRORE[#NOKEY]

'

CFPROG=CFPROG+1

'

'@UPDATEDBDATA TABELLA[ECFAN] _
    TIPOREC[CF] _
    IND[0 \CF=1\CCF=COD_CLI\] _
    ERRORE[#NOKEY]

'

'@DEFTRANSACTION GESTIONEDATI[GOSUB NESTEDUPD]
    LIVELLOISOLAMENTO[#SERIALIZABLE]

'

RETURN

'

'=====
'@SR NESTEDUPD
'=====

'

...

'

RETURN
```

La '@DEFTRANSACTION interna imposta un livello di isolamento SERIALIZABLE (3). Al suo termine e prima della conclusione della '@DEFTRANSACTION esterna l'ambiente ripristina il livello REPEATABLE_READ.

Mapping tipi dato tra BC e Microsoft SQL Server

Di seguito viene elencata la corrispondenza tra i tipi dato BC e Microsoft SQL Server.

TIPO BC	MSQL 2000	MSQL 2005	MSQL 2008	MSQL 2008 R2	MSQL 2012
A	DATETIME	DATETIME	DATE	DATE	DATE
C	VARCHAR	VARCHAR	VARCHAR	VARCHAR	VARCHAR
D	DECIMAL	DECIMAL	DECIMAL	DECIMAL	DECIMAL
E	INTEGER	INTEGER	INTEGER	INTEGER	INTEGER
I	SMALLINT	SMALLINT	SMALLINT	SMALLINT	SMALLINT
L	INTEGER	INTEGER	INTEGER	INTEGER	INTEGER
N	SMALLINT/TINYINT	SMALLINT/TINYINT	SMALLINT/TINYINT	SMALLINT/TINYINT	SMALLINT/TINYINT
S	DOUBLE	DOUBLE	DOUBLE	DOUBLE	DOUBLE
B	TEXT	VARCHAR(MAX)	VARCHAR(MAX)	VARCHAR(MAX)	VARCHAR(MAX)

 A partire dall'ambiente 21.0 la versione di Microsoft SQL Server del database, incide sulla determinazione del tipo, pertanto a fronte di una modifica/creazione/rigenerazione di un campo Data o Blob su un dfx, la conversione potrà cambiare il tipo di dato SQL

Flag su CFG per disabilitare la conversione dei tipi

Nel caso non si volesse gestire il cambio dei tipi per versione è stato introdotto sul CFG il flag '**Disabilita conversione tipi**' all'interno della sezione [Configurazione database]. Questo flag se impostato a 1 disabilita le conversioni eseguite in automatico dall'ambiente, in caso di:

- Creazione di un nuovo campo data o blob su tabella
- Modifica di un campo data o blob di una tabella
- Operazioni di rigenerazione tabelle da BCMANSQL.

Cast sui tipi DATE e DATETIME

Per sopperire ad un errore di errata interpretazione delle date in base alla lingua del database, presente sui campi DATETIME con le versioni MSQl 2005 e precedenti, l'ambiente esegue un cast a DATETIME su ogni campo data del BC:

```
...  
OR ( EQCliForAn.DataValiditaDa <= CAST({d '2012-07-26'} AS DATETIME)  
    AND EQCliForAn.DataValiditaA >= CAST({d '2012-07-26'} AS DATETIME)  
    )  
...
```

Dalla versione 2008 e successive di MSQl questo errore è stato corretto.

Con l'ambiente 21 è stato eliminato il cast per le versioni di DB superiori alla 2005, pertanto la query verrà scritta nel seguente modo:

```
...  
OR ( EQCliForAn.DataValiditaDa <= {d '2012-07-26'}  
    AND EQCliForAn.DataValiditaA >= {d '2012-07-26'}  
    )  
...
```

Sostituzione dei nomi estesi delle tabelle

Per ridurre la grandezza della query e conseguentemente diminuire il passaggio di dati sulla rete è stata introdotta a partire dall'ambiente 21, la sostituzione dei nomi delle tabelle con l'alias [T1], [T2] etc..

Tale funzionamento è condizionato al valore del flag '**Usa alias di tabella brevi**' all'interno del CFG, nella sezione [Configurazione database]. Se impostato a 1 abilita il meccanismo di sostituzione.

Query con SUBARG=1 (Nessuna modifica ai nomi)

```
SELECT CASE WHEN ( ( Divisioni.DataInizioValidita <= CAST ({d '2013-07-10'} AS DATETIME)
AND ( Divisioni.DataFineValidita = CAST ({d '1800-01-01'} AS DATETIME)
OR Divisioni.DataFineValidita >= CAST ({d '2013-07-10'} AS DATETIME)
)
AND CAST ({d '2013-07-10'} AS DATETIME) <> CAST ({d '1800-01-01'} AS DATETIME)
)
OR CAST ({d '2013-07-10'} AS DATETIME) = CAST ({d '1800-01-01'} AS DATETIME)
) THEN 1
ELSE 0
END AS [ATTIVO] ,
Divisioni.CodDivisione AS [CodDivisione] ,
Divisioni.Des AS [Des] ,
Divisioni.DataInizioValidita AS [DataInizioValidita] ,
Divisioni.DataFineValidita AS [DataFineValidita] ,
Divisioni.FirmaUltVarData AS [FirmaUltVarData] ,
Divisioni.FirmaUltVarOperatore AS [FirmaUltVarOperatore] ,
Divisioni.Note AS [Note] ,
DivisioniLegami.CodDivisioneEsterno AS [CodDivisioneEsterno]
FROM Divisioni AS [Divisioni]
LEFT OUTER JOIN DivisioniLegami AS [DivisioniLegami] ON ( DivisioniLegami.CodDivisioneEsolver = Divisioni.CodDivisione
AND DivisioniLegami.Preferenziale = 1
AND ( DivisioniLegami.DBGruppo = 'CH' )
)
WHERE ( Divisioni.DBGruppo = 'CH' )
```

In questo caso la query utilizza 931 caratteri

Query con SUBARG=0 (Sostituzioni dei nomi delle tabelle)

```
SELECT CASE WHEN ( ( T1.DataInizioValidita <= CAST ({d '2013-07-10'} AS DATETIME)
AND ( T1.DataFineValidita = CAST ({d '1800-01-01'} AS DATETIME)
OR T1.DataFineValidita >= CAST ({d '2013-07-10'} AS DATETIME)
)
AND CAST ({d '2013-07-10'} AS DATETIME) <> CAST ({d '1800-01-01'} AS DATETIME)
)
OR CAST ({d '2013-07-10'} AS DATETIME) = CAST ({d '1800-01-01'} AS DATETIME)
) THEN 1
ELSE 0
END AS [ATTIVO] ,
T1.CodDivisione AS [CodDivisione] ,
T1.Des AS [Des] ,
T1.DataInizioValidita AS [DataInizioValidita] ,
T1.DataFineValidita AS [DataFineValidita] ,
T1.FirmaUltVarData AS [FirmaUltVarData] ,
T1.FirmaUltVarOperatore AS [FirmaUltVarOperatore] ,
T1.Note AS [Note] ,
T2.CodDivisioneEsterno AS [CodDivisioneEsterno]
FROM Divisioni AS [T1]
LEFT OUTER JOIN DivisioniLegami AS [T2] ON ( T2.CodDivisioneEsolver = T1.CodDivisione
AND T2.Preferenziale = 1
AND ( T2.DBGruppo = 'CH' )
)
WHERE ( T1.DBGruppo = 'CH' )
```

In questo caso la query utilizza 775 caratteri.

Associazione del campo DbGruppo nella relazione tra tabelle

A partire dalla versione 21.0 d'ambiente viene modificata la logica di attribuzione del campo DbGruppo nella relazione tra tabelle. In pratica al posto di relazionare il campo con una costante contenente il valore del gruppo corrente, viene relazionato con il campo DbGruppo della tabella padre.

Pertanto prima la query veniva creata con la seguente sintassi:

```
Ambiente 20.3 o precedente
```

```

SELECT ClientiFornitori.TipoAnagrafica AS [TipoAnagrafica]
FROM ClientiFornitori AS [ClientiFornitori]
      LEFT OUTER JOIN Zone AS [Zone] ON
          ( Zone.CodZona = ClientiFornitori.CodZona
            AND Zone.DBGruppo = 'LP' )
      LEFT OUTER JOIN ClientiFornitori AS [Tabella1] ON
          ( Tabella1.TipoAnagrafica = 1
            AND Tabella1.CodCliFor = ClientiFornitori.CodAnagrCuiFatturare
            AND Tabella1.DBGruppo = 'LP' )
      LEFT OUTER JOIN Zone AS [Zone_1] ON
          ( Zone_1.CodZona = Tabella1.CodZona
            AND Zone_1.DBGruppo = 'LP' )
WHERE ( ClientiFornitori.DBGruppo = 'LP' )

```

Adesso al posto della costante 'LP' viene messo il campo DbGruppo della tabella padre.

Ambiente 21.0 o successivi

```

SELECT ClientiFornitori.TipoAnagrafica AS [TipoAnagrafica]
FROM ClientiFornitori AS [ClientiFornitori]
      LEFT OUTER JOIN Zone AS [Zone] ON
          ( Zone.CodZona = ClientiFornitori.CodZona
            AND Zone.DBGruppo = ClientiFornitori.DBGruppo )
      LEFT OUTER JOIN ClientiFornitori AS [Tabella2] ON
          ( Tabella2.TipoAnagrafica = 1
            AND Tabella2.CodCliFor = ClientiFornitori.CodAnagrCuiFatturare
            AND Tabella2.DBGruppo = ClientiFornitori.DBGruppo )
      LEFT OUTER JOIN Zone AS [Zone_1] ON
          ( Zone_1.CodZona = Tabella2.CodZona
            AND Zone_1.DBGruppo = Tabella2.DBGruppo )
WHERE ( ClientiFornitori.DBGruppo = 'LP' )

```

Questa implementazione si è resa necessaria per riuscire a gestire le casistiche di interrogazione della base dati tra gruppi diversi.

Semplificazioni e ottimizzazioni nella clausola WHERE

Uno dei punti di partenza per migliorare le prestazioni di una query è semplificare la clausola WHERE. In genere nella clausola WHERE è opportuno evitare la presenza di funzioni che impieghino campi.

Questo perché la presenza di una funzione che opera su un campo impedisce di usare o di definire in modo automatico un indice per migliorare le prestazioni.

Seguono alcuni suggerimenti su singoli casi specifici

Filtro su date

Quando è necessario effettuare un filtro che operi su un campo data e occorra prendere tutti i record relativi ad un anno o ad un mese specifico, NON usare YEAR/MONTH ma impiegare un range in modo esplicito.

Quindi alla forma:

```
select * from Employee where YEAR(JoiningDate)=2019 and Month(JoiningDate)=3
```

preferire la forma:

```
select * from Employee where JoiningDate>='2019-03-01' and JoiningDate<'2019-04-01'
```

Evitare le trim

Rispetto alla programmazione BC, i campi sul Db NON hanno spazi al fondo, per cui non è necessario "trimmarli" prima del loro uso, specialmente nei confronti.

Evitare le CONVERT applicate al campo IN

In BC la clausola IN / NOT IN avviene applicando la funzione INLISTSTR (o una delle sue varianti) ad un campo.

Non è necessario e per le prestazioni è controproducente, convertire un campo in stringa prima di applicare la funzione INLISTSTR.

Quindi alla forma:

```
'@GETDBDATA TABELLA[DATDAARIG@PHD$+GR$+"DATDAARIG"] _
TIPOREC[DAR] _
IND[0 \IDD=IDD_DOC\TOP=200\NPRO=DCRNPRO\NPRR=0\] _
CAMPI[NPRR CPA QTA] _
SELEZIONI[DARIDD=IDD_DOC AND DARTOP=200 AND DARNPRO=DCRNPRO] _
SELEZIONIESESE[INLISTSTR(FNS$(DARNPRR),COD_SEL$)>0] _
ERRORE[#ENDFIL]
```

preferire la forma:

```
'@GETDBDATA TABELLA[DATDAARIG@PHD$+GR$+"DATDAARIG"] _
TIPOREC[DAR] _
IND[0 \IDD=IDD_DOC\TOP=200\NPRO=DCRNPRO\NPRR=0\] _
CAMPI[NPRR CPA QTA] _
SELEZIONI[DARIDD=IDD_DOC AND DARTOP=200 AND DARNPRO=DCRNPRO] _
SELEZIONIESESE[INLISTSTR(DARNPRR,COD_SEL$)>0] _
ERRORE[#ENDFIL]
```

Legacy

- ['@GETQUERYDATA](#)
- [Le query in BC](#)
- [Le subquery in BC](#)
- [Le common table expressions \(CTEs\) in BC](#)

'@GETQUERYDATA

- [Variabili di ambiente utilizzate dalla '@GETQUERYDATA](#)
- ['@GETQUERYDATA con INIZIOGRUPPO, LOOPGRUPPO e FINEGRUPPO](#)
- ['@GETQUERYDATA con più indici di ordinamento](#)

Variabili di ambiente utilizzate dalla '@GETQUERYDATA

Le seguenti variabili sono utilizzate per il funzionamento della specifica '@GETQUERYDATA:

ZDB_OPER_<IdQuery>:

E' una variabile intera, che indica l'operazione di lettura che deve essere eseguita.

Può assumere i seguenti valori:

Valore	Descrizione
0	Legge record successivo
1	Legge record precedente
2	Legge il primo record
3	Legge ultimo record

Per "Primo record" si intende il primo record della tabella, che rispetti l'ordinamento definito e che soddisfi le condizioni imposte dai filtri e da eventuali controlli di '@GETQUERYDATA.

Dopo il posizionamento, il record individuato viene letto e restituito al programma chiamante.

Per "Ultimo record" si intende l'ultimo record della tabella, che rispetti l'ordinamento definito e che soddisfi le condizioni imposte dai filtri e da eventuali controlli di '@GETQUERYDATA.

Dopo il posizionamento, il record individuato viene letto e restituito al programma chiamante.

ZDB_INDICE_<IdQuery>

E' una variabile intera, che definisce l'indice da usare per la lettura. Deve essere utilizzato nel caso in cui siano indicate più chiavi sulla specifica, per stabilire con quale ordinamento eseguire la lettura. Per la lettura con il primo indice definito la variabile associata dovrà avere valore 1, per il secondo indice 2, e così via.

ZDB_POSBLOCK_<IdQuery>

Dove <IdQuery> è il nome della variabile che contiene l'identificatore della query

E' una variabile di tipo Long, che contiene il position block di un record. In uscita, contiene sempre il position block del record che viene restituito dalla specifica. La variabile e' definita in modo automatico dal traduttore

ZDB_GRUPPOCORR_<IDQUERY>: output

Dove <IDQUERY> è il nome della variabile che contiene l'identificatore della query.

Questa variabile è creata quando è presente una delle callback INIZIOGRUPPO, FINEGRUPPO. E' una variabile di tipo intero, che contiene in ingresso delle callback il progressivo di identificazione del gruppo di record.

ZDB_OPTDB_<IDQUERY>

Dove <IDQUERY> è il nome della variabile che contiene l'identificatore della query.

E' una variabile di tipo intero, utilizzata per controllare alcune ottimizzazioni dell'ambiente durante le varie operazioni verso il database

Valore	Posizionamento eseguito
4	L'operazione non richiede posizionamento assoluto

Dalla versione 21.0, è possibile ottimizzare la '@GETQUERYDATA con gruppi di rottura, ponendo l'istruzione ZDB_OPTDB_IDQUERY=4 prima della '@GETQUERYDATA. In questo modo, viene memorizzato il valore di chiave di rottura ad inizio gruppo, così da valorizzare le variabili di programma a fine gruppo.

ZDB_OPTDB_IDQUERY=4

```
'@GETQUERYDATA IDQUERY[IDQUERY] _
    INIZIOGRUPPO[GOSUB INIZIO_GRUPPO]_
    FINEGRUPPO[GOSUB FINE_GRUPPO]_
    LOOPGRUPPO[GOSUB LOOP_GRUPPO]_
    CONTROLLI[GOSUB CONTROLLI]_
    ORDINAMENTO[NUM_ORD]
```

'@GETQUERYDATA con INIZIOGRUPPO,LOOPGRUPPO e FINEGRUPPO

Per chiarire l'ordine con il quale si succedono il richiamo delle routine associate ai agli attributi INIZIOGRUPPO, FINEGRUPPO, e LOOPGRUPPO, prendiamo una query di esempio sui clienti suddivisi per zona:

Provincia	Comune	Cliente
AT	Asti	Rossi
AT	Asti	Arancioni
AT	Valfenera	Verdi
CN	Cuneo	Bianchi
TO	Torino	Marroni
TO	Torino	Fucsia
TO	Rivoli	Azzurri

Sulla query sono definiti i gruppi:

- Provincia
- Provincia + comune

Gli eventi che sono generati dalla lettura della tabella precedente sono:

Sequenza eventi	Routine richiamata/ ZDB_GRUPPOCORR	Provincia	Comune	Cliente
1	INIZIOGRUPPO/0			
2	INIZIOGRUPPO/1	AT	Asti	Rossi
3	INIZIOGRUPPO/2			
4	LOOPGRUPPO			
5	LOOPGRUPPO	AT	Asti	Arancioni
6	FINEGRUPPO/2			
7	INIZIOGRUPPO/2	AT	Valfenera	Verdi
8	LOOPGRUPPO			
9	FINEGRUPPO/2			
10	FINEGRUPPO/1			
11	INIZIOGRUPPO/1	CN	Cuneo	Bianchi
12	INIZIOGRUPPO/2			
13	LOOPGRUPPO			
14	FINEGRUPPO/2			
15	FINEGRUPPO/1			

16	INIZIOGRUPPO/1	TO	Torino	Marroni
17	INIZIOGRUPPO/2			
18	LOOPGRUPPO			
19	LOOPGRUPPO	TO	Torino	Fucsia
20	FINEGRUPPO/2			
21	INIZIOGRUPPO/2	TO	Rivoli	Azzurri
22	LOOPGRUPPO			
23	FINEGRUPPO/2			
24	FINEGRUPPO/1			
25	FINEGRUPPO/0			

'@GETQUERYDATA con più indici di ordinamento

Quest'esempio mostra come utilizzare una '@GETQUERYDATA su una query avente più indici di ordinamento. La presenza di più indici di ordinamento per una query permette:

Se la query è visualizzata in una griglia, permette all'utente di selezionare l'ordinamento operando sulle intestazioni delle colonne

Permettere il posizionamento simulando una chiave

Per prima cosa è necessario definire una query avente più indici di ordinamento.

Esempio1 - Definizione di una query con più indici di ordinamento

BC
'@DEFQUERY IDQUERY[IDQUERY] '@DEFQUERYTABLE_LV IDQUERY[IDQUERY] TABELLA["AGPAREA"] '@DEFQUERYFIELD_LV IDQUERY[IDQUERY] TABELLA["AGPAREA"] CAMPO["COD"]_ ALIAS["CodArea"] VAR[CODAREA\$] '@DEFQUERYFIELD_LV IDQUERY[IDQUERY] TABELLA["AGPAREA"] CAMPO["DES"]_ ALIAS["DesArea"] VAR[DESAREA\$] '@DEFQUERYFIELD_LV IDQUERY[IDQUERY] TABELLA["AGPAREA"] CAMPO["NSERV"]_ ALIAS["NomeServer"] VAR[NOMESERVER\$] '@DEFQUERYCFIELD_LV IDQUERY[IDQUERY] _ ALIAS["cfield"] _ FMT[NUMERICO[4]] _ FORMULA[LEN(ALIAS["NomeServer"])] _ VAR[CFIELD] '@DEFQUERYORDER_LV IDQUERY[IDQUERY] IND[0]_ DESC["Ordinamento per codice area"]_ ALIAS["CodArea"] '@DEFQUERYORDER_LV IDQUERY[IDQUERY] IND[1]_ DESC["Ordinamento per descrizione area"]_ ALIAS["DesArea"] '@DEFQUERYORDER_LV IDQUERY[IDQUERY] IND[2]_ DESC["Ordinamento per NomeServer"]_ ALIAS["NomeServer"]_ '@DEFQUERYORDER_LV IDQUERY[IDQUERY] IND[3]_ DESC["Ordinamento per CFIELD"]_ ALIAS["Cfield"]

Esempio2 - Definizione della griglia in modo che abbia le colonne ordinabili:

BC
'@INSCOLGRID NOMEGRIGLIA[GRIG_QRY] TIT["Codice Area"] FMT[TESTO]_ LARG[10] ORDINABILE[0 SUGIU] '@INSCOLGRID NOMEGRIGLIA[GRIG_QRY] TIT["Descrizione Area"] FMT[TESTO] _ LARG[20] ORDINABILE[1 SUGIU] '@INSCOLGRID NOMEGRIGLIA[GRIG_QRY] TIT["Nome Server"] FMT[TESTO] _ LARG[20] ORDINABILE[2 SUGIU] '@INSCOLGRID NOMEGRIGLIA[GRIG_QRY] TIT["Lun"] FMT[TESTO] _ LARG[2] ORDINABILE[3 SUGIU]

La specifica '@GETQUERYDATA, associata alla griglia, definisce le diverse chiavi per gestire il posizionamento per i relativi indici di ordinamento.

Esempio3 - Definizione della griglia associata alla griglia

BC
LENPOS[2]=VAL(ORDPOS\$) '@GETQUERYDATA IDQUERY[IDQUERY] _ IND[0 \CodArea=ORDPOS\$\] _ IND[1 \DesArea=ORDPOS\$\] _ IND[2 \NomeServer=ORDPOS\$\] _ IND[3 \Cfield=LENPOS\$\] _ ERRORE[#ENDFIL]

♥ Nel caso di CamelCase attivo è necessario indicare gli ALIAS tra doppi apici:

BC
LENPOS[2]=VAL(ORDPOS\$) '@GETQUERYDATA IDQUERY[IDQUERY] _ IND[0 \"CodArea\"=ORDPOS\$\] _ IND[1 \"DesArea\"=ORDPOS\$\] _ IND[2 \"NomeServer\"=ORDPOS\$\] _ IND[3 \"Cfield\"=LENPOS\$\] _ ERRORE[#ENDFIL]

Le query in BC

Nel tempo, il Linguaggio BC si evolve aggiornandosi di nuovi costrutti sempre più simili ai linguaggi di programmazione esterni a Sistemi. Questo permette di essere sempre più flessibile nella progettazione di nuovi programmi.

Ne consegue che man mano si vadano a rendere obsolete vecchie logiche ormai non più supportate dai nuovi standard di programmazione. Questa scheda ha il compito di riassumere i vecchi costrutti utilizzati nei vecchi sorgenti.

Le query in BC - versione 1

Le specifiche di lettura sono interpretate dall'ambiente di esecuzione per accedere al DBMS Pervasive o SQL con le modalità adeguate, su SQL una lettura è sempre effettuata tramite una query e viene gestita in modo compatibile dal programma e dall'ambiente.

Per SQL è possibile costruire direttamente una query e il BC mette a disposizione diverse specifiche a tale scopo.

Definire una query in BC

Di seguito riportiamo l'esempio di una query molto semplice e vediamo la sua definizione in BC e ne confrontiamo subito la corrispondenza con il linguaggio SQL.

Esempio

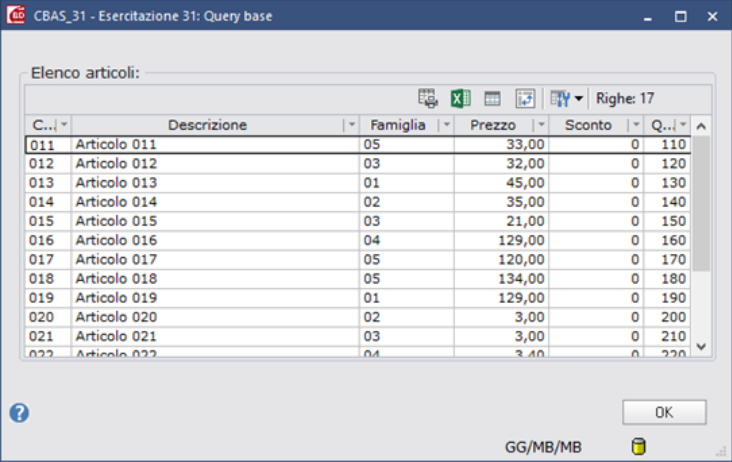
'=====
'@SR DEFINIZIONE_QUERY
'=====
'
' QUERY OTTENUTA ATTRAVERSO L'UTILIZZO DEL WIZARD DELLE QUERY
' CTRL+W SU MULTIEDIT E F7 DALLA VIDEATA CHE SI PRESENTA
'
' -----
' Data 5/11/2009 Ora 14.30 Stazione AG
' Descrizione: Articoli con quantità maggiore di 100
' -----
'
' Definizione iniziale della query


```

'
'@DEFQUERY IDQUERY[IDQUERY]
'
' Definizione della tabella di riferimento della query
' Tabella ARTICOLI
'
'@DEFQUERYFROM IDQUERY[IDQUERY] TABELLA[BLDART]
'
' Definizione dei campi della query
' Una sola '@DEFQUERYCOLUMN con tanti ESPR quanti sono i campi
'
'@DEFQUERYCOLUMN IDQUERY[IDQUERY] _
    ESPR[BLDART.COD AS [CodiceArticolo]] VAR[ARTCOD$] _ ' Codice articolo _
    ESPR[BLDART.DES AS [Descrizione]] VAR[ARTDES$] _ ' Descrizione _
    ESPR[BLDART.CODFAM AS [Famiglia]] VAR[ARTCODFAM$] _ ' Codice famiglia _
    ESPR[BLDART.PREZZO AS [PrezzoUnitario]] VAR[ARTPREZZO#] _ ' Prezzo di listino _
    ESPR[BLDART.SCONTO AS [ScontoArticolo]] VAR[ARTSCONTO] _ ' Sconto articolo _
    ESPR[BLDART.QUA AS [QuantitaMagazzino]] VAR[ARTQUA] _ ' Quantità in magazzino
'
' Filtri (WHERE)
' Applicazione del filtro: Selezionare tutti i dati DOVE la quantità è superiore a 100
'
'@DEFQUERYWHERE IDQUERY[IDQUERY] ESPR[BLDART.QUA>100 ]
'
' Ordinamento
' Definizione ordinamento ASCENDENTE per DESCRIZIONE
'
'@DEFQUERYORDERBY IND[1] ESPR[BLDART.DES/ASC]
'
RETURN

```

La sub SUB31_QUERYBASE.BC costruisce una query sulla tabella BLDART per cercare gli articoli con una quantità a magazzino maggiore o uguale a 100, la query è poi visualizzata in griglia:



The screenshot shows a window titled "CBAS_31 - Esercitazione 31: Query base". Inside, there's a section "Elenco articoli:" with a table. The table has columns: C... (likely Codice), Descrizione, Famiglia, Prezzo, Sconto, and Q... (likely Quantità). The table contains 17 rows of data. At the bottom right of the window is an "OK" button and a status bar showing "GG/MB/MB".

C...	Descrizione	Famiglia	Prezzo	Sconto	Q...
011	Articolo 011	05	33,00	0	110
012	Articolo 012	03	32,00	0	120
013	Articolo 013	01	45,00	0	130
014	Articolo 014	02	35,00	0	140
015	Articolo 015	03	21,00	0	150
016	Articolo 016	04	129,00	0	160
017	Articolo 017	05	120,00	0	170
018	Articolo 018	05	134,00	0	180
019	Articolo 019	01	129,00	0	190
020	Articolo 020	02	3,00	0	200
021	Articolo 021	03	3,00	0	210
022	Articolo 022	04	3,40	0	220

La definizione della query ('@DEFQUERY) richiede la tabella su cui deve essere effettuata ('@DEFQUERYFROM e/o '@DEFQUERYJOIN) e i campi che devono essere letti, che sono le colonne del record-set restituito ('@DEFQUERYCOLUMN):

```
SELECT CodArt AS [CodiceArticolo] ,
```

```

DesArt AS [Descrizione] ,
CodFamiglia AS [Famiglia] ,
Prezzo AS [PrezzoUnitario] ,
Sconto AS [ScontoArticolo] ,
MagQuantita AS [QuantitaMagazzino]
FROM BLDArticoli AS [BLDArticoli]
WHERE ( MagQuantita > 100 )
ORDER BY [Descrizione] ASC

```

dove sulla specifica ' *@DEFQUERYFROM* il parametro TABELLA[] (da solo) definisce la clausola FROM.

è prevista la condizione in lettura (' *@DEFQUERYWHERE*):

```

SELECT CodArt AS [CodiceArticolo] ,
DesArt AS [Descrizione] ,
CodFamiglia AS [Famiglia] ,
Prezzo AS [PrezzoUnitario] ,
Sconto AS [ScontoArticolo] ,
MagQuantita AS [QuantitaMagazzino]
FROM BLDArticoli AS [BLDArticoli]
WHERE (MagQuantita > 100 )
ORDER BY [Descrizione] ASC

```

ed è stato definito l'ordinamento (' *@DEFQUERYORDERBY*):

```

SELECT CodArt AS [CodiceArticolo] ,
DesArt AS [Descrizione] ,
CodFamiglia AS [Famiglia] ,
Prezzo AS [PrezzoUnitario] ,
Sconto AS [ScontoArticolo] ,
MagQuantita AS [QuantitaMagazzino]
FROM BLDArticoli AS [BLDArticoli]
WHERE (MagQuantita > 100 )
ORDER BY [Descrizione] ASC

```

eseguendo la query di cui sopra otteniamo il seguente record-set:

	CodiceArticolo	Descrizione	Famiglia	PrezzoUnitario	ScontoArticolo	QuantitaMagazzino
1	16	GOMMA COLORE	02	500.00	12.00	150
2	07	GOMMA STANDLER	04	500.00	25.00	150
3	06	PENNA BIC	01	100.00	15.00	200
4	15	PENNA BIC ROSSA	02	100.00	5.00	200

(La query così definita è puramente esemplificativa, in quanto può essere sostituita da una specifica ' *@GETDBDATA* che opera su una singola tabella)

Il wizard richiamabile dall'editor (combinazione tasti <Ctrl+W>) è un ottimo supporto alla costruzione di una query e semplifica di molto la scrittura del sorgente BC.

 Nel caso in cui la query sia definita su un sorgente diverso da quello principale, la *@DEFQUERY* va sempre fatta nel sorgente chiamante, mai in quello in cui è definita la query. Nel sorgente di definizione della query deve arrivare l'ID della query.

Corrispondenza tra BC e SQL

La seguente tabella mostra l'associazione tra le singole istruzioni BC e il comando SQL generato. Per maggior chiarezza, le specifiche sono state spezzate in più righe nella tabella ed il loro ordine potrebbe risultare modificato rispetto al vero sorgente BC.

BC	SQL
'@DEFQUERY IDQUERY[IDQUERY]	SELECT
'@DEFQUERYCOLUMN IDQUERY[IDQUERY] ESPR[BLDART.COD AS [CodiceArticolo] VAR[ARCODART\$]] ESPR[BLDART.DES AS [Descrizione] VAR[ARDESC\$]] ESPR[BLDART.CODFAM AS [Famiglia] VAR[ARCODFAM\$]] ESPR[BLDART.PREZZO AS [PrezzoUnitario] VAR[ARPRE#]] ESPR[BLDART.SCONTO AS [ScontoArticolo] VAR[ARSCO#]] ESPR[BLDART.QUA AS [QuantitaMagazzino] VAR[ARQUA]]	CodArt AS [CodiceArticolo] DesArt AS [Descrizione] CodFamiglia AS [Famiglia] Prezzo AS [PrezzoUnitario] Sconto AS [ScontoArticolo] MagQuantita AS [QuantitaMagazzino]
'@DEFQUERYFROM IDQUERY[IDQUERY] TABELLA[BLDART]	FROM BLDarticoli AS [BLDarticoli]
'@DEFQUERYWHERE IDQUERY[IDQUERY] ESPR[BLDART.QUA>100]	WHERE (MagQuantita > 100)
'@DEFQUERYORDERBY IDQUERY[IDQUERY] IND[1] ESPR[BLDART.DES/ASC]	ORDER BY [Descrizione] ASC

La clausola JOIN

È possibile indicare le tabelle in JOIN ('@DEFQUERYJOIN):

```
'=====
'@SR DEFINIZIONE_QUERY
'=====
'
' QUERY OTTENUTA ATTRAVERSO L'UTILIZZO DEL WIZARD DELLE QUERY
' CTRL+W SU MULTIEDIT E F7 DALLA VIDEATA CHE SI PRESENTA
'
' -----
' Data 5/11/2009 Ora 14.37 Stazione AG
' Descrizione:
' -----
'
IDQUERY[5]=0
'@DEFQUERY IDQUERY[IDQUERY]
'
' Definizione della tabella di riferimento della query
' Tabella BLDART
'
'@DEFQUERYFROM IDQUERY[IDQUERY] TABELLA[BLDART]
'
' Definizione della seconda tabella all'interno della query.
' Ogni tabella inserita deve essere in relazione con una tabella già presente
'
' Relazione tra BLDFAMIGLIA (BLDFamiglie) - Famiglie
' e BLDART (BLDarticoli) - Articoli
'
'@DEFQUERYJOIN IDQUERY[IDQUERY] TABELLA[BLDFAMIGLIA] TABELLAPADRE[BLDART] _
REL[BLDFAMIGLIA.COD=BLDART.CODFAM] OPERATOREJOIN["LEFT"]
'
' Definizione dei campi della query
' Una sola DEFQUERYCOLUMN con ESPR quanti sono i campi (anche se puntano a tabelle differenti)
'
'@DEFQUERYCOLUMN IDQUERY[IDQUERY] _
ESPR[BLDART.COD AS [CodiceArticolo]] VAR[ARTCOD$] _ ' Cod.articolo _
ESPR[BLDART.DES AS [Descrizione]] VAR[ARTDES$] _ ' Descrizione _
```

```
ESPR[BLDART.CODFAM] AS [Famiglia] VAR[ARTCODFAM$] _ ' Cod.famiglia _

ESPR[BLDART.PREZZO] AS [PrezzoUnitario] VAR[ARTPREZZO#] _ ' Prezzo unit. _

ESPR[BLDART.SCONTO] AS [ScontoArticolo] VAR[ARTSCONTO] _ ' Sconto art._

ESPR[BLDART.QUA] AS [QuantitaMagazzino] VAR[ARTQUA] _ ' Quantità mag._

ESPR[BLDFAMIGLIA.CODFAM] AS [CodiceFamiglia] VAR[FAMCOD$] _ ' Codice fam. _

ESPR[BLDFAMIGLIA.DESC] AS [DesFamiglia] VAR[FAMDES$] _ ' Descrizione

,

' Ordinamento

' Definizione ordinamento ASCENDENTE per DESCRIZIONE

,

'@DEFQUERYORDERBY IDQUERY[IDQUERY] IND[1] ESPR[BLDART.DES/ASC]

,

RETURN
```

dove sulla specifica `'@DEFQUERYJOIN` il parametro `TABELLA[]` definisce la tabella da mettere in join con la tabella principale indicata nel parametro `TABELLAPADRE[]`.

e i campi di tutte le tabelle per il record-set (`'@DEFQUERYCOLUMN`):

```
SELECT BLDArticoli.CodArt AS [CodiceArticolo]
      BLDArticoli.DesArt AS [Descrizione]
      BLDArticoli.CodFamiglia AS [Famiglia]
      BLDArticoli.Prezzo AS [PrezzoUnitario]
      BLDArticoli.Sconto AS [ScontoArticolo]
      BLDArticoli.MagQuantita AS [QuantitaMagazzino]

      BLDFamiglie.CodFamiglia AS [CodiceFamiglia]
      BLDFamiglie.DesFamiglia AS [DesFamiglia]

FROM BLDArticoli AS [BLDArticoli]

LEFT OUTER JOIN BLDFamiglie AS [BLDFamiglie] ON (
      BLDFamiglie.CodFamiglia = BLDArticoli.CodFamiglia

      AND (BLDFamiglie.DBGruppo = BLDArticoli.DBGruppo)

)

ORDER BY [Descrizione] ASC
```

(esempio riportato dalla sub: SUB32_JOIN.BC).

Corrispondenza tra BC e SQL

La seguente tabella mostra l'associazione tra le singole istruzioni BC e il comando SQL generato. Per maggior chiarezza, le specifiche sono state spezzate in più righe nella tabella ed il loro ordine potrebbe risultare modificato rispetto al vero sorgente BC.

BC	SQL
...	...
'@DEFQUERYJOIN IDQUERY[IDQUERY] TABELLA[BLDFAMIGLIA] TABELLAPADRE[BLDART] REL[BLDFAMIGLIA.COD=BLDART.CODFAM] OPERATOREJOIN["LEFT"]	LEFT OUTER JOIN BLDFamiglie AS [BLDFamiglie] ON (BLDFamiglie.CodFamiglia = BLDArticoli.CodFam)
...	...

Campi calcolati in query e clausola GROUPBY

È possibile inserire nel record-set un campo calcolato (`'@DEFQUERYCOMPUTEDCOLUMN`).

Esempio - SUB33_QUERY_CCALC.BC:

```
,

' Definizione di un campo calcolato che deve corrispondere a:

' se lo sconto = 0 allora il campo calcolato è uguale al campo prezzo
```

```
' altrimenti il campo calcolato è uguale al campo prezzo - la percentuale di sconto
'
PREZZODEFINITIVO#[8,2]=0
'
'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDQUERY]_
    ESPR[IF BLDART.SCONTO=0 THEN BLDART.PREZZO _
        ELSE BLDART.PREZZO-((BLDART.PREZZO*BLDART.SCONTO)/100) AS
        [PrezzoDefinitivo] VAR[PREZZODEFINITIVO#]]
```

o introdurre la clausola GROUPBY (' @DEFQUERYCOLUMN), SUB_034.BC:

```
'
' Definizione dei campi della query
' Una sola DEFQUERYCOLUMN con tanti ESPR quanti sono i campi
' Nel parametro ESPR viene definita una funzione di RAGGRUPPAMENTO (GROUPBY/SUM).
' Se definite TUTTI i campi devono avere una funzione associata.
'
'@DEFQUERYCOLUMN IDQUERY[IDQUERY]_
    ESPR[GROUPBY(BLDFAMIGLIA.COD) AS [CodiceFamiglia]] VAR[FAMCOD$] _ ' Codice fam._
    ESPR[GROUPBY(BLDFAMIGLIA.DES) AS [Descrizione]] VAR[FAMDES$] _ ' Descrizione _
    ESPR[SUM(BLDART.PREZZO) AS [PrezzoUnitario]] VAR[ARTPREZZO#] ' Prezzo unit.
```

sintassi SQL:

```
SELECT BLDFamiglie.CodFamiglia AS [CodiceFamiglia],
    BLDFamiglie.DesFamiglia AS [Descrizione],
    SUM(BLDArticoli.Prezzo) AS [PrezzoUnitario]

FROM BLDFamiglie AS [BLDFamiglie]
LEFT OUTER JOIN BLDArticoli AS [BLDArticoli] ON (
    BLDArticoli.CodFamiglia = BLDFamiglie.CodFamiglia
    AND (BLDArticoli.DBGruppo = BLDFamiglie.DBGruppo)
)

GROUP BY BLDFamiglie.CodFamiglia
,BLDFamiglie.DesFamiglia
```

-

Definizione delle tabelle e delle relazioni

La definizione delle relazioni tra tabelle è necessaria quando la query ha necessità di operare su più tabelle e non solo su una.

La definizione di una relazione avviene tramite l'utilizzo della specifica ' @DEFQUERYJOIN su cui va indicato:

- le tabelle in relazione: il parametro TABELLA serve per indicare la tabella da relazionare e TABELLAPADRE la tabella su cui avviene la relazione;
- la relazione tramite i campi chiave tramite il parametro REL;
- la tipologia della relazione tramite il campo OPERATOREJOIN.



Per l'elenco delle tipologie di join supportate consultare la scheda OperatoreJoin del Manuale di riferimento del BC.

Il codice BC

Nell'esempio della query che vogliamo costruire, si vuole aggiungere il codice e la descrizione della famiglia articoli contenuti nella tabella BLDFAMIGLIA. Inseriamo quindi la tabella BLDFAMIGLIA in JOIN con la tabella BLDART.

```
BC
```

```
...
'@DEFQUERYJON IDQUERY[IDQUERY] TABELLA[BLDFAMIGLIA] TABELLAPADRE[BLDART] _
      REL[BLDFAMIGLIA.COD=BLDART.CODFAM] OPERATOREJOIN["LEFT"]
...
```

Esempio di lettura del recordset

Per leggere più record da una tabella è necessario utilizzare la specifica '@GETQUERYDATA.

L'ambiente di esecuzione esegue un preciso flusso operativo per la lettura dei record, ovvero:

- effettua una chiamata al DBMS che restituisce il risultato sotto forma di record-set;
- preleva il record-set e lo memorizza in una struttura tabellare in memoria;
- procede con una lettura sequenziale della struttura.
- Richiama per ogni record la callback specificata dal parametro AZIONE

```
BC
...
'@GETQUERYDATA IDQUERY[IdQuery] _
      OGGETTO[OggettoEditoriale] _
      AZIONE[GES[GOSUB elabora_recordset]]
...

'=====
'@SR elabora_recordset
'=====

...

Return
...
```

Definizione dei campi (colonne del record-set)

La definizione di una query deve prevedere la definizione di almeno un campo, non è pertanto possibile, in BC, formalizzare tramite una query una sintassi SQL del tipo: *SELECT * FROM*, nel codice BC devono essere definiti i campi delle tabelle da interrogare con la specifica '@DEFQUERYCOLUMN.

Si noti che nel parametro ESPR c'è una sintassi del tipo *TABELLA.CAMPO [Alias]*.

Da ora in poi, nelle specifiche di costruzione della query, sarà dunque equivalente indicare *TABELLA.CAMPO* oppure *[Alias]* per riferirsi a questo campo.

```
BC
```

...

'@DEFQUERYCOLUMN IDQUERY[IDQUERY] _

ESPR[BLDART.COD AS [CodiceArticolo]] VAR[ARTCOD\$] _ ' Cod.articolo _

ESPR[BLDART.DES AS [Descrizione]] VAR[ARTDES\$] _ ' Descrizione _

ESPR[BLDART.CODFAM AS [Famiglia]] VAR[ARTCODFAM\$] _ ' Cod.famiglia _

ESPR[BLDART.PREZZO AS [PrezzoUnitario]] VAR[ARTPREZZO#] _ ' Prezzo unit. _

ESPR[BLDART.SCONTO AS [ScontoArticolo]] VAR[ARTSCONTO] _ ' Sconto art._

ESPR[BLDART.QUA AS [QuantitaMagazzino]] VAR[ARTQUA] _ ' Quantità mag._

ESPR[BLDFAMIGLIA.CODFAM AS [CodiceFamiglia]] VAR[FAMCOD\$] _ ' Codice fam. _

ESPR[BLDFAMIGLIA.DESC AS [DesFamiglia]] VAR[FAMDES\$] _ ' Descrizione

...

SQL

...

SELECT BLDArticoli.CodArt AS [CodiceArticolo]

BLDArticoli.DesArt AS [Descrizione]

BLDArticoli.CodFamiglia AS [Famiglia]

BLDArticoli.Prezzo AS [PrezzoUnitario]

BLDArticoli.Sconto AS [ScontoArticolo]

BLDArticoli.MagQuantita AS [QuantitaMagazzino]

BLDFamiglie.CodFamiglia AS [CodiceFamiglia]

BLDFamiglie.DesFamiglia AS [DesFamiglia]

...

Definizione dei filtri (condizioni)

La definizione di una query normalmente prevede la definizione di almeno un filtro in lettura. Se da un lato i campi operano un filtro sulle colonne delle tabelle da leggere, le condizioni di lettura operano un filtro sulle righe. Quanto migliori sono i filtri di lettura, tanto più preciso e limitato in grandezza sarà il risultato della query (record-set) e, di conseguenza, tanto più veloce e performante sarà il programma scritto. Nel codice BC i filtri in lettura (clausola WHERE) sono definiti tramite la specifica ' @DEFQUERYWHERE.

BC	SQL
...	...
'@DEFQUERYWHERE IDQUERY[IDQUERY] ESPR[BLDART.QUA>100]	WHERE (BLDArticoli.MagQuantita > 100)
...	...

Tabelle di gruppo

Sulle tabelle di gruppo il gruppo è fisicamente la prima colonna della tabella:

	DBGruppo	BCQVerAmbID	BCQVerAmbDesc	BCQVerAmbPath	BCQVerAr
1	B1	_18.10	Versione 18.10	\\NTSIST\F\AMBIENTE\w_1810\	0
2	B1	_18.10a	Versione 18.10a	\\NTSIST\F\AMBIENTE\w_1810a\	0
3	B1	_18.11	Versione 18.11	\\NTSIST\F\AMBIENTE\w_1811\	0
4	B1	_18.11a	Versione 18.11a	\\NTSIST\F\AMBIENTE\w_1811a\	0
5	B1	_18.12	Versione 18.12	\\NTSIST\F\AMBIENTE\w_1812\	0
6	B1	_18.12a	Versione 18.12a	\\NTSIST\F\AMBIENTE\w_1812a\	0
7	B1	_18.12b	Versione 18.12b	\\NTSIST\F\AMBIENTE\w_1812b\	0
8	B1	_18.13	Versione 18.13	\\NTSIST\F\AMBIENTE\w_1813\	0
9	B1	_18.14	Versione 18.14	\\NTSIST\F\AMBIENTE\w_1814\	0
10	B1	_18.14a	Versione 18.14a	\\NTSIST\F\AMBIENTE\w_1814a\	0
11	B1	_18.15	Versione 18.15	\\NTSIST\F\AMBIENTE\w_1815\	0
12	B1	_18.15a	Versione 18.15a	\\NTSIST\F\AMBIENTE\w_1815a\	0
13	B1	_18.16	Versione 18.16	\\NTSIST\F\AMBIENTE\w_1816\	0
14	B1	_18.16a	Versione 18.16a	\\NTSIST\F\AMBIENTE\w_1816a\	0

è, quindi, una informazione fisicamente in tabella gestita in modo automatico dall'ambiente. Normalmente, in tutte le operazioni fatte sulla tabella, l'ambiente aggiunge in automatico la clausola WHERE come di seguito riportato:

SQL
...
WHERE (<tabella>.DBGruppo='XX')
...

Per poter fare in modo che il programma gestisca in modo autonomo l'eventuale filtro in lettura sul gruppo è necessario definire la query MULTIGRUPPO.

Definizione degli ordinamenti

Un aspetto importante nella definizione di una query è l'indicazione dell'ordinamento o degli ordinamenti possibili coi quali esporre o consultare i dati.
Un indice di ordinamento si definisce come elenco di campi, ognuno avente la proprietà di ordinamento ascendente (dal più piccolo al più grande) o discendente (dal più grande al più piccolo). Rispetto alla normale definizione degli indici delle tabelle, gli indici della query hanno il vantaggio di poter definire sia campi con proprietà "ascendente" che campi con proprietà "discendente". Rimane ovviamente fondamentale l'ordine con il quale si indicano i campi.
Nel codice BC l'ordinamento è definito tramite la specifica '@DEFQUERYORDERBY.

BC	SQL
...	...
@DEFQUERYORDERBY IDQUERY[IDQUERY] IND[1] ESPR[BLDART.DES/ASC]	ORDER BY [Descrizione] ASC
...	...

-

Definizione dei raggruppamenti

Tra le funzionalità più importanti di una query c'è il raggruppamento. Un raggruppamento su un campo corrisponde ad una funzione di aggregazione da applicare al campo stesso al termine della lettura del record-set.
Se si indica un raggruppamento su un campo, è obbligatorio indicare un raggruppamento per ognuno di tutti gli altri campi della query.

Le tipologie di raggruppamento consentite sul singolo campo sono:

- GROUPBY - raggruppa per ...
- COUNT - conta i record estratti dalla query
- COUNT DISTINCT - conteggio solamente dei record non doppi
- MIN - Valore minimo
- MAX - Valore massimo.

Per i campi numerici si aggiungono le funzioni di aggregazione:

- SUM - Sommatoria
- SUM DISTINCT - Calcola la somma di tutti i valori del raggruppamento escludendo le righe con valori del campo specificato ripetuti
- AVG - Valore medio.
- AVG DISTINCT- Calcola la media di tutti i valori del raggruppamento escludendo le righe con valori del campo specificato ripetuti.

La definizione del raggruppamento sul campo è fatta tramite la specifica '@DEFQUERYCOLUMN.

BC
...
'@DEFQUERYCOLUMN IDQUERY[IDQUERY] _
ESPR[GROUPBY(BLDFAMIGLIA.COD) AS [CodiceFamiglia]] VAR[FAMCOD\$] _ ' Codice fam._
ESPR[GROUPBY(BLDFAMIGLIA.DES) AS [Descrizione]] VAR[FAMDES\$] _ ' Descrizione _
ESPR[SUM(BLDART.PREZZO) AS [PrezzoUnitario]] VAR[ARTPREZZO#] ' Prezzo unit.
...

SQL
...
SUM(BLDArticoli.Prezzo) AS [PrezzoUnitario]
GROUP BY BLDFamiglie.CodFamiglia,
BLDFamiglie.DesFamiglia
...

E' stato definito un raggruppamento SUM sul campo 'PrezzoUnitario' pertanto è stato inserito un raggruppamento GROUPBY sugli altri campi.

Definizione campi calcolati e filtri su campi calcolati

Non sempre, all'interno della base dati, si hanno a disposizione tutte le informazioni già formattate secondo le esigenze del programma. Il caso più semplice in questo senso è la Ragione Sociale dell'anagrafica generale: essa è divisa in due campi (RagioneSociale1 e RagioneSociale2) e, se si vuole riportare come un'unica colonna, si è obbligati a passare tramite un campo calcolato come somma dei due campi più uno spazio in mezzo. Oppure, di cui l'esempio sotto riferito, abbiamo l'esigenza di definire un campo calcolato che deve corrispondere al prezzo di un articolo scontato se la percentuale dello sconto da applicare è diversa da zero.

- Esempio:
- se la percentuale dello sconto è uguale 0 devo visualizzare il campo prezzo;
 - se la percentuale dello sconto è diversa da 0 devo visualizzare il "prezzo-{%sconto}"

La definizione di un campo calcolato è fatta tramite la specifica '@DEFQUERYCOMPUTEDCOLUMN'.

BC
...
'
' Definizione di un campo calcolato che deve corrispondere a:
' se lo sconto = 0 allora il campo calcolato è uguale al campo prezzo
' altrimenti il campo calcolato è uguale al campo prezzo - la percentuale di sconto
'
PREZZODEFINITIVO#[8,2]=0
'
'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDQUERY] _
ESPR[IF BLDART.SCONTO=0 THEN BLDART.PREZZO _
ELSE BLDART.PREZZO-((BLDART.PREZZO*BLDART.SCONTO)/100) AS [PrezzoDefinitivo] _
VAR[PREZZODEFINITIVO#]]
...

SQL
...
CASE
WHEN BLDArticoli.Sconto = 0
THEN BLDArticoli.Prezzo
ELSE BLDArticoli.Prezzo - ((BLDArticoli.Prezzo * BLDArticoli.Sconto) / 100)
END AS [PrezzoDefinitivo]
...

Definizione dei filtri (condizioni) su raggruppamenti

La definizione di una clausola *WHERE* (filtro) nel linguaggio SQL vieta l'utilizzo di formule di aggregazione al proprio interno. Questo tipo di filtri devono essere indicati nella clausola *HAVING*. Il linguaggio BC prevede una sola specifica di filtro per gestire entrambe le casistiche; la differenziazione avviene con l'uso di parametri diversi:

- '@DEFQUERYWHERE ESPR[...] per definire una condizione di filtro
- '@DEFQUERYWHERE ESPRHAVING[...] per definire una condizione di filtro su campo aggregato.

Volendo visualizzare solo gli articoli che hanno un prezzo definitivo non inferiore ai 50€ devo inserire un filtro sul campo calcolato [PrezzoDefinitivo] nel seguente modo:

```
BC
...
'
SOGLIA_MINIMA_PREZZO#[15,2]=50
'
'@DEFQUERYWHERE IDQUERY[IDQUERY] ESPRHAVING[SUM([PrezzoDefinitivo])>=SOGLIA_MINIMA_PREZZO]
'
...
```

```
SQL
...
'
HAVING (
    SUM((
        CASE
            WHEN BLDArticoli.Sconto = 0
            THEN BLDArticoli.Prezzo
            ELSE BLDArticoli.Prezzo - ((BLDArticoli.Prezzo * BLDArticoli.Sconto) / 100)
        END
    )) >= 50
)
'
...
```

Precisazioni sulla conversione in SQL dei filtri su raggruppamenti

Il BC ha un grande vantaggio rispetto al linguaggio SQL: permette di indicare l'alias nell'espressione di filtro (con notevole guadagno in sintesi), mentre SQL obbliga a replicare l'intera formula di raggruppamento già espressa nella sintassi di SELECT.

Per definire una condizione sui filtri di raggruppamento è possibile impiegare due specifiche:

'@DEFQUERYWHERE (forma semplificata) e '@DEFQUERYCONGRID.
e con la specifica '@DEFQUERYWHERE si usa l'attributo ESPRHAVING.

Come sopra riportato dall'esempio è possibile notare come l'ambiente opera alcuni automatismi sulla traduzione dei filtri indicati in istruzioni SQL al fine di garantire la corretta sintassi; sono quindi inseriti in automatico per i campi presenti nel filtro su raggruppamenti le funzioni di raggruppamento come richiesto dal linguaggio SQL.

Questa conversione avviene secondo la seguente tabella:

Funzione di raggruppamento del campo	Operazione compiuta dall'ambiente
GROUP BY	Nessuna operazione di adeguamento
SUM	Si aggiunge al campo la funzione SUM
MAX	Si aggiunge al campo la funzione MAX
MIN	Si aggiunge al campo la funzione MIN
AVG	Si aggiunge al campo la funzione AVG

La conversione non avviene però in maniera uniforme, ma secondo i seguenti criteri:

1. Se il campo è indicato mediante il suo ALIAS, allora al campo è applicata sempre la conversione descritta dalla tabella precedente;
2. Se il campo è indicato mediante la notazione Descrittore.Nomebreve, allora la conversione è applicata solo se il campo è stato indicato all'interno dell'attributo FORMULA1 o FORMULA2 della '@DEFQUERYWHEREGRID.

Le subquery in BC

Nel tempo, il Linguaggio BC si evolve aggiornandosi di nuovi costrutti sempre più simili ai linguaggi di programmazione esterni a Sistemi. Questo permette di essere sempre più flessibile nella progettazione di nuovi programmi.

Ne consegue che man mano si vadano a rendere obsolete vecchie logiche ormai non più supportate dai nuovi standard di programmazione. Questa scheda ha il compito di riassumere i vecchi costrutti utilizzati nei vecchi sorgenti.

Le subquery (o sottoquery) in BC - versione 1

Cos'è una subquery (o sottoquery)

Le subquery o sottoquery consentono di nidificare una query all'interno di un'altra query per costruire routine complesse, nonché per recuperare insiemi di dati che sarebbe impossibile costruire senza ricorrere ad una elaborazione che proceda a passaggi successivi con risultati intermedi e facendo uso di file e/o strutture temporanee.

Normalmente è possibile definire delle query con due tipi di subquery:

- subquery correlate, una subquery correlata dipende e utilizza riferimenti della query esterna;
- subquery non correlate, una subquery non correlata è indipendente dalla query esterna all'interno della quale è contenuta.

Entrambi i tipi di subquery possono restituire un unico valore (scalare) per poter essere posizionate ovunque all'interno di un'istruzione SELECT dove ci si aspetta uno o nessun valore.

Oppure possono restituire più di un valore e in questo caso devono essere posizionate ovunque all'interno di un'istruzione SELECT dove si prevede un insieme di valori.

Le subquery o sottoquery sono a tutti gli effetti delle SELECT utilizzate all'interno di un'altra SELECT e nella sintassi del linguaggio SQL devono essere espresse sempre all'interno di parentesi tonde in modo da differenziarsi dalla query nativa e di essere utilizzate come parametro da passare alle clausole; inoltre è possibile nidificarle anche a profondità notevoli.

Normalmente ogni SELECT restituisce logicamente una tabella formata da righe e colonne, nel caso delle subquery è necessario fare una distinzione in quanto possono restituire:

1. un valore singolo (**Scalar subqueries**);
2. una singola riga (**Predicate subqueries**);
3. una singola colonna oppure una normale tabella (**Table subqueries**).

Le diverse tipologie di subquery possono trovare posto in diversi punti dell'istruzione che le contiene.

Scalar subqueries

Sono query indipendenti in grado di restituire un unico valore (detto appunto valore scalare) e possono essere utilizzate in associazione ad un valore scalare, la si può usare in qualsiasi punto sia possibile utilizzare un valore di colonna.

Si tratta della tipologia di subquery più semplice e il suo uso più frequente può essere come operatore di confronto.

Utilizzando una Scalar subquery è inutile tentare di estrarre da una tabella più valori perché in questo caso i record coinvolti sarebbero più di uno e una Scalar subquery non è in grado di accettare un risultato del genere e il risultato sarà una notifica di errore: "*Subquery returned more than 1 value*" oppure "*Operand should contain 1 column(s)*".

Predicate subqueries

Svolgono la funzione di estendere i costrutti logici basati sull'utilizzo della clausola WHERE, vengono usate a volte con le clausole EXISTS e NOT EXISTS; la clausola EXISTS è vera quando la subquery restituisce almeno una riga, mentre è falsa nel caso opposto. Ovviamente NOT EXISTS funziona al contrario.

Table subqueries

Con questa categoria si identificano interrogazioni annidate nella clausola FROM.

Vantaggi e svantaggi di una subquery

Le subquery vengono spesso utilizzate per ottimizzare e rendere più semplici le interrogazioni verso il DBMS, inoltre sono vantaggiose perché permettono di strutturare la query per isolare ogni parte della sua definizione.

Possono essere un'ottima alternativa alla realizzazione di join che per eseguire la stessa operazione potrebbero essere piuttosto complesse e meno facili da leggere.

Il motore del database è in grado di ottimizzare l'esecuzione delle subquery trasformandole in join (se sono soddisfatti determinati criteri) ottenendo un rendimento migliore in termini di prestazioni.

Le subquery correlate sono rieseguite ad ogni riga estratta dalla query esterna e, di conseguenza, non sono molto performanti possono quindi richiedere più tempo per l'esecuzione rispetto ad una join a causa di come le elabora l'utilità di ottimizzazione del database.

Meglio evitarle quando possibile, spesso infatti una subquery correlata è trasformabile in una join ma ovviamente esistono scenari che esigono l'uso delle subquery invece delle join.

Altro svantaggio è che non è possibile modificare una tabella e selezionare dati dalla stessa tabella all'interno di una subquery nella stessa istruzione SQL .

Definizione delle subquery (o sottoquery) in BC

La definizione di una subquery non prevede differenze nella sua definizione rispetto ad una qualsiasi altra query, è ovviamente importante la sua definizione all'interno della query che la contiene.

Negli esempi di seguito riportati si fa riferimento alle seguenti tre tabelle:

BLDSVIL_Categorie: Tabella categorie articoli

	CategID	CategDescrizione	CategNote	CategFLDisponibile
1	10	Cancelleria	Annotazioni varie categoria 'Cancelleria'	1
2	20	Ferramenta	Annotazioni varie categoria 'Ferramenta'	1
3	30	Pulizia	Annotazioni varie categoria 'Pulizia'	1
4	40	Mobili ufficio	Annotazioni varie categoria 'Mobili ufficio'	1
5	50	Elettricità e Luce	Annotazioni varie categoria 'Elettricità/Lu...	1

BLDSVIL_Famiglie: Tabella famiglie articoli

	FamCatID	FamID	FamDescrizione	FamNote	FamFLDisponibile
1	10	10	Penne e pennarelli	Annotazioni varie famiglia 'Penne e pennarelli'	1
2	10	20	Matite	Annotazioni varie famiglia 'Matite'	1
3	10	30	Utensili	Annotazioni varie famiglia 'Utensili'	1
4	10	40	Ricambi	Annotazioni varie famiglia 'Ricambi'	1
5	10	50	Altro	Annotazioni varie famiglia 'Altro'	1
6	20	60	Viti	Annotazioni varie famiglia 'Viti'	1
7	20	70	Tasselli	Annotazioni varie famiglia 'Tasselli'	1
8	20	80	Chiavi	Annotazioni varie famiglia 'Chiavi'	1
9	20	90	Utensili	Annotazioni varie famiglia 'Utensili'	1
10	20	100	Altro	Annotazioni varie famiglia 'Altro'	1
11	30	110	Detersivi	Annotazioni varie famiglia 'Detersivi'	1
12	30	120	Stoffa e carta	Annotazioni varie famiglia 'Stoffa e carta'	1
13	30	130	Utensili	Annotazioni varie famiglia 'Utensili'	1
14	30	140	Ricambi	Annotazioni varie famiglia 'Ricambi'	1
15	30	150	Altro	Annotazioni varie famiglia 'Altro'	1
16	40	160	Scrivanie	Annotazioni varie famiglia 'Scrivanie'	1
17	40	170	Sedie	Annotazioni varie famiglia 'Sedie'	1
18	40	180	Complementi di arredo	Annotazioni varie famiglia 'Complementi di arredo'	1
19	40	190	Altro	Annotazioni varie famiglia 'Altro'	1
20	50	200	Lampade	Annotazioni varie famiglia 'Lampade'	1
21	50	210	Cavi e prolunghe	Annotazioni varie famiglia 'Cavi e prolunghe'	1
22	50	220	Ricambi	Annotazioni varie famiglia 'Ricambi'	1
23	50	230	Altro	Annotazioni varie famiglia 'Altro'	1

BLDSVIL_Articoli: Tabella articoli

	ArtCatID	ArtFamID	ArtID	ArtDescrizione	ArtNote	ArtFLDisponibile	ArtUnitaConf	ArtQuantitaConf	ArtQuantitaUnita	ArtCostoUnit
1	10	10	1	Penna a sfera colore nero	Annotazioni varie famiglia 'Penna a sfera colore ...	1	15	5.00	75	1.00
2	10	10	2	Penna a sfera colore blu	Annotazioni varie famiglia 'Penna a sfera colore ...	1	15	5.00	75	1.00
3	10	10	3	Penna a sfera colore rosso	Annotazioni varie famiglia 'Penna a sfera colore r...	1	15	5.00	75	1.00
4	10	20	4	Potamine 0.5mm	Annotazioni varie famiglia 'Potamine 0.5mm'	1	10	5.00	50	1.00
5	10	20	5	Potamine 1.2mm	Annotazioni varie famiglia 'Potamine 1.2mm'	1	10	5.00	50	1.00
6	10	20	6	Potamine 1.4mm	Annotazioni varie famiglia 'Potamine 1.4mm'	1	10	5.00	50	1.00
7	10	30	7	Forbici	Annotazioni varie famiglia 'Forbici'	1	1	10.00	10	2.00
8	10	30	8	Cutter	Annotazioni varie famiglia 'Cutter'	1	1	10.00	10	2.00
9	10	30	9	Righello	Annotazioni varie famiglia 'Righello'	1	1	10.00	10	2.00
10	10	40	10	Rotolo scotch	Annotazioni varie famiglia 'Rotolo scotch'	1	5	10.00	50	0.00
11	10	40	11	Colla Pitt	Annotazioni varie famiglia 'Colla Pitt'	1	5	10.00	50	0.00
12	10	40	12	Bianchetto	Annotazioni varie famiglia 'Bianchetto'	1	5	10.00	50	0.00
13	10	50	13	Post-it segnalibro	Annotazioni varie famiglia 'Post-it segnalibro'	1	4	20.00	80	0.00
14	10	50	14	Post-it quadretti grigi	Annotazioni varie famiglia 'Post-it quadretti grigi'	1	4	20.00	80	0.00
15	10	50	15	Post-it quadretti rosa	Annotazioni varie famiglia 'Post-it quadretti rosa'	1	4	20.00	80	0.00
16	20	60	16	Vite legno 30mm diametro 4mm	Annotazioni varie famiglia 'Vite legno 30mm diam...	1	50	10.00	500	0.00
17	20	60	17	Vite legno 35mm diametro 5mm	Annotazioni varie famiglia 'Vite legno 35mm diam...	1	50	10.00	500	0.00
18	20	60	18	Vite legno 40mm diametro 6mm	Annotazioni varie famiglia 'Vite legno 40mm diam...	1	50	10.00	500	0.00
19	20	70	19	Tassello nylon 40mm diametro 6mm	Annotazioni varie famiglia 'Tassello nylon 40mm ...	1	50	10.00	500	0.00
20	20	70	20	Tassello nylon 50mm diametro 8mm	Annotazioni varie famiglia 'Tassello nylon 50mm ...	1	50	10.00	500	0.00
21	20	70	21	Tassello nylon 60mm diametro 10mm	Annotazioni varie famiglia 'Tassello nylon 60mm ...	1	50	10.00	500	0.00
22	20	80	22	Brugola 6mm	Annotazioni varie famiglia 'Brugola 6mm'	1	6	5.00	30	1.00
23	20	80	23	Brugola 8mm	Annotazioni varie famiglia 'Brugola 8mm'	1	6	5.00	30	1.00
24	20	80	24	Brugola 10mm	Annotazioni varie famiglia 'Brugola 10mm'	1	6	5.00	30	1.00
25	20	90	25	Cacciavite taglio	Annotazioni varie famiglia 'Cacciavite taglio'	1	3	5.00	15	3.00
26	20	90	26	Cacciavite stella	Annotazioni varie famiglia 'Cacciavite stella'	1	3	5.00	15	3.00
27	20	90	27	Cacciavite stella mini	Annotazioni varie famiglia 'Cacciavite stella mini'	1	3	5.00	15	3.00
28	20	100	28	Rondelle 8mm diametro interno 4mm	Annotazioni varie famiglia 'Rondelle 8mm diametr...	1	30	5.00	150	0.00
29	20	100	29	Rondelle 10mm diametro interno 5...	Annotazioni varie famiglia 'Rondelle 10mm diam...	1	30	5.00	150	0.00
30	20	100	30	Rondelle 12mm diametro interno 6...	Annotazioni varie famiglia 'Rondelle 12mm diam...	1	30	5.00	150	0.00
31	30	110	31	Detergente liquido	Annotazioni varie famiglia 'Detergente liquido'	1	2	10.00	20	1.00
32	30	110	32	Detergente polvere	Annotazioni varie famiglia 'Detergente polvere'	1	2	10.00	20	1.00
33	30	110	33	Detergente gel	Annotazioni varie famiglia 'Detergente gel'	1	2	10.00	20	1.00
34	30	120	34	Rotolo panno carta	Annotazioni varie famiglia 'Rotolo panno carta'	1	2	20.00	40	0.00
35	30	120	35	Rotolo carta riciclata	Annotazioni varie famiglia 'Rotolo carta riciclata'	1	2	20.00	40	0.00
36	30	120	36	Rotolo carta scottex	Annotazioni varie famiglia 'Rotolo carta scottex'	1	2	20.00	40	0.00
37	30	130	37	Scopa	Annotazioni varie famiglia 'Scopa'	1	1	5.00	5	4.00

dove gli articoli appartengo ad una famiglia che a sua volta appartiene ad una categoria.

Subquery (o sottoquery) scalare correlate utilizzate per la definizione di una colonna e utilizzate nella clausola WHERE

Esempio:

Definizione di una query che deve visualizzare:

- l'elenco delle categorie articoli (codice+descrizione);
- il numero delle famiglie articoli (calcolato sulla tabella famiglie);
- il numero articoli (calcolato sulla tabella articoli);
- il numero delle confezioni degli articoli (calcolato sulla tabella articoli);
- il numero degli articoli (calcolato sulla tabella articoli);
- il totale del valore degli articoli (calcolato sulla tabella articoli).

Inoltre deve effettuare il controllo ('@DEFQUERYWHERE) sul totale del valore calcolato dalla tabella articoli per visualizzare solo le righe con il totale valore > di 300.

Anteprima risultato della query:

	CategID	CategDescrizione	TOT_FAM	TOT_ART	TOT_CONF	TOT_PEZZI	TOT_VALORE
1	10	Cancelleria	5	15	150.00	795.00	435.0000
2	40	Mobili ufficio	4	12	42.00	42.00	2292.0000
3	50	Elettricit� e Luce	4	12	120.00	225.00	945.0000

vediamo la sua definizione sul sorgente BC, di seguito   riportata la definizione della query principale:

BC

```
'=====

'@SR DEF_QUERY

'=====

'

IDQUERY[8]=0

IDSUBQUERY[8]=0

IDSUBQUERY_2[8]=0

IDSUBQUERY_3[8]=0

IDSUBQUERY_4[8]=0

IDSUBQUERY_5[8]=0

IDSUBQUERY_6[8]=0

'

' Definizione query principale sulla tabella Articoli

'

'@DEFQUERY IDQUERY[IDQUERY]

'

' Definizione Subquery (mi serve avere l'ID)

'

'@DEFSUBQUERY IDQUERY[IDSUBQUERY] IDQUERYPADRE[IDQUERY]

'@DEFSUBQUERY IDQUERY[IDSUBQUERY_2] IDQUERYPADRE[IDQUERY]

'@DEFSUBQUERY IDQUERY[IDSUBQUERY_3] IDQUERYPADRE[IDQUERY]

'@DEFSUBQUERY IDQUERY[IDSUBQUERY_4] IDQUERYPADRE[IDQUERY]

'@DEFSUBQUERY IDQUERY[IDSUBQUERY_5] IDQUERYPADRE[IDQUERY]

'@DEFSUBQUERY IDQUERY[IDSUBQUERY_6] IDQUERYPADRE[IDQUERY]

'

'@DEFQUERYFROM IDQUERY[IDQUERY] TABELLA[BSVT001]

'@DEFQUERYPADRE IDQUERY[IDQUERY] _

        ESPR[[BSVT001.IDCAT]] VAR[TR1IDCAT] _

        ESPR[[BSVT001.CCDES]] VAR[TR1CCDES$]

'

' Inserimento utilizzo subquery SUBQ[] con l'ID sopra definito

'

TOT_FAM[9]=0

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDQUERY] ESPR[SUBQ(IDSUBQUERY) AS [TOT_FAM] VAR[TOT_FAM]]

'

' Inserimento utilizzo subquery SUBQ[] con l'ID sopra definito

'

TOT_ART[9]=0

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDQUERY] ESPR[SUBQ(IDSUBQUERY_2) AS [TOT_ART] VAR[TOT_ART]]

'

' Inserimento utilizzo subquery SUBQ[] con l'ID sopra definito

'

TOT_CONF[9]=0

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDQUERY] ESPR[SUBQ(IDSUBQUERY_3) AS [TOT_CONF]] VAR[TOT_CONF]

'

' Inserimento utilizzo subquery SUBQ[] con l'ID sopra definito

'

TOT_PEZZI[9]=0
```

```
'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDQUERY] ESPR[SUBQ(IDSUBQUERY_4) AS [TOT_PEZZI]] VAR[TOT_PEZZI]
'
' Inserimento utilizzo subquery SUBQ[] con l'ID sopra definito
'
TOT_VALORE[9]=0
'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDQUERY] ESPR[SUBQ(IDSUBQUERY_5) AS [TOT_VALORE]] VAR[TOT_VALORE]
'
' Inserimento utilizzo in WHERE della subquery SUBQ[] con l'ID sopra definito
'
'@DEFQUERYWHERE IDQUERY[IDQUERY] ESPR[SUBQ(IDSUBQUERY_6)>300]
'
' Completamento definizione Subquery
'
GOSUB DEF_SUBQUERY
GOSUB DEF_SUBQUERY_2
GOSUB DEF_SUBQUERY_3
GOSUB DEF_SUBQUERY_4
GOSUB DEF_SUBQUERY_5
GOSUB DEF_SUBQUERY_6
'
RETURN
```

e questa la definizione delle subquery:

```
BC
```

```
'=====

'@SR DEF_SUBQUERY

'=====

'

' Subquery su tabella famiglie per contare le famiglie

'

'@DEFQUERYFROM IDQUERY[IDSUBQUERY] TABELLA[BSVT002]

'@DEFQUERYPARTITION IDQUERY[IDSUBQUERY] _

    ESPR[COUNT(BSVT002.IDFAM) AS [TOT_IDFAM]]

'@DEFQUERYWHERE IDQUERY[IDSUBQUERY] ESPR[BSVT002.IDCAT=BSVT001.IDCAT]

'

RETURN

'

'=====

'@SR DEF_SUBQUERY_2

'=====

'

' Subquery su tabella articoli per contare gli articoli

'

'@DEFQUERYFROM IDQUERY[IDSUBQUERY_2] TABELLA[BSVT003]

'@DEFQUERYPARTITION IDQUERY[IDSUBQUERY_2] _

    ESPR[COUNT(BSVT003.IDART) AS [TOT_IDART]]

'@DEFQUERYWHERE IDQUERY[IDSUBQUERY_2] ESPR[BSVT003.IDCAT=BSVT001.IDCAT]

'

RETURN

'

'=====

'@SR DEF_SUBQUERY_3

'=====

'

' Subquery su tabella articoli per contare le confezioni

'

'@DEFQUERYFROM IDQUERY[IDSUBQUERY_3] TABELLA[BSVT003]

'@DEFQUERYPARTITION IDQUERY[IDSUBQUERY_3] _

    ESPR[SUM(BSVT003.CLQUCO) AS [SUM_CLQUCO]]

'@DEFQUERYWHERE IDQUERY[IDSUBQUERY_3] ESPR[BSVT003.IDCAT=BSVT001.IDCAT]

'

RETURN

'

'=====

'@SR DEF_SUBQUERY_4

'=====

'

' Subquery su tabella articoli per calcolare le unità articoli

'(numero confezioni * articoli per confezione)

'

'@DEFQUERYFROM IDQUERY[IDSUBQUERY_4] TABELLA[BSVT003]

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDSUBQUERY_4] _
```



```
        ESPR(SUM(BSVT003.CIUNCO*BSVT003.CLQUCO) AS [TOT_PEZZI_D] FMT[NUMERICO[9]])

'@DEFQUERYWHERE  IDQUERY[IDSUBQUERY_4] ESPR[BSVT003.IDCAT=BSVT001.IDCAT]
'
'
RETURN
'
'
'=====
'@SR DEF_SUBQUERY_5
'=====
'
'
' Subquery su tabella articoli per calcolare il totale valore articoli
' (numero confezioni * articoli per confezione * costo unitario)
'
'@DEFQUERYFROM  IDQUERY[IDSUBQUERY_5] TABELLA[BSVT003]
'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDSUBQUERY_5] _
        ESPR(SUM(BSVT003.CIUNCO*BSVT003.CLQUCO*BSVT003.CDCOUN) AS [TOT_VALORE_D] _
        FMT[NUMERICO[9]])
'@DEFQUERYWHERE  IDQUERY[IDSUBQUERY_5] ESPR[BSVT003.IDCAT=BSVT001.IDCAT]
'
'
RETURN
'
'
'=====
'@SR DEF_SUBQUERY_6
'=====
'
'
' Subquery su tabella articoli per calcolare il totale valore articoli
' (utilizzata sulla query principale nella '@DEFQUERYWHERE)
'
'@DEFQUERYFROM  IDQUERY[IDSUBQUERY_6] TABELLA[BSVT003]
'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDSUBQUERY_6] _
        ESPR(SUM(BSVT003.CIUNCO*BSVT003.CLQUCO*BSVT003.CDCOUN) AS [TOT_VALORE_D2] _
        FMT[NUMERICO[9]])
'@DEFQUERYWHERE  IDQUERY[IDSUBQUERY_6] ESPR[BSVT003.IDCAT=BSVT001.IDCAT]
'
'
RETURN
```

che generano in esecuzione il seguente codice SQL:

```
SQL
```

```
SELECT  BLDSVIL_Categorie.CategID AS [CategID] ,
        BLDSVIL_Categorie.CategDescrizione AS [CategDescrizione] ,
        ( SELECT      COUNT(BLDSVIL_Famiglie.FamID) AS [TOT_IDFAM]
          FROM        BLDSVIL_Famiglie AS [BLDSVIL_Famiglie]
          WHERE       ( BLDSVIL_Famiglie.FamCatID = BLDSVIL_Categorie.CategID )
        ) AS [TOT_FAM] ,
        ( SELECT      COUNT(BLDSVIL_Articoli.ArtID) AS [TOT_IDART]
          FROM        BLDSVIL_Articoli AS [BLDSVIL_Articoli]
          WHERE       ( BLDSVIL_Articoli.ArtCatID = BLDSVIL_Categorie.CategID )
        ) AS [TOT_ART] ,
        ( SELECT      SUM(Tabella2.ArtQuantitaConf) AS [SUM_CLQUCO]
          FROM        BLDSVIL_Articoli AS [Tabella2]
          WHERE       ( Tabella2.ArtCatID = BLDSVIL_Categorie.CategID )
        ) AS [TOT_CONF] ,
        ( SELECT      SUM(Tabella3.ArtUnitaConf * Tabella3.ArtQuantitaConf) AS [TOT_PEZZI_D]
          FROM        BLDSVIL_Articoli AS [Tabella3]
          WHERE       ( Tabella3.ArtCatID = BLDSVIL_Categorie.CategID )
        ) AS [TOT_PEZZI] ,
        ( SELECT      SUM(Tabella4.ArtUnitaConf * Tabella4.ArtQuantitaConf
                           * Tabella4.ArtCostoUnit) AS [TOT_VALORE_D]
          FROM        BLDSVIL_Articoli AS [Tabella4]
          WHERE       ( Tabella4.ArtCatID = BLDSVIL_Categorie.CategID )
        ) AS [TOT_VALORE]
FROM    BLDSVIL_Categorie AS [BLDSVIL_Categorie]
WHERE   ( ( SELECT      SUM(Tabella5.ArtUnitaConf * Tabella5.ArtQuantitaConf
                           * Tabella5.ArtCostoUnit) AS [TOT_VALORE_D2]
          FROM        BLDSVIL_Articoli AS [Tabella5]
          WHERE       ( Tabella5.ArtCatID = BLDSVIL_Categorie.CategID )
        ) > 300 )
```

Subquery (o sottoquery) scalare correlate nidificate

Esempio:

Definizione di una query che deve visualizzare gli articoli disponibili, per sapere se un articolo è disponibile viene eseguita una subquery per leggere la famiglia dell'articolo che a sua volta esegue una subquery per leggere la categoria dell'articolo; l'informazione "disponibile" risiede sulla tabella delle categorie articoli.

Anteprima risultato della query:

	ArtCatID	ArtFamID	ArtID	ArtDescrizione	FL_DISP
1	10	10	1	Penna a sfera colore nero	1
2	10	10	2	Penna a sfera colore blu	1
3	10	10	3	Penna a sfera colore rosso	1
4	10	20	4	Portamine 0.5mm	1
5	10	20	5	Portamine 1.2mm	1
6	10	20	6	Portamine 1.4mm	1
7	10	30	7	Forbici	1
8	10	30	8	Cutter	1
9	10	30	9	Righello	1
10	10	40	10	Rotolo scotch	1
11	10	40	11	Colla Pritt	1
12	10	40	12	Bianchetto	1
13	10	50	13	Post-It segnalibro	1
14	10	50	14	Post-It quadrati gialli	1
15	10	50	15	Post-It quadrati rosa	1
16	20	60	16	Vite legno 30mm diametro 4mm	1
17	20	60	17	Vite legno 35mm diametro 5mm	1
...					

vediamo la sua definizione sul sorgente BC, di seguito è riportata la definizione della query:

```
BC
```

```
'=====

'@SR DEF_QUERY

'=====

'

IDQUERY[8]=0

IDSUBQUERY[8]=0

IDSUBQUERY_2[8]=0

'

' Definizione query principale sulla tabella Articoli

'

'@DEFQUERY IDQUERY[IDQUERY]

'

' Definizione Subquery (mi serve avere l'ID)

'

'@DEFSUBQUERY IDQUERY[IDSUBQUERY] IDQUERYPADRE[IDQUERY]

'@DEFSUBQUERY IDQUERY[IDSUBQUERY_2] IDQUERYPADRE[IDSUBQUERY]

'

'@DEFQUERYFROM IDQUERY[IDQUERY] TABELLA[BSVT003]

'@DEFQUERYSUBQUERY IDQUERY[IDQUERY] _

    ESPR[[BSVT003.IDCAT]] VAR[TR3IDCAT] _

    ESPR[[BSVT003.IDFAM]] VAR[TR3IDFAM] _

    ESPR[[BSVT003.IDART]] VAR[TR3IDART] _

    ESPR[[BSVT003.CCDES]] VAR[TR3CCDES$]

'

' Inserimento utilizzo subquery SUBQ[] con l'ID sopra definito

'

FL_DISP[1]=0

'@DEFQUERYSUBQUERY IDQUERY[IDQUERY] ESPR[SUBQ(IDSUBQUERY) AS [FL_DISP] VAR[FL_DISP]]

'@DEFQUERYWHERE IDQUERY[IDQUERY] ESPR[[FL_DISP]=1]

'

' Completamento definizione Subquery

'

GOSUB DEF_SUBQUERY

GOSUB DEF_SUBQUERY_2

'

RETURN

'

'=====

'@SR DEF_SUBQUERY

'=====

'

' Subquery di primo livello su tabella famiglie per cercare la famiglia correlata

'

'@DEFQUERYFROM IDQUERY[IDSUBQUERY] TABELLA[BSVT002]

'

' Inserimento utilizzo subquery SUBQ[] con l'ID sopra definito

'

'@DEFQUERYSUBQUERY IDQUERY[IDSUBQUERY] ESPR[SUBQ(IDSUBQUERY_2) AS [FL_DISP2] FMT[NUMERICO[1]]]
```

```
'@DEFQUERYWHERE IDQUERY[IDSUBQUERY] ESPR[BSVT002.IDFAM=BSVT003.IDFAM]
'
RETURN
'
'=====
'@SR DEF_SUBQUERY_2
'-----
'
' Subquery di secondo livello su tabella categorie per cercare la categoria correlata
'
'@DEFQUERYFROM IDQUERY[IDSUBQUERY_2] TABELLA[BSVT001]
'@DEFQUERYPARTIAL IDQUERY[IDSUBQUERY_2] ESPR[BSVT001.CNFDISP AS [FL_DISP1]]
'@DEFQUERYWHERE IDQUERY[IDSUBQUERY_2] ESPR[BSVT001.IDCAT=BSVT002.IDCAT]
'
RETURN
```

che genera in esecuzione il seguente codice SQL:

```
SQL

SELECT  BLDVVIL_Articoli.ArtCatID AS [ArtCatID] ,
        BLDVVIL_Articoli.ArtFamID AS [ArtFamID] ,
        BLDVVIL_Articoli.ArtID AS [ArtID] ,
        BLDVVIL_Articoli.ArtDescrizione AS [ArtDescrizione] ,
        ( SELECT ( SELECT BLDVVIL_Categorie.CategFLDisponibile AS [FL_DISP1]
                      FROM  BLDVVIL_Categorie AS [BLDVVIL_Categorie]
                      WHERE   ( BLDVVIL_Categorie.CategID = BLDVVIL_Famiglie.FamCatID )
                    ) AS [FL_DISP2]
          FROM  BLDVVIL_Famiglie AS [BLDVVIL_Famiglie]
          WHERE ( BLDVVIL_Famiglie.FamID = BLDVVIL_Articoli.ArtFamID )
        ) AS [FL_DISP]
FROM    BLDVVIL_Articoli AS [BLDVVIL_Articoli]
WHERE   ( ( (SELECT ( SELECT BLDVVIL_Categorie.CategFLDisponibile AS [FL_DISP1]
                      FROM  BLDVVIL_Categorie AS [BLDVVIL_Categorie]
                      WHERE   ( BLDVVIL_Categorie.CategID = BLDVVIL_Famiglie.FamCatID )
                    ) AS [FL_DISP2]
          FROM  BLDVVIL_Famiglie AS [BLDVVIL_Famiglie]
          WHERE ( BLDVVIL_Famiglie.FamID = BLDVVIL_Articoli.ArtFamID )
        ) ) = 1 )
```

Subquery (o sottoquery) scalare sulla stessa tabella della query principale

Esempio:

Definizione di una query che:

- visualizza tutti gli articoli;
- su ogni riga viene riportata la colonna "Numero confezioni", prima subquery che calcola il numero delle confezioni totali della famiglia a cui l'articolo appartiene;
- su ogni riga viene riportata la colonna "Totale pezzi", seconda subquery che calcola il numero totale di pezzi della famiglia a cui l'articolo appartiene applicando un moltiplicatore (campo calcolato dalla query principale).

Anteprima risultato della query:

	ArtCatID	ArtFamID	ArtID	ArtDescrizione	NUM_PEZZI	MOLTIPLICA	NUM_CONF	TOT_PEZZI
1	10	10	1	Penna a sfera colore nero	75.00	2	15.00	450.00
2	10	10	2	Penna a sfera colore blu	75.00	2	15.00	450.00
3	10	10	3	Penna a sfera colore rosso	75.00	2	15.00	450.00
4	10	20	4	Portamine 0.5mm	50.00	1	15.00	150.00
5	10	20	5	Portamine 1.2mm	50.00	1	15.00	150.00
6	10	20	6	Portamine 1.4mm	50.00	1	15.00	150.00
7	10	30	7	Forbici	10.00	1	30.00	30.00
8	10	30	8	Cutter	10.00	1	30.00	30.00
9	10	30	9	Righello	10.00	1	30.00	30.00
10	10	40	10	Rotolo scotch	50.00	1	30.00	150.00
11	10	40	11	Colla Pritt	50.00	1	30.00	150.00
12	10	40	12	Bianchetto	50.00	1	30.00	150.00
13	10	50	13	Post-It segnalibro	80.00	1	60.00	240.00
14	10	50	14	Post-It quadrati gialli	80.00	1	60.00	240.00
15	10	50	15	Post-It quadrati rosa	80.00	1	60.00	240.00
16	50	200	58	Lampada small	5.00	1	15.00	15.00
17	50	200	59	Lampada medium	5.00	1	15.00	15.00

...

vediamo la sua definizione sul sorgente BC, di seguito è riportata la definizione della query:

BC

```
'=====

'@SR DEF_QUERY

'=====

'

IDQUERY[8]=0

IDSUBQUERY[8]=0

IDSUBQUERY2[8]=0

'

' Definizione query principale sulla tabella Articoli

'

'@DEFQUERY IDQUERY[IDQUERY]

'

' Definizione Subquery (mi serve avere l'ID)

'

'@DESUBFQUERY IDQUERY[IDSUBQUERY] IDQUERYPADRE[IDQUERY]

'@DEFSUBQUERY IDQUERY[IDSUBQUERY2] IDQUERYPADRE[IDQUERY]

'

'@DEFQUERYFROM IDQUERY[IDQUERY] TABELLA[BSVT003]

'@DEFQUERYCOLUMN IDQUERY[IDQUERY] _

    ESPR[GROUPBY(BSVT003.IDCAT)] VAR[TR3IDCAT] _

    ESPR[GROUPBY(BSVT003.IDFAM)] VAR[TR3IDFAM] _

    ESPR[GROUPBY(BSVT003.IDART)] VAR[TR3IDART] _

    ESPR[GROUPBY(BSVT003.CCDES)] VAR[TR3CCDES$]

'

NUM_PEZZI[9]=0

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDQUERY] ESPR[SUM(BSVT003.CIUNCO*BSVT003.CLQUCO) AS [NUM_PEZZI]

FMT[NUMERICO[9]] VAR[NUM_PEZZI]]

'

MULTIPLICATORE[2]=0

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDQUERY] ESPR[IF BSVT003.IDFAM=10 THEN 2 ELSE 1 AS[MOLTIPLICA]

FMT[NUMERICO[1]] VAR[MULTIPLICATORE]]

'

' Inserimento utilizzo subquery SUBQ[] con l'ID sopra definito

'

NUM_CONF[1]=0

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDQUERY] ESPR[SUBQ(IDSUBQUERY) AS [NUM_CONF] VAR[NUM_CONF]]

'

' Inserimento utilizzo subquery SUBQ[] con l'ID sopra definito

'

TOT_PEZZI[1]=0

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDQUERY] ESPR[SUBQ(IDSUBQUERY2) AS [TOT_PEZZI] VAR[TOT_PEZZI]]

'

'@DEFQUERYWHERE IDQUERY[IDQUERY] SELEZIONI[(BSVT003.IDCAT=10]

'@DEFQUERYWHERE IDQUERY[IDQUERY] OPERATORE[OR] SELEZIONI[(BSVT003.IDCAT=50)]

'

' Completamento definizione Subquery

'

GOSUB DEF_SUBQUERY

GOSUB DEF_SUBQUERY2
```

```
'
RETURN

'

'=====

'@SR_DEF_SUBQUERY

'=====

'

' Subquery sulla stessa tabella (Articoli) per contare le confezioni

'

'@DEFQUERYFROM IDQUERY[IDSUBQUERY] TABELLA[BSVT003@1]

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDSUBQUERY] ESPR[SUM(BSVT003@1.CLQUCO) AS [NUM_CONF2]
FMT[NUMERICO[9]]]

'@DEFQUERYWHERE IDQUERY[IDSUBQUERY] ESPR[BSVT003@1.IDFAM=BSVT003.IDFAM]

'

RETURN

'

'=====

'@SR_DEF_SUBQUERY2

'=====

'

' Subquery sulla stessa tabella (Articoli) per calcolare le unità articoli

'(*moltiplicatore, campo calcolato dalla query principale)

'

'@DEFQUERYFROM IDQUERY[IDSUBQUERY2] TABELLA[BSVT003@2]

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDSUBQUERY2]
ESPR[SUM(BSVT003@2.CIUNCO*BSVT003@2.CLQUCO)*SUM([MOLTIPLICA]) AS [TOT_PZZI2] FMT[NUMERICO[9]]]

'@DEFQUERYWHERE IDQUERY[IDSUBQUERY2] ESPR[BSVT003@2.IDFAM=BSVT003.IDFAM]

'

RETURN
```

che genera in esecuzione il seguente codice SQL:

```
SQL
```

```
SELECT  BLDSVIL_Articoli.ArtCatID AS [ArtCatID] ,
        BLDSVIL_Articoli.ArtFamID AS [ArtFamID] ,
        BLDSVIL_Articoli.ArtID AS [ArtID] ,
        BLDSVIL_Articoli.ArtDescrizione AS [ArtDescrizione] ,
        SUM(BLDSVIL_Articoli.ArtUnitaConf * BLDSVIL_Articoli.ArtQuantitaConf) AS [NUM_PEZZI] ,
        CASE WHEN BLDSVIL_Articoli.ArtFamID = 10 THEN 2
              ELSE 1
        END AS [MOLTIPLICA] ,
        ( SELECT      SUM(Tabella2.ArtQuantitaConf) AS [NUM_CONF2]
          FROM        BLDSVIL_Articoli AS [Tabella2]
          WHERE       ( Tabella2.ArtFamID = BLDSVIL_Articoli.ArtFamID )
        ) AS [NUM_CONF] ,
        ( SELECT      SUM(Tabella3.ArtUnitaConf * Tabella3.ArtQuantitaConf)
          * SUM(( CASE WHEN BLDSVIL_Articoli.ArtFamID = 10 THEN 2
                      ELSE 1
                    END )) AS [TOT_PEZZI2]
          FROM        BLDSVIL_Articoli AS [Tabella3]
          WHERE       ( Tabella3.ArtFamID = BLDSVIL_Articoli.ArtFamID )
        ) AS [TOT_PEZZI]
FROM    BLDSVIL_Articoli AS [BLDSVIL_Articoli]
WHERE   ( ( BLDSVIL_Articoli.ArtCatID = 10 )
        OR ( BLDSVIL_Articoli.ArtCatID = 50 )
        )
GROUP BY BLDSVIL_Articoli.ArtCatID ,
        BLDSVIL_Articoli.ArtFamID ,
        BLDSVIL_Articoli.ArtID ,
        BLDSVIL_Articoli.ArtDescrizione
```

⚠ La definizione di una subquery sulla stessa tabella della query esterna richiede che sia identificata in modo esplicito la tabella su cui lavora identificandola in modo univoco, nel codice sorgente BC potete notare l'utilizzo del suffisso "@1" e "@2" utilizzati sulle due subquery che generano "Tabella1" e "Tabella2" sul codice SQL.

Subquery (o sottoquery) non correlata all'interno dell'operatore IN

Esempio:
Definizione di una query che visualizza gli articoli appartenenti solo ad alcune categorie.

Anteprima risultato della query:

	ArtCatID	ArtFamID	ArtID	ArtDescrizione
1	10	10	1	Penna a sfera colore nero
2	10	10	2	Penna a sfera colore blu
3	10	10	3	Penna a sfera colore rosso
4	10	20	4	Portamine 0.5mm
5	10	20	5	Portamine 1.2mm
6	10	20	6	Portamine 1.4mm
7	10	30	7	Forbici
8	10	30	8	Cutter
9	10	30	9	Righello
10	10	40	10	Rotolo scotch
11	10	40	11	Colla Pritt
12	10	40	12	Bianchetto
13	10	50	13	Post-It segnalibro
14	10	50	14	Post-It quadrati gialli
15	10	50	15	Post-It quadrati rosa
16	30	110	31	Detergente liquido
17	30	110	32	Detergente polvere

vediamo la sua definizione sul sorgente BC, di seguito è riportata la definizione della query:

```
BC
```



```
'=====

'@SR DEF_QUERY

'=====

'

IDQUERY[8]=0

IDSUBQUERY[8]=0

'

' Definizione query principale sulla tabella Articoli

'

'@DEFQUERY IDQUERY[IDQUERY]

'

' Definizione Subquery (mi serve avere l'ID)

'

'@DEFSUBQUERY IDQUERY[IDSUBQUERY] IDQUERYPADRE[IDQUERY]

'

'@DEFQUERYFROM IDQUERY[IDQUERY] TABELLA[BSVT003]

'@DEFQUERYPADRE IDQUERY[IDQUERY] _

    ESPR[BSVT003.IDCAT] VAR[TR3IDCAT] _

    ESPR[BSVT003.IDFAM] VAR[TR3IDFAM] _

    ESPR[BSVT003.IDART] VAR[TR3IDART] _

    ESPR[BSVT003.CCDES] VAR[TR3CCDES$]

'

' Inserimento utilizzo nella clausola IN della subquery SUBQ[] con l'ID sopra definito

'

'@DEFQUERYWHERE IDQUERY[IDQUERY] ESPR[INLIST(BSVT003.IDCAT,SUBQ[IDSUBQUERY])>0]

'

' Completamento definizione Subquery

'

GOSUB DEF_SUBQUERY

'

RETURN

'

'=====

'@SR DEF_SUBQUERY

'=====

'

' Subquery su tabella categorie per restituire l'elenco categorie che viene testato

' nella INLIST sulla query principale

'

'@DEFQUERYFROM IDQUERY[IDSUBQUERY] TABELLA[BSVT001]

'@DEFQUERYPADRE IDQUERY[IDSUBQUERY] ESPR[BSVT001.IDCAT AS [ID_CAT]]

'@DEFQUERYWHERE IDQUERY[IDSUBQUERY] ESPR[INLIST(BSVT001.IDCAT,10,30,50)>0]

'

RETURN
```

quest il record-set restituito dalla subquery:

	ID_CAT
1	10
2	30
3	50

di seguito il codice SQL generato in esecuzione:

SQL

```
SELECT  BLDSVIL_Articoli.ArtCatID AS [ArtCatID] ,
        BLDSVIL_Articoli.ArtFamID AS [ArtFamID] ,
        BLDSVIL_Articoli.ArtID AS [ArtID] ,
        BLDSVIL_Articoli.ArtDescrizione AS [ArtDescrizione]
FROM    BLDSVIL_Articoli AS [BLDSVIL_Articoli]
WHERE   ( BLDSVIL_Articoli.ArtCatID IN (
        SELECT  BLDSVIL_Categorie.CategID AS [ID_CAT]
        FROM    BLDSVIL_Categorie AS [BLDSVIL_Categorie]
        WHERE   ( BLDSVIL_Categorie.CategID IN ( 10, 30, 50 ) ) ) )
```

Subquery (o sottoquery) all'interno della clausola JOIN

All'interno della clausola JOIN e della clausola FROM è possibile utilizzare una tabella derivata, si tratta di una tabella che viene creata durante l'esecuzione della query esterna, quindi di un result-set ottenuto da una query. Una tabella derivata rappresenta un tipo speciale di subquery che viene posizionata nella clausola JOIN o nella clausola FROM della query ed è referenziata tramite un alias. Nota bene: è obbligatorio assegnare un nome alla subquery per poterla referenziare nelle altre parti della query principale.

Esempio:

Definizione di una query che visualizza tutti gli articoli, due delle colonne del record-set sono definite da due subquery inserite in JOIN.

Anteprima risultato della query:

	ArtID	ArtDescrizione	CategID	FamID	ArtDataUltAgg	ArtNote	ArtCostoUnit	ArtPerPrecAgg	ArtUnità
1	1	Penna a sfera colore nero	10	10	2014-01-01	Note Penna a sfera colore nero	1.10	201312	15
2	2	Penna a sfera colore blu	10	10	2014-01-01	Note Penna a sfera colore blu	1.10	201312	10
3	3	Penna a sfera colore rosso	10	10	2014-01-01	Note Penna a sfera colore rosso	1.10	201312	5
4	4	Portamina 0,5mm hb	10	20	2014-01-01	Note Portamina 0,5mm hb	1.00	201312	2
5	5	Portamina 1,2mm hb	10	20	2014-01-01	Note Portamina 1,2mm hb	1.50	201312	2
6	6	Portamina 1,4mm hb	10	20	2014-01-01	Note Portamina 1,4mm hb	2.00	201312	2
7	7	Forbici	10	30	2014-01-01	Note Forbici	2.00	201312	5
8	8	Cutter	10	30	2014-01-01	Note Cutter	2.50	201312	2
9	9	Righello	10	30	2014-01-01	Note Righello	1.50	201312	5
10	10	Rotolo scotch	10	40	2014-01-01	Note Rotolo scotch	0.75	201312	3
11	11	Bianchetto	10	40	2014-01-01	Note Bianchetto	1.50	201312	4
12	12	Colla pritt	10	40	2014-01-01	Note Colla pritt	1.20	201312	5
13	13	Post-it segnalibro	10	50	2014-01-01	Note Post-it segnalibro	0.50	201312	15
14	14	Post-it quadrati gialli	10	50	2014-01-01	Note Post-it quadrati gialli	0.80	201312	20
15	15	Post-it rettangolari gialli	10	50	2014-01-01	Note Post-it rettangolari gialli	0.90	201312	20
16	16	Vite legno 0,4mm	20	60	2014-01-01	Note Vite legno 0,4mm	0.01	201312	2
17	17	Vite acciaio 0,4mm	20	60	2014-01-01	Note Vite acciaio 0,4mm	0.01	201312	2

le colonne "CategID" e "FamID" sono definite dalle subquery in JOIN, di seguito è riportata la definizione della query sul sorgente BC:

BC

```
'=====
'@SR DEF_QUERY
'=====
'
IDQUERY[5]  =0
IDSUBQUERY[5] =0
IDSUBQUERY2[5]=0
'
' Definizione query principale sulla tabella Articoli
'
'@DEFQUERY IDQUERY[IDQUERY]
'
' Definizione Subquery (mi serve avere l'ID)
'
'@DEFSUBQUERY IDQUERY[IDSUBQUERY] IDQUERYPADRE[IDQUERY]
'
' Subquery su tabella categorie per avere una tabella in memoria
' l'elenco categorie
'
'@DEFQUERYFROM IDQUERY[IDSUBQUERY] TABELLA[BSVT001]
'
'@DEFQUERYCOLUMN IDQUERY[IDSUBQUERY] ESPR[BSVT001.IDCAT]
VAR[TR1IDCAT]
'
'@DEFSUBQUERY IDQUERY[IDSUBQUERY2] IDQUERYPADRE[IDQUERY]
'
' Subquery su tabella famiglie per avere in memoria l'elenco famiglie
'
'@DEFQUERYFROM IDQUERY[IDSUBQUERY2] TABELLA[BSVT002]
'
'@DEFQUERYCOLUMN IDQUERY[IDSUBQUERY2] ESPR[BSVT002.IDFAM]
VAR[TR2IDFAM]
'
'@DEFQUERYFROM IDQUERY[IDQUERY] TABELLA[BSVT003]
'
'@DEFQUERYCOLUMN IDQUERY[IDQUERY] ESPR[BSVT003.*] VAR[]
'
'@DEFQUERYJOIN IDQUERY[IDQUERY] TABELLA[SUBQ(IDSUBQUERY) AS [SUBQUERY1]]
TABELLAPADRE[BSVT003] _
    REL[[SUBQUERY1].CategID=BSVT003.IDCAT] _
    OPERATOREJOIN["INNER"]
'
'@DEFQUERYCOLUMN IDQUERY[IDQUERY] ESPR[SUBQUERY1].CategID] VAR[TR1IDCAT]
'
'@DEFQUERYJOIN IDQUERY[IDQUERY] TABELLA[SUBQ(IDSUBQUERY2) AS [SUBQUERY2]]
TABELLAPADRE[BSVT003] _
    REL[[SUBQUERY2].FamID=BSVT003.IDFAM] _
    OPERATOREJOIN["INNER"]
'
'@DEFQUERYCOLUMN IDQUERY[IDQUERY] ESPR[SUBQUERY2].FamID] VAR[TR2IDFAM]
```

RETURN

questo è il record-set restituito dalla subquery 1 inserita nella prima clausola JOIN:

	CategID
1	10
2	20
3	30
4	40
5	50

e questo il record-set restituito dalla subquery 2 inserita nella seconda clausola JOIN:

	FamID
1	10
2	20
3	30
4	40
5	50
6	60
7	70
8	80
9	90
10	100
11	110
12	120
13	130
14	140
15	150
16	160
17	170
...	

di seguito il codice SQL generato in esecuzione:

SQL	
<pre>SELECT BLDSVIL_Articoli.ArtID AS [ArtID] , BLDSVIL_Articoli.ArtDescrizione AS [ArtDescrizione] , SUBQUERY1.CategID AS [CategID] , SUBQUERY2.FamID AS [FamID] , BLDSVIL_Articoli.ArtDataUltAgg AS [ArtDataUltAgg] , BLDSVIL_Articoli.ArtNote AS [ArtNote] , BLDSVIL_Articoli.ArtCostoUnit AS [ArtCostoUnit] , BLDSVIL_Articoli.ArtPerPrecAgg AS [ArtPerPrecAgg] , BLDSVIL_Articoli.ArtUnitaConf AS [ArtUnitaConf] , BLDSVIL_Articoli.ArtQuantitaConf AS [ArtQuantitaConf] , BLDSVIL_Articoli.ArtQuantitaUnita AS [ArtQuantitaUnita] , BLDSVIL_Articoli.ArtFLDisponibile AS [ArtFLDisponibile] , BLDSVIL_Articoli.ArtCatID AS [ArtCatID] , BLDSVIL_Articoli.ArtFamID AS [ArtFamID] FROM BLDSVIL_Articoli AS [BLDSVIL_Articoli] INNER JOIN (SELECT Tabella4.CategID AS [CategID] FROM BLDSVIL_Categorie AS [Tabella4]) AS [SUBQUERY1] ON (SUBQUERY1.CategID = BLDSVIL_Articoli.ArtCatID) INNER JOIN (SELECT Tabella5.FamID AS [FamID] FROM BLDSVIL_Famiglie AS [Tabella5]) AS [SUBQUERY2] ON (SUBQUERY2.FamID = BLDSVIL_Articoli.ArtFamID)</pre>	

⚠ E' necessario definire un ALIAS alla subquery e utilizzare il nome esteso del campo per definire la relazione tra la subquery e la tabella indicata in PADRE[].

BC	
<pre>'@DEFQUERYJOIN IDQUERY[IDQUERY] TABELLA[SUBQ(IDSUBQUERY) AS [SUBQUERY1]] TABELLAPADRE[BSVT003] _ REL[[SUBQUERY1].CategID=BSVT003.IDCAT] _ OPERATOREJOIN["INNER"]</pre>	

SQL	
<pre>INNER JOIN (SELECT Tabella4.CategID AS [CategID] FROM BLDSVIL_Categorie AS [Tabella4]) AS [SUBQUERY1] ON (SUBQUERY1.CategID = BLDSVIL_Articoli.ArtCatID)</pre>	

Subquery (o sottoquery) all'interno della clausola FROM

All'interno della clausola JOIN e della clausola FROM è possibile utilizzare una tabella derivata, si tratta di una tabella che viene creata durante l'esecuzione della query esterna, quindi di un result-set ottenuto da una query.

Una tabella derivata rappresenta un tipo speciale di subquery che viene posizionata nella clausola JOIN o nella clausola FROM della query ed è referenziata tramite un alias.

Nota bene: è obbligatorio assegnare un nome alla subquery per poterla referenziare nelle altre parti della query principale.

Esempio:

Definizione di una query che visualizza tutti gli articoli che hanno il valore totale di tutte le unità in magazzino superiore ad 80€.

(campo calcolato: <valore totale>=<numero confezioni>*<unità per confezione>*<costo unitario>)

Anteprima risultato della query:

	ArtID	ArtDescrizione	TOT_VALORE_D	CategID	FamID
1	1	Penna a sfera colore nero	165.0000	10	10
2	2	Penna a sfera colore blu	110.0000	10	10
3	14	Post-It quadrati gialli	128.0000	10	50
4	18	Tassello nylon 0,4mm	3000.0000	20	70
5	19	Tassello nylon 0,6mm	3000.0000	20	70
6	20	Tassello nylon 0,8mm	3000.0000	20	70
7	41	Scrivania lung.110cm	270.0000	40	160
8	42	Scrivania lung.100cm	240.0000	40	160
9	43	Scrivania lung.90cm	210.0000	40	160
10	44	Sedia rigida	150.0000	40	170
11	45	Sedia morbida	175.0000	40	170
12	46	Sedia morbida con braccioli	200.0000	40	170
13	49	Armadio alto h.190cm	85.0000	40	180
14	54	Lampada da scrivania bassa	175.0000	50	200
15	55	Lampada da scrivania alta	120.0000	50	200
16	56	Lampada da pavimento	135.0000	50	200
17	61	Lampadine basso consumo	160.0000	50	220

...

le colonne "CategID" e "FamID" sono definite dalle subquery in JOIN, di seguito è riportata la definizione della query sul sorgente BC:

BC

```
'=====
'@SR DEF_QUERY
'=====
'
IDQUERY[5]  =0
IDSUBQUERY1[5]=0
IDSUBQUERY2[5]=0
IDSUBQUERY3[5]=0
'
TOT_VALORE[13,2] =0
TOT_VALORE_D[13,2]=0
'
' Definizione query principale sulla tabella Articoli
'
'@DEFQUERY IDQUERY[IDQUERY]
'
' Definizione Subquery (mi serve avere l'ID)
'
'@DEFSUBQUERY IDQUERY[IDSUBQUERY1] IDQUERYPADRE[IDQUERY]
'
' Subquery 1 su tabella categorie per avere l'elenco categorie in memoria
'
'@DEFQUERYFROM IDQUERY[IDSUBQUERY1] TABELLA[BSVT001]
'
'@DEFQUERYCOLUMN IDQUERY[IDSUBQUERY1] ESPR[BSVT001.IDCAT]
VAR[TR1IDCAT]
'
'@DEFSUBQUERY IDQUERY[IDSUBQUERY2] IDQUERYPADRE[IDQUERY]
'
' Subquery 2 su tabella famiglie per avere l'elenco famiglie in memoria
'
'@DEFQUERYFROM IDQUERY[IDSUBQUERY2] TABELLA[BSVT002]
'
'@DEFQUERYCOLUMN IDQUERY[IDSUBQUERY2] ESPR[[BSVT002.IDFAM]
VAR[TR2IDFAM]]
'
'@DEFSUBQUERY IDQUERY[IDSUBQUERY3] IDQUERYPADRE[IDQUERY]
'
' Subquery 3 su tabella articoli per restituire l'elenco articoli
' con un valore totale di dettaglio > di 50
'
'@DEFQUERYFROM IDQUERY[IDSUBQUERY3] TABELLA[BSVT003]
'
'@DEFQUERYCOLUMN IDQUERY[IDSUBQUERY3] ESPR[GROUPBY(BSVT003.*)]
VAR[]
'
'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IDSUBQUERY3] _
    ESPR[SUM(BSVT003.CIUNCO*BSVT003.CLQUCO*BSVT003.CDCOUN) _
    AS [TOT_VALORE_D] FMT[NUMERICO[9] VAR[TOT_VALORE_D]]
```

```

'
'@DEFQUERYWHERE IDQUERY[IDSUBQUERY3] ESPR[([TOT_VALORE_D]>80)]
'
'@DEFQUERYFROM IDQUERY[IDQUERY] TABELLA[SUBQ(IDSUBQUERY3) AS
[QUERY_PRINCIPALE]]
'
'@DEFQUERYCOLUMN IDQUERY[IDQUERY] ESPR[[QUERY_PRINCIPALE].ArtID]
VAR[TR3IDART] _
        ESPR[[QUERY_PRINCIPALE].ArtDescrizione] VAR[TR3CCDES$] _
        ESPR[[QUERY_PRINCIPALE].TOT_VALORE_D] VAR[TOT_VALORE]
'
'@DEFQUERYJOIN IDQUERY[IDQUERY] TABELLA[SUBQ(IDSUBQUERY1) AS [SUBQUERY1]] _
        TABELLAPADRE[[QUERY_PRINCIPALE]] _
        REL[[SUBQUERY1].CategID=[QUERY_PRINCIPALE].ArtCatID] _
        OPERATOREJOIN["INNER"]
'
'@DEFQUERYCOLUMN IDQUERY[IDQUERY] ESPR[[SUBQUERY1].CategID] VAR[TR1IDCAT]
'
'@DEFQUERYJOIN IDQUERY[IDQUERY] TABELLA[SUBQ(IDSUBQUERY2) AS [SUBQUERY2]] _
        TABELLAPADRE[[QUERY_PRINCIPALE]] _
        REL[[SUBQUERY2].FamID=[QUERY_PRINCIPALE].ArtFamID] _
        OPERATOREJOIN["INNER"]
'
'@DEFQUERYCOLUMN IDQUERY[IDQUERY] ESPR[[SUBQUERY2].FamID] VAR[TR2IDFAM]
'
RETURN

```

questo è il record-set restituito dalla subquery 1 inserita nella prima clausola JOIN:

	CategID
1	10
2	20
3	30
4	40
5	50

questo il record-set restituito dalla subquery 2 inserita nella seconda clausola JOIN:

	FamID
1	10
2	20
3	30
4	40
5	50
6	60
7	70
8	80
9	90
10	100
11	110
12	120
13	130
14	140
15	150
16	160
17	170
18	180
19	190
...	

e questo il record-set della subquery 3 inserita nella clausola FROM:

	ArtID	ArtDescrizione	TOT_VALORE_D	ArtDa
1	1	Penna a sfera colore nero	165.0000	2014
2	2	Penna a sfera colore blu	110.0000	2014
3	14	Post-it quadrati gialli	128.0000	2014
4	18	Tassello nylon 0.4mm	3000.0000	2014
5	19	Tassello nylon 0.6mm	3000.0000	2014
6	20	Tassello nylon 0.8mm	3000.0000	2014
7	41	Scrivania lung.110cm	270.0000	2014
8	42	Scrivania lung.100cm	240.0000	2014
9	43	Scrivania lung.90cm	210.0000	2014
10	44	Sedia rigida	150.0000	2014
11	45	Sedia morbida	175.0000	2014
12	46	Sedia morbida con braccioli	200.0000	2014
13	49	Armadio alto h.190cm	85.0000	2014
14	54	Lampada da scrivania bassa	175.0000	2014
15	55	Lampada da scrivania alta	120.0000	2014
16	56	Lampada da pavimento	135.0000	2014
17	61	Lampadine basso consumo	160.0000	2014

di seguito il codice SQL generato in esecuzione:

SQL

```
SELECT  QUERY_PRINCIPALE.ArtID AS [ArtID] ,
        QUERY_PRINCIPALE.ArtDescrizione AS [ArtDescrizione] ,
        QUERY_PRINCIPALE.TOT_VALORE_D AS [TOT_VALORE_D] ,
        SUBQUERY1.CategID AS [CategID] ,
        SUBQUERY2.FamID AS [FamID]
FROM    ( SELECT      Tabella7.ArtID AS [ArtID] ,
                      Tabella7.ArtDescrizione AS [ArtDescrizione] ,
                      SUM(Tabella7.ArtUnitaConf * Tabella7.ArtQuantitaConf
                          * Tabella7.ArtCostoUnit) AS [TOT_VALORE_D] ,
                      Tabella7.ArtDataUltAgg AS [ArtDataUltAgg] ,
                      Tabella7.ArtNote AS [ArtNote] ,
                      Tabella7.ArtCostoUnit AS [ArtCostoUnit] ,
                      Tabella7.ArtPerPrecAgg AS [ArtPerPrecAgg] ,
                      Tabella7.ArtUnitaConf AS [ArtUnitaConf] ,
                      Tabella7.ArtQuantitaConf AS [ArtQuantitaConf] ,
                      Tabella7.ArtQuantitaUnita AS [ArtQuantitaUnita] ,
                      Tabella7.ArtFLDisponibile AS [ArtFLDisponibile] ,
                      Tabella7.ArtCatID AS [ArtCatID] ,
                      Tabella7.ArtFamID AS [ArtFamID]
        FROM      BLDSVIL_Articoli AS [Tabella7]
        WHERE     ( (( Tabella7.ArtUnitaConf * Tabella7.ArtQuantitaConf
                        * Tabella7.ArtCostoUnit ) > 80) )
        GROUP BY  Tabella7.ArtDataUltAgg ,
                  Tabella7.ArtNote ,
                  Tabella7.ArtDescrizione ,
                  Tabella7.ArtCostoUnit ,
                  Tabella7.ArtPerPrecAgg ,
                  Tabella7.ArtUnitaConf ,
                  Tabella7.ArtQuantitaConf ,
                  Tabella7.ArtQuantitaUnita ,
                  Tabella7.ArtFLDisponibile ,
                  Tabella7.ArtID ,
                  Tabella7.ArtCatID ,
                  Tabella7.ArtFamID
        ) AS [QUERY_PRINCIPALE]
INNER JOIN ( SELECT Tabella5.CategID AS [CategID]
              FROM   BLDSVIL_Categorie AS [Tabella5]
            ) AS [SUBQUERY1] ON ( SUBQUERY1.CategID = QUERY_PRINCIPALE.ArtCatID )
INNER JOIN ( SELECT Tabella6.FamID AS [FamID]
              FROM   BLDSVIL_Famiglie AS [Tabella6]
            ) AS [SUBQUERY2] ON ( SUBQUERY2.FamID = QUERY_PRINCIPALE.ArtFamID )
```

⚠ E' necessario definire un ALIAS alla subquery e utilizzare il nome esteso del campo per definire la relazione tra la subquery e la tabella indicata in PADRE[].

BC

```
'@DEFQUERYJOIN IDQUERY[IDQUERY] TABELLA[SUBQ(IDSUBQUERY) AS [SUBQUERY1]]
TABELLAPADRE[BSVT003] _
REL[[SUBQUERY1].CategID=BSVT003.IDCAT] _
OPERATOREJOIN["INNER"]
```


SQL

```
INNER JOIN ( SELECT Tabella4.CategID AS [CategID]
FROM   BLDSVIL_Categorie AS [Tabella4]
) AS [SUBQUERY1] ON ( SUBQUERY1.CategID = BLDSVIL_Articoli.ArtCatID )
```

Subquery multidominio/multigruppo/multistazione all'interno della clausola FROM

L'ambiente BC implementa dei "meccanismi" trasversali nella gestione dei dati : gruppo, stazione e dominio. Questi concetti permettono di gestire all'interno di una unica tabella fisica del database, più partizioni di dati.

Normalmente i concetti sono trasparenti nel modo di programmare in quanto si opera su un unico gruppo, un'unica stazione e un unico dominio. E' possibile pilotare questi comportamenti mediante parametri specifici della DEFQUERY :

- MULTIGRUPPO
- MULTISTAZIONE
- MULTIDOMINIO

Sotto viene riportato il comportamento d'ambiente per gestire le combinazioni di query e sotto query usate nelle from, multi gruppo e multi dominio.

Il comportamento descritto è relativo a come la sottoquery è vista come tabella dalla query padre.

Multi gruppo:

	Query TABELLA multi gruppo	Query TABELLA NON multi gruppo
Query padre multi gruppo	Omissione del campo gruppo nella clausola ON E' il programma che deve operare il filtro	Indicazione del campo gruppo nella clausola ON
Query padre NON multi gruppo	Omissione del campo gruppo nella clausola ON E' il programma che deve operare il filtro	Indicazione del campo gruppo nella clausola ON

Multistazione:

	Query TABELLA multistazione	Query TABELLA NON multistazione
Query padre multistazione	Omissione del campo stazione nella clausola ON E' il programma che deve operare il filtro	Indicazione del campo stazione nella clausola ON
Query padre NON multistazione	Omissione del campo stazione nella clausola ON E' il programma che deve operare il filtro	Indicazione del campo stazione nella clausola ON

Multi dominio:

	Query TABELLA multi dominio	Query TABELLA NON multi dominio
Query padre multi dominio	Omissione del campo dominio nella clausola ON E' il programma che deve operare il filtro	Indicazione del campo dominio nella clausola ON
Query padre NON multi dominio	Omissione del campo dominio nella clausola ON E' il programma che deve operare il filtro	Indicazione del campo dominio nella clausola ON

Le common table expressions (CTEs) in BC

Nel tempo, il Linguaggio BC si evolve aggiornandosi di nuovi costrutti sempre più simili ai linguaggi di programmazione esterni a Sistemi. Questo permette di essere sempre più flessibile nella progettazione di nuovi programmi.

Ne consegue che man mano si vadano a rendere obsolete vecchie logiche ormai non più supportate dai nuovi standard di programmazione. Questa scheda ha il compito di riassumere i vecchi costrutti utilizzati nei vecchi sorgenti.

Le common table expressions (CTEs) in BC - versione 1

Cos'è una common table expression

La common table expression (CTE) è uno strumento di programmazione SQL che permette di semplificare query complesse.

Esistono 2 tipologie di CTE:

- Non ricorsive, equivalenti alla definizione di una o più viste temporanee, valide per il periodo di esecuzione della query;
- Ricorsive, che permettono di interrogare strutture gerarchiche complesse.

Struttura CTE non ricorsiva

Sintassi SQL

La CTE si specifica usando la keyword WITH:

```
WITH <NomeCTE> ( <NomiColonna 1, NomeColonna 2, NomeColonna n> ) AS
(
    <QUERY DI DEFINIZIONE CTE>
),
....

<NomeCTE n> ( <NomiColonna 1, NomeColonna 2, NomeColonna n > ) AS
(
    <QUERY DI DEFINIZIONE CTE N>
)
SELECT <QueryPrincipale>
```

- WITH è la keyword che indica la presenza di una o più CTE (ognuna separata da virgole);
- Ad ogni CTE si associa un nome (<NomeCTE >) che verrà usato per riferirsi nel codice SQL come se fosse una vista;
- Tra parentesi tonde vi è la definizione della query di definizione del recordset della CTE.

Caratteristiche principali di una CTE non ricorsiva

Per utilizzare una CTE non ricorsiva è bene ricordare alcune regole base:

- In una CTE non ricorsiva è possibile definire più query CTE.
- Una CTE può far riferimento a se stessa e alle CTE definite in precedenza nella stessa clausola WITH. Il riferimento in avanti non è consentito.
- Non è consentito specificare più di una clausola WITH in una CTE. Se ad esempio una definizione *<QUERY DI DEFINIZIONE CTE>* include una subquery, tale subquery non può includere una clausola WITH annidata che definisce un'altra CTE.
- La clausola ORDER BY non può essere usata, tranne quando si specifica una clausola TOP.

Sintassi BC

La scrittura in BC di una CTE avviene secondo il costrutto di una subquery con l'aggiunta di un nuovo parametro sulla specifica '@DEFQUERY' che identifica il nome della CTE.

Esempio

SQL

```
/* Esempio di una CTE non ricorsiva
   che prevede la definizione di due CTE:
   - la prima 'Uffici'
   - la seconda 'Impiegati' in relazione con
     la prima
*/
WITH
/* Prima CTE */
Uffici (DesUfficioU, IdUfficioU)
AS (SELECT Descrizione AS [DesUfficioU],
        IDUfficio AS [IdUfficioU]
    FROM BFT_Uffici AS [BFT_Uffici]),
/* Seconda CTE */
Impiegati (DesImpiegatoI, DesUfficioI, IdUfficioI)
AS (SELECT LTRIM(RTRIM(BFT_Impiegati.Nome)) + ' ' + LTRIM(RTRIM(BFT_Impiegati.Cognome)) AS [DesImpiegatoI],
        LTRIM(RTRIM(Uffici.DesUfficioU)) AS [DesUfficioI],
        BFT_Impiegati.IDUfficio AS [IdUfficioI]
    FROM BFT_Impiegati AS [BFT_Impiegati]
    /* relazione con la prima CTE */
    INNER JOIN Uffici AS [Uffici]
        ON (Uffici.IdUfficioU = BFT_Impiegati.IDUfficio)
    WHERE (
        BFT_Impiegati.IDResponsabile > 0
        AND BFT_Impiegati.IDImpiegato > 0
    ))
/* Query principale su CTE */
SELECT DesImpiegatoI AS [DescrizioneImpiegato],
        DesUfficioI AS [DescrizioneUfficio]
FROM Impiegati AS [Impiegati]
ORDER BY [DescrizioneImpiegato] ASC;
```

BC

```

' CTE multipla con seconda CTE dipendete dalla prima

DIM IdQryPrincipale[INT]=0
DIM IdQryCTE1[INT]=0
DIM IdQryCTE2[INT]=0
DIM DesImpiegato[STRING(255)]=""
DIM DesUfficio[STRING(255)]=""

' Definizione della query principale
'@DEFQUERY IDQUERY[IdQryPrincipale]

' Definizione della prima query CTE
'@DEFQUERYCTE IDQUERY[IdQryCTE1] IDQUERYPADRE[IdQryPrincipale] ALIAS["Uffici"]

'@DEFQUERYFROM IDQUERY[IdQryCTE1] TABELLA[BFT_UFFICI]
'@DEFQUERYSOURCE IDQUERY[IdQryCTE1] ESPR[BFT_UFFICI.Descrizione AS [DesUfficioU]]
'@DEFQUERYSOURCE IDQUERY[IdQryCTE1] ESPR[BFT_UFFICI.IDUfficio AS [IdUfficioU]]

' Definizione della seconda query CTE
'@DEFQUERYCTE IDQUERY[IdQryCTE2] IDQUERYPADRE[IdQryPrincipale] ALIAS["Impiegati"]

'@DEFQUERYFROM IDQUERY[IdQryCTE2] TABELLA[BFT_IMPIEGATI]

' Relazione della seconda CTE con la prima
'@DEFQUERYJOIN IDQUERY[IdQryCTE2] TABELLA[[Uffici]] TABELLAPADRE[BFT_IMPIEGATI] _
REL[[Uffici].IdUfficioU = BFT_IMPIEGATI.IDufficio] OPERATOREJOIN["JOIN"]

'@DEFQUERYSOURCE IDQUERY[IdQryCTE2] _
ESPR[FNTRSD$(BFT_IMPIEGATI.Nome) + "
" + FNTRSD$(BFT_IMPIEGATI.Cognome) AS [DesImpiegatoI] FMT[TESTO[255]]]

'@DEFQUERYSOURCE IDQUERY[IdQryCTE2] _
ESPR[FNTRSD$([Uffici].DesUfficioU) AS [DesUfficioI] FMT[TESTO[255]]]

'@DEFQUERYSOURCE IDQUERY[IdQryCTE2] ESPR[BFT_IMPIEGATI.IDufficio AS [IdUfficioI]]

' Prendo solo gli impiegati
'@DEFQUERYWHERE IDQUERY[IdQryCTE2] ESPR[BFT_IMPIEGATI.IDResponsabile>0 AND BFT_IMPIEGATI.IDImpiegato>0]

' Definizione della query principale in relazione con la CTE
'@DEFQUERYFROM IDQUERY[IdQryPrincipale] TABELLA[[Impiegati]]

'@DEFQUERYSOURCE IDQUERY[IdQryPrincipale] _
ESPR[[Impiegati].DesImpiegatoI AS [DescrizioneImpiegato]] VAR[DesImpiegato] _
ESPR[[Impiegati].DesUfficioI AS [DescrizioneUfficio]] VAR[DesUfficio]

' Ordino
'@DEFQUERYORDERBY IDQUERY[IdQryPrincipale] ESPR[[DescrizioneImpiegato]]

```

Struttura CTE ricorsiva

Sintassi SQL

La sintassi SQL di una CTE ricorsiva estende quella non ricorsiva:

```

WITH <NomeCTE> ( <NomiColonna 1, NomeColonna 2, NomeColonna n> ) AS
(
    <QUERY DI DEFINIZIONE CTE - PARTE FISSA>
    UNION ALL
    <QUERY DI DEFINIZIONE CTE - PARTE RICORSIVA>
)

SELECT <QueryPrincipale>

```

- La query di definizione prevede due parti: una fissa e d una ricorsiva. Le due parti sono due query (o subquery).
- Le due parti sono separate dalla clausola UNION ALL.
- La parte ricorsiva usa il nome della CTE per innescare la ricorsione.

Caratteristiche principali di una CTE ricorsiva

Per utilizzare una CTE ricorsiva è bene ricordare alcune regole base:

- Il numero delle colonne nei membri ricorsivi e non ricorsivi deve essere lo stesso.
- Il tipo di dati di una colonna nel membro ricorsivo deve essere lo stesso del tipo di dati della colonna corrispondente nel membro non ricorsivo.
- La clausola FROM di un membro ricorsivo deve fare riferimento solo una volta all'espressione <NomeCTE>.
- Gli elementi seguenti non sono consentiti nella definizione <QUERY DI DEFINIZIONE CTE - PARTE RICORSIVA>:
 - SELECT DISTINCT
 - GROUP BY
 - HAVING
 - Aggregazione scalare
 - LEFT, RIGHT, OUTER JOIN (**è consentito solo INNER JOIN**)
 - Sottoquery
- Una CTE ricorsiva formulata in modo non corretto può provocare un ciclo infinito. Ad esempio, se la definizione della query del membro ricorsivo restituisce gli stessi valori per entrambe le colonne padre e figlio, si crea un ciclo infinito.
- Non è possibile utilizzare una vista che contiene una CTE ricorsiva per aggiornare i dati.

Funzionamento CTE ricorsiva

I passi logici eseguiti sono:

1. Esecuzione della parte fissa e produzione del primo recordset
2. Esecuzione della parte ricorsiva che deve riferirsi alla CTE stessa. La parte ricorsiva esamina il recordset prodotto dalla parte fissa o dalla precedente ricorsione e produce un nuovo recordset
3. Se la parte ricorsiva ha prodotto un recordset non vuoto viene eseguito nuovamente il passo 2.

Il risultato finale è la somma di tutti i recordset prodotti:

- dalla parte fissa
- dalle n ricorsioni

Nell'esempio avremo quindi:

Passo	Recordset esaminato dalla parte ricorsiva	Recordset prodotto dalla parte fissa/ ricorsiva	Recordset finale
0		n 1	n 1
1	n 1	n 2	n 1 2
2	n 2	n 3	n 1 2 3
...			
50	n 50	n	n 1 2 3 ... 50

Sintassi BC

La scrittura in BC di una CTE ricorsiva si caratterizza dalla presenza nella definizione del sotto parametro PARTE del parametro CTE che specifica quale delle due è la parte ricorsiva.

Esempio

SQL
<pre>/* Esempio di una CTE ricorsiva che prevede la definizione di due CTE: - la prima definizione è la parte fissa - la seconda la seconda è la parte ricorsiva */ WITH Report (DesImpiegato, Posizione, IDImpiegato, Livello, Sort) AS (/* Parte fissa */ SELECT CAST(LTRIM(RTRIM(BFT_Impiegati.Nome)) + ' ' + LTRIM(RTRIM(BFT_Impiegati.Cognome)) AS VARCHAR(255)) AS [DesImpiegato], BFT_Impiegati.Posizione AS [Posizione], BFT_Impiegati.IDImpiegato AS [IDImpiegato], 1 AS [Livello], CAST(LTRIM(RTRIM(BFT_Impiegati.Nome)) + ' ' + LTRIM(RTRIM(BFT_Impiegati.Cognome)) AS VARCHAR(255)) AS [Sort] FROM BFT_Impiegati AS [BFT_Impiegati] WHERE (BFT_Impiegati.IDResponsabile = 0) /* Operatore */ UNION ALL /* Parte ricorsiva */ SELECT CAST(REPLICATE(' ', Report_1.Livello) + LTRIM(RTRIM(BFT_Impiegati.Nome)) + ' ' + LTRIM(RTRIM(BFT_Impiegati.Cognome)) AS VARCHAR(255)) AS [DesImpiegato], BFT_Impiegati.Posizione AS [Posizione], BFT_Impiegati.IDImpiegato AS [IDImpiegato], Report_1.Livello + 1 AS [Livello], CAST(RTRIM(Report_1.Sort) + ' ' + LTRIM(RTRIM(BFT_Impiegati.Nome)) + ' ' + LTRIM(RTRIM(BFT_Impiegati.Cognome)) AS VARCHAR(255)) AS [Sort] FROM BFT_Impiegati AS [BFT_Impiegati] /* Indicazione della CTE che fa scatenare la ricorsione*/ INNER JOIN Report AS [Report_1] ON (Report_1.IDImpiegato = BFT_Impiegati.IDResponsabile)) /* Query principale*/ SELECT IDImpiegato AS [IDImpiegato], DesImpiegato AS [DesImpiegato], Posizione AS [Posizione], Livello AS [Livello] FROM Report AS [Report] ORDER BY Sort ASC;</pre>
BC

```

' CTE ricorsiva senza JOIN

DIM IdQryPrincipale[INT]= 0
DIM IdQryCTE1[INT]=0
DIM IdQryCTE2[INT]=0
DIM IDImpiegato[INT]=0
DIM DesImpiegato[STRING(255)]=""
DIM Posizione[STRING(50)]=""
DIM Livello[INT]=0

' Definizione della query principale
'@DEFQUERY IDQUERY[IdQryPrincipale]

' Definizione della query CTE1 - parte fissa
'@DEFQUERYCTE IDQUERY[IdQryCTE1] IDQUERYPADRE[IdQryPrincipale] ALIAS["Report"] PARTE[#FISSA]

'@DEFQUERYFROM IDQUERY[IdQryCTE1] TABELLA[BFT_IMPIEGATI]

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IdQryCTE1] _
                        ESPR[FNTRSD$(BFT_IMPIEGATI.Nome) + "
" + FNTRSD$(BFT_IMPIEGATI.Cognome) AS [DesImpiegato] FMT[TESTO[255]]]

'@DEFQUERYCOLUMN IDQUERY[IdQryCTE1] ] _
                        ESPR[BFT_IMPIEGATI.Posizione AS [Posizione]] _
                        ESPR[BFT_IMPIEGATI.IDImpiegato AS [IDImpiegato]]

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IdQryCTE1] _
                        ESPR[1 AS [Livello] FMT[NUMERICO[2]]]

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IdQryCTE1] ] _
                        ESPR[FNTRSD$(BFT_IMPIEGATI.Nome) + "
" + FNTRSD$(BFT_IMPIEGATI.Cognome) AS [Sort] FMT[TESTO[255]]]

'@DEFQUERYWHERE IDQUERY[IdQryCTE1] ESPR[BFT_IMPIEGATI.IDResponsabile=0]

' Definizione della query CTE2 - parte ricorsiva
'@DEFQUERYCTE IDQUERY[IdQryCTE2] IDQUERYPADRE[IdQryPrincipale] ALIAS["Report"] PARTE[#RICORSIVA]

'@DEFQUERYFROM IDQUERY[IdQryCTE2] TABELLA[BFT_IMPIEGATI]

' Questa INNER JOIN con la CTE della parte fissa innesca la ricorsione
'@DEFQUERYJOIN IDQUERY[IdQryCTE2] TABELLA[[Report]] TABELLAPADRE[BFT_IMPIEGATI] _
                REL[[Report].IDImpiegato = BFT_IMPIEGATI.IDResponsabile] OPERATOREJOIN["JOIN"]

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IdQryCTE2] _
                        ESPR[STRING$([Report].Livello, "|") + FNTRSD$(BFT_IMPIEGATI.Nome) + "
" + FNTRSD$(BFT_IMPIEGATI.Cognome) AS [DesImpiegato] FMT[TESTO[255]]]

'@DEFQUERYCOLUMN IDQUERY[IdQryCTE2] _
                        ESPR[BFT_IMPIEGATI.Posizione AS [Posizione]] _
                        ESPR[BFT_IMPIEGATI.IDImpiegato AS [IDImpiegato]]

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IdQryCTE2] _
                        ESPR[[Report].Livello + 1 AS [Livello] FMT[NUMERICO[2]]]

'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IdQryCTE2] _
                        ESPR[FNTR$([Report].Sort)+"| " + FNTRSD$(BFT_IMPIEGATI.Nome) + "
" + FNTRSD$(BFT_IMPIEGATI.Cognome) AS [Sort] FMT[TESTO[255]]]

' Definizione della query principale in relazione con la CTE
'@DEFQUERYFROM IDQUERY[IdQryPrincipale] TABELLA[[Report]]

'@DEFQUERYCOLUMN IDQUERY[IdQryPrincipale] _
                        ESPR[[Report].IDImpiegato AS [IDImpiegato]] VAR[IDImpiegato] _
                        ESPR[[Report].DesImpiegato AS [DesImpiegato]] VAR[DesImpiegato] _
                        ESPR[[Report].Posizione AS [Posizione]] VAR[Posizione] _
                        ESPR[[Report].Livello AS [Livello]] VAR[Livello]

'@DEFQUERYCOLUMN IDQUERY[IdQryPrincipale] _
                        ESPR[[Report].Sort AS [Sort]] NOOUTPUT[1]

'@DEFQUERYORDERBY IDQUERY[IdQryPrincipale] ESPR[[Sort]]

```

Risultato:

Results		Messages		
	IDImpiegato	DesImpiegato	Posizione	Livello
1	1	Mario Rossi	Amministratore delegato	1
2	273	Paolo Verdi	Vicepresidente delle vendite	2
3	16	Davide Fucsia	Direttore Marketing	3
4	23	Maria Rosa	Specialista Marketing	4
5	274	Giovanni Bianchi	Direttore vendite	3
6	276	Linda Blu	Rappresentante commerciale	4
7	275	Michele Gialli	Rappresentante commerciale	4
8	285	Sara Viola	Direttore vendite estero	3
9	286	Laura Arancione	Rappresentante commerciale	4

Utilizzo CTE con clausola UNION

In BC è necessario seguire una sintassi ben precisa per poter gestire le CTE con più query in union. La sinstassi SQL prevede che tutte le CTE vengano definite ad inizio query, in modo che siano tutte disponibili per l'utilizzo all'interno dello statement.

```
WITH <NomeCTE> ( <NomiColonna 1, NomeColonna 2, NomeColonna n> ) AS
(
    <QUERY DI DEFINIZIONE CTE>
),
....

<NomeCTE n> ( <NomiColonna 1, NomeColonna 2, NomeColonna n > ) AS
(
    <QUERY DI DEFINIZIONE CTE N>
)
SELECT <QueryPrincipale1>
UNION [ALL]
SELECT <QueryPrincipale2>
UNION [ALL]
SELECT <QueryPrincipaleN>
```

In BC è necessario definire prima tutti gli identificativi delle query e poi esplicitare nella UNION prima le query con CTE e poi le altre.

```
SQL
```



```
/* Esempio di due CTE presenti in entrambe le
   query messe in UNION.*/

/* Prima CTE: estrae tutti gli uffici con più di un impiegato*/
WITH Impiegati (IdUfficio, ContaUfficio)
AS (SELECT IDUfficio AS [IdUfficio],
          COUNT(IDImpiegato) AS [ContaUfficio]
   FROM BFT_Impiegati AS [BFT_Impiegati]
   GROUP BY IdUfficio
   HAVING (COUNT((IDImpiegato)) > 1)),

/* Seconda CTE: estrarre tutti gli uffici con un solo impiegato*/
Impiegati2 (IdUfficio, ContaUfficio)
AS (SELECT IDUfficio AS [IdUfficio],
          COUNT(IDImpiegato) AS [ContaUfficio]
   FROM BFT_Impiegati AS [BFT_Impiegati]
   GROUP BY IdUfficio
   HAVING (COUNT((IDImpiegato)) = 1))

/* Prima query con la prima CTE*/
SELECT BFT_Uffici.Descrizione AS [DescrizioneUfficio],
       Impiegati.ContaUfficio AS [NumeroImpiegati]
FROM BFT_Uffici AS [BFT_Uffici]
     INNER JOIN Impiegati AS [Impiegati]
       ON (Impiegati.IdUfficio = BFT_Uffici.IDUfficio)

/* Clausola di UNION*/
UNION

/* Seconda query con la seconda CTE*/
SELECT BFT_Uffici.Descrizione AS [DescrizioneUfficio],
       Impiegati2.ContaUfficio AS [NumeroImpiegati]
FROM BFT_Uffici AS [BFT_Uffici]
     INNER JOIN Impiegati2 AS [Impiegati2]
       ON (Impiegati2.IdUfficio = BFT_Uffici.IDUfficio)

/* Clausola di UNION*/
UNION

/* Terza query senza CTE*/
SELECT 'Ufficio Fittizio' AS [DescrizioneUfficio],
       12 AS [NumeroImpiegati]
FROM BFT_Uffici AS [BFT_Uffici]

/* Ordinamento sulla UNION*/
ORDER BY [DescrizioneUfficio] ASC;
```

BC

```

' Definizione degli identificativi delle query e delle CTE
DIM IdQryUnion[INT]= 0
DIM IdQryPrincipale[INT]=0
DIM IdQryPrincipale2[INT]=0
DIM IdQryPrincipale3[INT]=0
DIM IdQryCTE1[INT]=0
DIM IdQryCTE2[INT]=0
DIM Livello[INT]=0
DIM DesUfficio[STRING(255)]=""
DIM NumImpiegati[INT]=0

'@DEFQUERY IDQUERY[IdQryUnion]
'@DEFQUERY IDQUERY[IdQryPrincipale]
'@DEFQUERY IDQUERY[IdQryPrincipale2]
'@DEFQUERY IDQUERY[IdQryPrincipale3]

'@DEFQUERYCTE IDQUERY[IdQryCTE1] IDQUERYPADRE[IdQryPrincipale] ALIAS["Impiegati"]
'@DEFQUERYCTE IDQUERY[IdQryCTE2] IDQUERYPADRE[IdQryPrincipale2] ALIAS["Impiegati2"]

' -----
' Definizione della prima CTE

'@DEFQUERYFROM IDQUERY[IdQryCTE1] TABELLA[BFT_IMPIEGATI]

'@DEFQUERYCOLUMN IDQUERY[IdQryCTE1] _
    ESPR[GROUPBY(BFT_IMPIEGATI.IDUfficio) AS [IdUfficio]] _
    ESPR[COUNT(BFT_IMPIEGATI.IDImpiegato) AS [ContaUfficio]]

' Uffici con più di un impiegato
'@DEFQUERYWHERE IDQUERY[IdQryCTE1] ESPRHAVING[([ContaUfficio]>1)]

' Definizione della query principale in relazione con la CTE
'@DEFQUERYFROM IDQUERY[IdQryPrincipale] TABELLA[BFT_UFFICI]

'@DEFQUERYJOIN IDQUERY[IdQryPrincipale] TABELLA[[Impiegati]] TABELLAPADRE[BFT_UFFICI] _
    REL[[Impiegati].IdUfficio = BFT_UFFICI.IDUfficio] OPERATOREJOIN["INNER"]

'@DEFQUERYCOLUMN IDQUERY[IdQryPrincipale] _
    ESPR[BFT_UFFICI.Descrizione AS [DescrizioneUfficio]] VAR[DesUfficio] _
    ESPR[[Impiegati].ContaUfficio AS [NumeroImpiegati]] VAR[NumImpiegati]

' -----
' Definizione della seconda CTE

'@DEFQUERYFROM IDQUERY[IdQryCTE2] TABELLA[BFT_IMPIEGATI]

'@DEFQUERYCOLUMN IDQUERY[IdQryCTE2] _
    ESPR[GROUPBY(BFT_IMPIEGATI.IDUfficio) AS [IdUfficio]] _
    ESPR[COUNT(BFT_IMPIEGATI.IDImpiegato) AS [ContaUfficio]]

' Uffici con un solo impiegato
'@DEFQUERYWHERE IDQUERY[IdQryCTE2] ESPRHAVING[([ContaUfficio]=1)]

' Definizione della query principale in relazione con la CTE
'@DEFQUERYFROM IDQUERY[IdQryPrincipale2] TABELLA[BFT_UFFICI]

'@DEFQUERYJOIN IDQUERY[IdQryPrincipale2] TABELLA[[Impiegati2]] TABELLAPADRE[BFT_UFFICI] _
    REL[[Impiegati2].IdUfficio = BFT_UFFICI.IDUfficio] OPERATOREJOIN["INNER"]

'@DEFQUERYCOLUMN IDQUERY[IdQryPrincipale2] _
    ESPR[BFT_UFFICI.Descrizione AS [DescrizioneUfficio]] VAR[DesUfficio] _
    ESPR[[Impiegati2].ContaUfficio AS [NumeroImpiegati]] VAR[NumImpiegati]

' -----
' Creo la query con record fittizio

'@DEFQUERYFROM IDQUERY[IdQryPrincipale3] TABELLA[BFT_UFFICI]
'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IdQryPrincipale3] ESPR["Ufficio
Fittizio" AS [DescrizioneUfficio] VAR[DesUfficio]]
'@DEFQUERYCOMPUTEDCOLUMN IDQUERY[IdQryPrincipale3] ESPR[12 AS [NumeroImpiegati] VAR[NumImpiegati]]
'
' -----
' Definisco la Union
'@DEFQUERYUNION IDQUERYUNION[IdQryUnion] _
    QUERY[IdQryPrincipale] _
    QUERY[IdQryPrincipale2] _

```

```
QUERY[IdQryPrincipale3]
,
'@DEFQUERYORDERBY IDQUERY[IdQryUnion] ESPR[[DescrizioneUfficio]]
```

Risultato:

	DescrizioneUfficio	NumeroImpiegati
1	Direzione	1
2	Ufficio Commerciale	6
3	Ufficio Fittizio	12
4	Ufficio Marketing	2